



Ministère de l'Enseignement Supérieur et de la
Recherche Scientifique
Direction Générale des Etudes Technologiques
Institut Supérieur des Etudes Technologiques de
Bizerte
Département Technologies de l'Informatique



DEP.	TI
AN.	2026
REF.	SEM2612

Rapport de
PROJET DE FIN D'ÉTUDES

En vue de l'obtention de :
Licence appliquée en technologie d'information

**Modélisation et implémentation d'une application web pour la
gestion d'une maison intelligente**

Élaboré par :
Mohamed Hedi Loussaief & Mohamed Amine Bennour

Encadré par :

Mr Bilel Zemzem (ISET)

Mr Hassen Kaabouri (ENTREPRISE)

Effectué à :

Entreprise : Intuitive Group

- **Adresse :** 46, Av Taieb Mhiri, La Marsa, Tunis, 2070, TN
- **Tél :** 22 299 944
- **Mail :** contact@intuitiv.fr

Année universitaire : 2025 / 2026

Dédicace

À ceux qui se sont sacrifiés pour que je puisse avancer, à ceux dont l'amour, le soutien et les encouragements m'ont accompagné tout au long de ce parcours.

À ceux qui se sont courbés pour que je puisse me tenir debout,

À mes chers parents

À ma mère Samira Ben Slema

Lorsque mon nom sera prononcé aujourd'hui, c'est vers toi que mes pensées se tourneront en premier. Tu as toujours été présente dans l'ombre, par tes prières, tes sacrifices et ton amour inconditionnel. Tu as cru en moi dans les moments les plus difficiles et tu m'as donné la force de poursuivre mes rêves. Ce travail est le fruit de tes efforts autant que des miens. Je te remercie du fond du cœur pour tout ce que tu as fait pour moi.

À mon père Fathi Loussaief

Toi qui as travaillé sans relâche pour m'offrir les meilleures conditions possibles et un avenir prometteur. Par ton courage, ton dévouement et tes sacrifices, tu as été un exemple et une source d'inspiration. Je suis fier de t'offrir aujourd'hui le résultat de toutes ces années d'efforts.

À ma chère sœur Imen Loussaief

Pour tous les moments partagés, les encouragements, le soutien et la complicité qui nous unissent depuis toujours. Ta présence à mes côtés a été une véritable source de réconfort tout au long de ce parcours.

À mes chers amis

Merci pour votre amitié sincère, votre soutien constant et tous les moments de partage qui ont rendu cette aventure plus agréable et plus enrichissante.

À vous tous, je dédie ce modeste travail en témoignage de ma profonde gratitude, de mon respect et de mon affection.

Mohamed Hedi Loussaief

Dédicace

À ceux qui ont toujours cru en moi et m'ont soutenu tout au long de mon parcours.

À mes chers parents

À ma mère Lamia Rebaya

Pour ton amour inconditionnel, tes prières, ton soutien et tous les sacrifices que tu as consentis pour mon bonheur et ma réussite. Je te remercie du fond du cœur et te dédie ce travail avec toute mon affection.

À mon père Hamed Bennour

Pour ton dévouement, tes encouragements et tous les efforts que tu as fournis afin de m'offrir les meilleures conditions de réussite. Je t'adresse ma profonde reconnaissance et mon respect.

À ma chère sœur Tasnim Bennour

Pour ton soutien, ton affection et les précieux moments partagés. Merci d'avoir toujours été présente à mes côtés.

À mes amis

Merci pour votre présence, vos encouragements et votre confiance qui m'ont accompagné tout au long de cette aventure.

À vous tous, je dédie ce modeste travail en témoignage de ma gratitude et de mon profond respect.

Mohamed Amine Bennour

Remerciements

Avant tout, nous remercions Dieu le Tout-Puissant de nous avoir accordé la santé, la volonté, le courage et la patience nécessaires pour mener à bien ce projet de fin d'études dans les meilleures conditions.

Nous tenons à exprimer notre profonde gratitude à notre encadrant académique, Monsieur **Bilel Zemzem**, pour son accompagnement, sa disponibilité, ses précieux conseils ainsi que ses orientations tout au long de la réalisation de ce travail. Son soutien et ses remarques constructives nous ont permis d'améliorer continuellement la qualité de notre projet.

Nous adressons également nos sincères remerciements à notre encadrant professionnel, Monsieur **Hassen Kaabouri**, au sein de l'entreprise **Intuitive Group**, pour son accueil chaleureux, sa confiance, son encadrement et son aide précieuse durant toute la période du stage. Nous avons énormément appris grâce à son expérience professionnelle, ses conseils techniques et son suivi permanent.

Nos remerciements s'adressent aussi à toute l'équipe d'**Intuitive Group** pour leur disponibilité, leur collaboration et l'environnement professionnel motivant dans lequel nous avons eu l'opportunité d'évoluer durant ce stage.

Nous exprimons également notre reconnaissance envers tous les enseignants de l'Institut Supérieur des Études Technologiques de Bizerte qui ont contribué à notre formation académique et au développement de nos compétences tout au long de notre parcours universitaire.

Nous remercions vivement les membres du jury qui nous font l'honneur d'évaluer ce travail et d'assister à notre soutenance. Nous leur sommes reconnaissants pour l'intérêt qu'ils portent à notre projet.

Enfin, nous adressons une pensée particulière à nos familles et à toutes les personnes qui nous ont soutenus moralement et encouragés durant cette expérience académique et professionnelle.

Merci à tous.

Table des matières

Introduction Générale	1
1 Cadre Général du Projet	3
Introduction	3
1.1 Présentation de l'organisme d'accueil	3
1.2 Étude de l'existant	4
1.2.1 Description des solutions existantes	4
1.2.2 Critique de l'existant	4
1.2.3 Solution proposée	5
1.3 Méthodologie utilisée	5
1.3.1 Framework Scrum	6
1.3.2 Langages de modélisation	6
1.3.3 SysML	8
Conclusion	9
2 Analyse et Spécification des Besoins	10
Introduction	10
2.1 Spécification des besoins	10
2.1.1 Diagramme des exigences	10
2.1.2 Exigences non fonctionnelles	11
2.2 Modélisation des besoins	12
2.2.1 Identification des acteurs	12
2.2.2 Diagramme de cas d'utilisation global	13
2.2.3 Diagramme de classes global	14
2.3 Pilotage du projet avec Scrum	14
2.3.1 Équipe et rôles	15
2.3.2 Le Backlog du produit	15
2.3.3 Planification des sprints	17
2.4 Environnement de travail	18
2.4.1 Architecture IoT du système	18
2.4.2 Environnement matériel	20
2.4.3 Environnement logiciel	24
2.4.4 Outils de développement	26
Conclusion	28

3	Sprint 1 : Découverte et accès à la plateforme	29
	Introduction	29
3.1	Sprint Goal	29
3.2	Sprint Backlog	30
3.3	Analyse	31
3.3.1	Diagramme de cas d'utilisation du Sprint 1	31
3.3.2	Description textuelle	31
3.4	Conception : Diagramme de classes du Sprint 1	32
3.5	Réalisation et tests	34
3.5.1	Planification des sprints	34
3.5.2	Préparation de l'environnement de développement	35
3.5.3	Passage du diagramme de classes vers la base de données MongoDB	37
3.5.4	Architecture de sécurité et gestion de l'authentification	40
3.5.5	Architecture du système	41
3.5.6	Présentation des Interfaces réalisées	42
	Conclusion	48
4	Sprint 2 : Gestion des équipements IoT	49
	Introduction	49
4.1	Sprint Goal	49
4.2	Sprint Backlog	49
4.3	Analyse	50
4.3.1	Diagramme de cas d'utilisation	50
4.3.2	Description textuelle des cas d'utilisation	51
4.4	Conception	54
4.4.1	Diagramme de blocs du système Smart House	54
4.4.2	Diagramme de classes	55
4.4.3	Diagramme de blocs interne	56
4.4.4	Diagramme de définition du bloc cam	57
4.4.5	Diagramme d'activité du système IoT	58
4.5	Réalisation	60
4.5.1	Choix de la carte électronique	60
4.5.2	Câblage et connexions des composants	61
4.5.3	Implémentation du système embarqué	62
4.5.4	Communication MQTT et transmission des données	63
4.5.5	Intégration du module ESP32-CAM	63
4.6	Test	64
4.6.1	Test du câblage et des capteurs	64
4.6.2	Test de transmission des données MQTT	67

4.6.3	Test du module ESP32-CAM	68
4.6.4	Validation du fonctionnement global du système	69
4.7	Interfaces réalisées	70
	Conclusion	71
5	Sprint 3 : Notifications et Historique	72
	Introduction	72
5.1	Sprint Goal	72
5.2	Sprint Backlog	72
5.3	Analyse	73
5.3.1	Diagramme de cas d'utilisation	73
5.3.2	Description textuelle des cas d'utilisation	75
5.3.3	Diagramme de séquence système	76
5.4	Conception	78
5.4.1	Diagramme de classes	78
5.4.2	Diagramme de déploiement	79
5.5	Présentation des interfaces réalisées	81
	Conclusion	87
6	Sprint 4 : Profil et abonnement	88
	Introduction	88
6.1	Sprint Goal	88
6.2	Sprint Backlog	88
6.3	Analyse	89
6.3.1	Diagramme de cas d'utilisation du Sprint 4	89
6.3.2	Description textuelle des cas d'utilisation	90
6.3.3	Diagramme de séquence du Sprint 4	94
6.4	Conception	94
6.5	Diagramme de classes Sprint 4	95
6.5.1	Diagramme d'activité — Gestion des abonnements	96
6.5.2	Diagramme d'activité — Soumettre une réclamation	98
6.6	Présentation des interfaces réalisées	99
	Conclusion	102
7	Sprint 5 : Administration	103
	Introduction	103
7.1	Sprint Goal	103
7.2	Sprint Backlog	103
7.3	Analyse	104
7.3.1	Diagramme de cas d'utilisation	104

7.3.2	Description textuelle des cas d'utilisation	105
7.4	Conception	109
7.4.1	Diagramme de classes du Sprint 5	109
7.4.2	Diagramme d'activité	111
7.5	Réalisation et tests	113
7.5.1	Interface de gestion des abonnements	113
7.5.2	Interface de gestion des réclamations	114
7.5.3	Interface de consultation des capteurs	114
7.5.4	Interface de gestion des utilisateurs	115
7.6	Tests fonctionnels	116
	Conclusion	117
	Conclusion générale	118
	Bibliographies/Webographies	119

Table des figures

1.1	Cycle de vie du framework Scrum	6
1.2	Logo UML	7
1.3	Logo SysML	8
2.1	Diagramme des exigences du système Smart House	11
2.2	Diagramme de cas d'utilisation du système	13
2.3	Diagramme de classes	14
2.4	Présentation de l'équipe SCRUM	15
2.5	Diagramme de Gantt des sprints du projet	17
2.6	Architecture IoT du système Smart House	19
2.7	La carte ESP32-WROOM-32 CP2102	21
2.8	Capteur DHT22	22
2.9	Capteur MQ2	22
2.10	Capteur de flamme	23
2.11	Capteur PIR	23
2.12	ESP32-CAM OV2640	23
2.13	ESP32-CAM-MB	24
2.14	Logo Angular	24
2.15	Logo NestJS	25
2.16	Logo C++	25
2.17	Logo MongoDB	26
2.18	Logo Visual Studio Code	26
2.19	Logo Postman	26
2.20	Logo Arduino IDE	27
2.21	Logo GitLab	27
2.22	Logo Docker	27
3.1	Diagramme de cas d'utilisation du Sprint 1	31
3.2	Diagramme de classes du Sprint1	33
3.3	Planification des sprints sur Jira	34
3.4	Organisation des dépôts GitLab du projet Smart House	36
3.5	Exécution des conteneurs Docker du projet Smart House	37
3.6	Collection MongoDB des utilisateurs	38
3.7	Collection MongoDB des abonnements	38
3.8	Stockage du jeton JWT dans un cookie côté client	40

3.9	Architecture globale du Sprint 1	41
3.10	Interface d'accueil de l'application Smart House	42
3.11	Interface de création de compte	43
3.12	Interface de connexion	44
3.13	Échec de connexion	45
3.14	Interface des offres d'abonnement	46
3.15	Interface de paiement	46
3.16	Interface d'échec de paiement	47
4.1	Diagramme de cas d'utilisation du Sprint2	51
4.2	Diagramme de blocs du système Smart House	54
4.3	Diagramme de classes du système de maison intelligente	55
4.4	Diagramme de blocs interne du système Smart House	56
4.5	Diagramme de blocs du système de surveillance Smart House avec ESP32-CAM	58
4.6	Diagramme d'activité du système IoT et de l'application Smart House	59
4.7	Schéma du câblage	62
4.8	Montage matériel du prototype Smart House	65
4.9	Montage matériel détaillé	66
4.10	Affichage des données capteurs et de la connexion MQTT dans la console série	67
4.11	Affichage des données et alertes sur le moniteur série	68
4.12	Module ESP32-CAM utilisé dans le système	69
4.13	Interface de gestion des équipements	70
4.14	Interface d'historique des mesures de température et d'humidité	71
5.1	Diagramme de cas d'utilisation — Gestion des notifications et des membres	74
5.2	Diagramme de séquence système — Ajouter un membre	77
5.3	Diagramme de séquence système — Consulter les notifications	78
5.4	Diagramme de classes du système Smart House	79
5.5	Diagramme de déploiement du système Smart House IoT	80
5.6	Interface de consultation des notifications	81
5.7	Interface de consultation de l'historique	82
5.8	Saisie des informations d'un nouveau membre	83
5.9	Confirmation de l'ajout réussi d'un membre	84
5.10	Échec de l'ajout d'un membre	85
5.11	Interface de suppression d'un membre	86
6.1	Diagramme de cas d'utilisation de gestion du profil et des abonnements	90

6.2	Diagramme de Séquence Système (Boîte Noire) — Soumettre une Réclamation	94
6.3	Diagramme de classes Sprint 4	96
6.4	Diagramme d'activité — Gestion des abonnements	97
6.5	Diagramme d'activité — Soumettre une réclamation	98
6.6	Interface de gestion du profil	99
6.7	Interface de gestion des abonnements	100
6.8	Interface de soumission des réclamations	101
6.9	Détection automatique des contenus offensants dans une réclamation .	101
7.1	Diagramme de cas d'utilisation du Sprint 5	104
7.2	Diagramme de classes du Sprint 5	110
7.3	Diagramme d'activité — Traitement des réclamations	112
7.4	Interface de gestion des abonnements	113
7.5	Interface de gestion des réclamations	114
7.6	Interface de consultation des données des capteurs	115
7.7	Interface de gestion des utilisateurs	116

Liste des tableaux

2.1	Fonctionnalités selon les acteurs du système	12
2.2	Backlog du produit	15
2.3	Spécifications techniques de la carte ESP32-WROOM-32 CP2102	21
3.1	Sprint Backlog - Authentification et Abonnement	30
3.2	Description textuelle du cas d'utilisation numéro 1	32
4.1	Sprint Backlog - Gestion des équipements IoT	50
4.2	Cas d'utilisation : Gérer les capteurs	52
4.3	Cas d'utilisation : Consulter la diffusion vidéo et images	53
4.4	Connexions des composants avec la carte ESP32	61
5.1	Sprint Backlog - Notifications et Historique	73
5.2	Description textuelle du cas d'utilisation : Ajouter un membre	75
5.3	Description textuelle du cas d'utilisation : Consulter les notifications . .	76
6.1	Sprint Backlog – Profil & Abonnement	89
6.2	Description textuelle du cas d'utilisation : Gérer le profil	91
6.3	Description textuelle du cas d'utilisation : Gérer les abonnements	92
6.4	Description textuelle du cas d'utilisation : Soumettre une réclamation . .	93
7.1	Sprint Backlog - Administration	103
7.2	Cas d'utilisation : Gérer les réclamations	106
7.3	Cas d'utilisation : Gérer les abonnements	108
7.4	Tests fonctionnels du Sprint 5	117

Introduction Générale

Avec l'évolution rapide des technologies numériques, l'Internet des Objets (IoT) est devenu une technologie incontournable dans de nombreux domaines. L'IoT désigne l'ensemble des objets connectés à Internet capables de collecter, d'échanger et de traiter des données afin d'automatiser certaines tâches et d'améliorer la prise de décision. Parmi ces innovations, les maisons intelligentes (Smart House) représentent une solution moderne permettant d'améliorer le confort, la sécurité ainsi que la gestion énergétique des habitations. Grâce à l'intégration de capteurs, de systèmes embarqués et d'applications web, il devient possible de surveiller et de contrôler l'environnement domestique en temps réel.

Actuellement, plusieurs solutions permettent de gérer à distance les équipements d'une habitation via un smartphone ou un ordinateur. Ces systèmes offrent la possibilité de consulter différentes informations et d'interagir avec certains appareils connectés. Cependant, malgré leurs avantages, ils présentent encore certaines limites, notamment des coûts d'installation parfois élevés, une configuration complexe ou encore des difficultés d'intégration entre différents équipements provenant de technologies variées. De plus, certaines plateformes manquent de flexibilité pour répondre efficacement aux besoins spécifiques des utilisateurs.

Dans ce cadre, notre projet de fin d'études a pour objectif de concevoir et de développer un système de gestion de maison intelligente basé sur les technologies IoT. La solution proposée repose sur une carte connectée à plusieurs capteurs intelligents et associée à une application web interactive. Le système permet de collecter les données de l'environnement domestique en temps réel, de détecter les situations anormales, de générer des alertes et de centraliser l'ensemble des informations au sein d'une plateforme unique. L'application permet également à l'utilisateur de consulter les données collectées, de gérer les équipements connectés et d'interagir avec les différents services proposés par la plateforme.

Le présent rapport est organisé en plusieurs chapitres. Le premier chapitre présente le cadre général du projet, l'organisme d'accueil, l'étude de l'existant, la problématique ainsi que la méthodologie de gestion adoptée. Le deuxième chapitre est consacré à l'analyse et à la spécification des besoins, à travers l'identification des acteurs, des exigences fonctionnelles et non fonctionnelles, ainsi que la planification du projet selon le framework Scrum. Les chapitres suivants décrivent les différents sprints réalisés, en détaillant pour chacun les objectifs, les user stories, les choix de conception, les fonctionnalités développées et les interfaces obtenues. Enfin, le dernier chapitre met l'accent sur les fonctionnalités d'administration, les tests effectués et les résultats obtenus, avant de conclure sur les apports du projet et les perspectives d'amélioration.

Chapitre 1

Cadre Général du Projet

Introduction

Dans ce chapitre, nous présentons l'entreprise Intuitive Group, qui constitue le cadre de réalisation de notre stage, ainsi que son domaine d'activité et son environnement de travail. Nous abordons ensuite la problématique à laquelle répond notre projet de stage, en précisant les objectifs visés. Enfin, nous décrivons le framework adopté pour mener à bien ce travail, en mettant l'accent sur le framework agile Scrum.

1.1 Présentation de l'organisme d'accueil

Intuitive Group est une entreprise française spécialisée dans le développement de solutions numériques innovantes. Elle accompagne ses clients dans la conception et la réalisation de projets web et mobiles, en combinant une expertise technique solide avec une approche centrée sur les besoins de l'utilisateur. Soucieuse de la qualité, de la performance et de la satisfaction client, l'entreprise s'appuie sur des méthodologies modernes de gestion de projet afin de proposer des solutions fiables, évolutives et adaptées aux exigences du marché.

Fiche signalétique

Intuitive Group

Raison sociale :	Intuitive Group
Adresse :	46, Av. Taieb Mhiri, La Marsa, Tunis, 2070, TN
Téléphone :	22 299 944
Email :	contact@intuitiv.fr

1.2 Étude de l'existant

L'étude de l'existant constitue une étape importante dans la compréhension des solutions actuellement utilisées dans le domaine des maisons intelligentes. Elle permet d'analyser les fonctionnalités offertes par les systèmes Smart House existants, d'identifier leurs avantages ainsi que leurs limites, puis de mieux définir les besoins auxquels notre projet doit répondre. Cette étude sert également de base pour proposer une solution plus adaptée, sécurisée et facile à utiliser.

1.2.1 Description des solutions existantes

Actuellement, les systèmes de maison intelligente (Smart House) s'appuient sur l'intégration de capteurs connectés et de dispositifs automatisés destinés à superviser et contrôler différents éléments de l'habitat. Ces solutions utilisent généralement des capteurs de température, d'humidité, de mouvement et de gaz, ainsi que des caméras de surveillance afin de collecter des informations sur l'environnement domestique. Les données recueillies sont ensuite transmises en temps réel vers des applications web ou mobiles, offrant à l'utilisateur la possibilité de consulter et de gérer son domicile à distance.

Parmi les solutions les plus connues figurent **Google Home**, **Samsung Smart-Things** et **Home Assistant** qui permettent la centralisation et l'automatisation des équipements connectés. Ces plateformes permettent de connecter différents objets intelligents et d'automatiser certaines tâches domestiques telles que l'éclairage, la surveillance, le contrôle de la température et la gestion des appareils électroménagers.

1.2.2 Critique de l'existant

Malgré les avancées significatives réalisées dans le domaine des maisons intelligentes, plusieurs contraintes subsistent dans les solutions actuellement disponibles. L'un des principaux obstacles demeure le coût relativement élevé de certaines plateformes et des équipements associés. En effet, la mise en place d'un écosystème Smart House complet nécessite souvent l'acquisition de plusieurs capteurs, actionneurs et dispositifs compatibles, ce qui représente un investissement important pour de nombreux utilisateurs.

Par ailleurs, certaines solutions reposent fortement sur un écosystème propriétaire, ce qui peut limiter le choix des équipements compatibles et entraîner des coûts supplémentaires lors de l'extension du système. Bien que plusieurs plateformes proposent des fonctionnalités avancées de supervision et d'automatisation, les possibilités de personnalisation restent parfois limitées par les contraintes imposées par les fabricants.

Ces limites mettent en évidence la nécessité de développer des solutions plus accessibles, flexibles et personnalisables, capables de répondre efficacement aux besoins spécifiques des utilisateurs tout en réduisant les coûts de déploiement.

1.2.3 Solution proposée

Afin de répondre à ces différentes limitations, notre projet a pour objectif de concevoir un système de gestion de maison intelligente intégré, fiable et performant. La solution proposée repose sur l'utilisation de plusieurs capteurs connectés, notamment des capteurs de température, d'humidité, de mouvement, de gaz et de flamme, ainsi qu'une caméra de surveillance permettant un suivi continu de l'environnement domestique en temps réel.

Le système développé vise à centraliser la gestion et la supervision des différents équipements de la maison au sein d'une plateforme unique accessible à travers une interface web simple, ergonomique et intuitive. Grâce à cette application, l'utilisateur peut surveiller l'état de son habitation à distance, consulter les données collectées en temps réel, gérer les équipements connectés et recevoir des notifications en cas de détection d'anomalies ou d'événements inhabituels.

Cette approche permet non seulement d'améliorer la sécurité, le confort et l'efficacité énergétique de l'habitat, mais également de réduire les coûts de mise en œuvre grâce à l'utilisation de composants matériels économiques, notamment la carte ESP32 et des capteurs à faible coût. Le système proposé constitue ainsi une solution abordable et accessible, offrant des fonctionnalités avancées de supervision et d'automatisation sans nécessiter un investissement important de la part de l'utilisateur.

1.3 Méthodologie utilisée

Les méthodes traditionnelles de gestion de projets se basent sur une spécification initiale très précise des besoins, suivie d'une phase de réalisation puis d'une livraison finale du produit. Dans ce type d'approche séquentielle, chaque intervenant doit terminer l'ensemble de ses tâches avant de les transmettre à l'étape suivante, ce qui rend difficile la prise en compte des changements et des évolutions éventuelles des besoins du client.

Afin de surmonter ces limites, les méthodologies agiles ont été introduites. Elles favorisent une approche itérative et collaborative permettant une adaptation continue aux besoins du client tout au long du cycle de développement. Parmi les méthodes agiles les plus connues, on peut citer Scrum, Kanban, Extreme Programming (XP), Lean Software Development et Crystal. Ces approches mettent l'accent sur la communication entre les membres de l'équipe, la livraison fréquente de fonctionnalités opérationnelles et l'amélioration continue du produit.

Dans le cadre de notre projet Smart House, nous avons opté pour le framework Scrum en raison de sa simplicité, de sa flexibilité et de sa capacité à gérer efficacement les changements de besoins. Cette méthode permet de découper le projet en plusieurs itérations appelées *Sprints*, chacune aboutissant à un incrément fonctionnel du produit. Elle facilite également le suivi de l'avancement du projet, la collaboration entre les membres de l'équipe et l'intégration progressive des fonctionnalités.

1.3.1 Framework Scrum

Le framework Scrum est un cadre de travail itératif et incrémental utilisé pour gérer les projets de développement de logiciels et de produits. Il est basé sur des principes agiles qui mettent l'accent sur la collaboration, la flexibilité et l'adaptation au changement. Grâce à son organisation en sprints, Scrum permet de livrer régulièrement des versions fonctionnelles du système tout en assurant une amélioration continue du produit développé.

La figure ci-dessous représente le cycle de vie du framework Scrum :

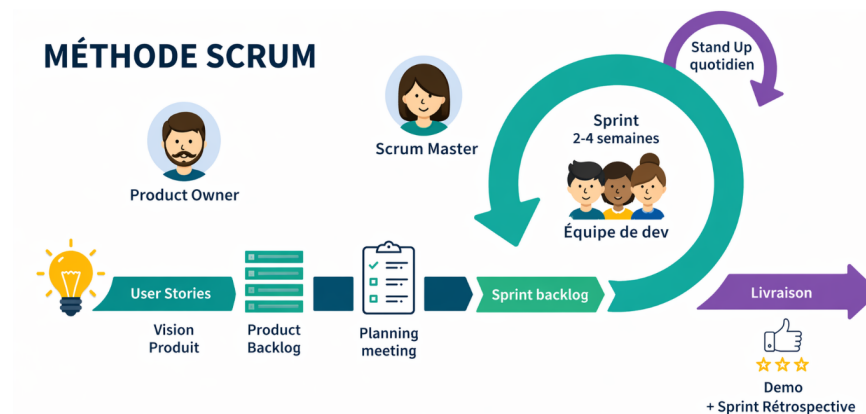


FIGURE 1.1 – Cycle de vie du framework Scrum

Le framework Scrum repose sur trois rôles principaux :

- **Product Owner** : responsable de définir les objectifs et les exigences du projet.
- **Scrum Master** : responsable de faciliter le processus de développement.
- **L'équipe de développement** : responsable de la conception et de la mise en œuvre du produit.[21]

1.3.2 Langages de modélisation

Pour la modélisation et la conception de notre système Smart House, nous avons utilisé les langages de modélisation UML (*Unified Modeling Language*) et SysML (*Systems Modeling Language*). Ces deux langages permettent de représenter graphiquement

les différents aspects du système afin de faciliter son analyse, sa conception et sa documentation.

UML

UML (Unified Modeling Language) est un langage de modélisation standardisé largement utilisé dans le domaine du génie logiciel. Il fournit un ensemble de diagrammes permettant de décrire la structure, le comportement et les interactions d'un système informatique. UML facilite la compréhension des besoins, améliore la communication entre les différents acteurs du projet et contribue à la production d'une documentation claire et cohérente. Grâce à ses outils de modélisation, il permet de visualiser l'architecture du système avant sa phase de développement.



FIGURE 1.2 – Logo UML

Les principaux diagrammes UML utilisés dans le cadre de ce projet sont :

- **Le diagramme de cas d'utilisation** : il permet d'identifier les acteurs du système ainsi que les fonctionnalités qui leur sont offertes.
- **Le diagramme de classes** : il représente la structure statique du système en décrivant les classes, leurs attributs, leurs méthodes et les relations qui les unissent.
- **Le diagramme de séquence** : il illustre les échanges de messages entre les différents objets du système selon un ordre chronologique.
- **Le diagramme de déploiement** : il décrit l'architecture physique du système ainsi que la répartition des composants logiciels sur les différents équipements matériels.

Cette modélisation nous a permis d'obtenir une vision claire et détaillée de la solution proposée avant son implémentation.[\[1\]](#)

1.3.3 SysML

SysML (Systems Modeling Language) est un langage de modélisation dérivé d'UML, spécialement conçu pour l'analyse, la spécification, la conception et la validation des systèmes complexes. Contrairement à UML, qui est principalement orienté vers le développement logiciel, SysML permet de modéliser des systèmes multidisciplinaires intégrant à la fois des composants matériels, logiciels, électroniques et des processus métier. Il est largement utilisé dans plusieurs domaines tels que l'aérospatiale, l'automobile, les télécommunications et les systèmes embarqués.

Dans le cadre de notre projet Smart House, SysML a été utilisé pour représenter les exigences du système, décrire son architecture et modéliser les interactions entre ses différents composants. Son utilisation a permis d'obtenir une vision globale et structurée de la solution proposée tout en assurant une meilleure traçabilité des besoins tout au long du cycle de développement.



FIGURE 1.3 – Logo SysML

Les principaux objectifs de la modélisation SysML sont :

- Identifier et analyser les besoins du système.
- Définir les exigences fonctionnelles et non fonctionnelles.
- Concevoir l'architecture globale du système.
- Modéliser les interactions et le comportement des composants.
- Vérifier et valider la conformité de la solution par rapport aux besoins exprimés.

Les diagrammes SysML les plus utilisés dans le cadre de ce projet sont :

- **Diagramme des exigences** : permet de représenter, organiser et tracer les exigences fonctionnelles et non fonctionnelles du système.

- **Diagramme de définition de blocs (BDD)** : décrit la structure statique du système ainsi que les relations entre ses différents composants.
- **Diagramme de blocs internes (IBD)** : représente les échanges de données et les interactions entre les composants internes du système.
- **Diagramme d'activité** : modélise les processus, les flux de contrôle et les comportements dynamiques du système.

L'utilisation de SysML a contribué à une meilleure compréhension des besoins du projet Smart House et a facilité la conception d'une architecture cohérente, évolutive et adaptée aux exigences du système.[2]

Conclusion

Dans ce premier chapitre, nous avons présenté le cadre général du projet en commençant par la description de l'organisme d'accueil, suivie d'une analyse de l'existant mettant en évidence ses limites et ses contraintes. Nous avons également défini la méthodologie adoptée pour la réalisation du projet, permettant d'assurer une meilleure organisation du travail et une compréhension globale du système à concevoir.

Le chapitre suivant sera consacré à l'analyse et à la spécification détaillée des besoins fonctionnels et non fonctionnels.

Chapitre 2

Analyse et Spécification des Besoins

Introduction

Ce chapitre présente la phase de démarrage consacrée à l'analyse et à la définition des besoins du système. Il porte principalement sur l'identification des besoins fonctionnels et non fonctionnels, l'élaboration du diagramme global des cas d'utilisation, ainsi que la mise en place de l'organisation du projet selon le framework Scrum. Nous y abordons également la création du backlog produit, la planification des différents sprints et la présentation de l'environnement matériel et logiciel utilisé pour la réalisation du projet.

2.1 Spécification des besoins

Dans le cadre du développement du système Smart House, une phase d'analyse approfondie a été réalisée afin de bien comprendre les attentes des utilisateurs ainsi que les contraintes techniques du projet. Cette étape constitue une base essentielle pour définir les fonctionnalités du système, structurer les besoins et assurer une conception cohérente et adaptée aux objectifs du projet.

2.1.1 Diagramme des exigences

Afin d'identifier et de structurer les principales fonctionnalités attendues du système Smart House, nous avons réalisé un diagramme des exigences basé sur le langage SysML. Ce diagramme permet d'organiser les besoins fonctionnels du système et de mettre en évidence les différentes fonctionnalités indispensables au bon fonctionnement de la plateforme intelligente. Il facilite également la compréhension des interactions entre les composants du système et constitue une base importante pour les étapes de conception et de développement.

La figure ci-dessous représente le diagramme des exigences du système Smart House.

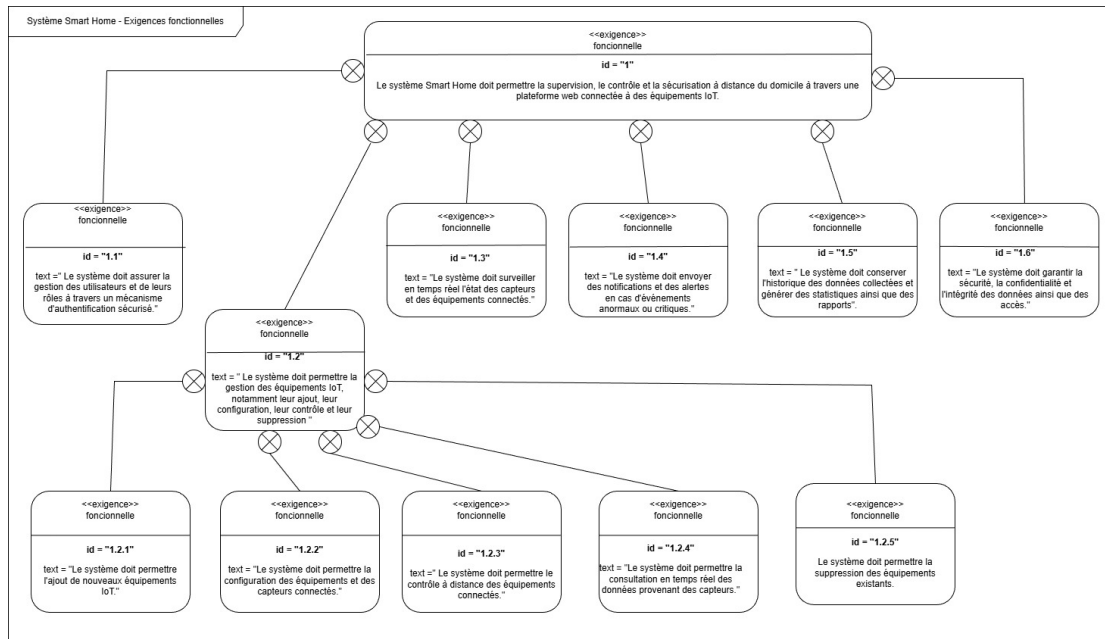


FIGURE 2.1 – Diagramme des exigences du système Smart House

Le diagramme des exigences présenté ci-dessus illustre les principaux besoins fonctionnels du système Smart House ainsi que les relations entre les différentes exigences. Il met en évidence les fonctionnalités essentielles de la plateforme, notamment la gestion sécurisée des utilisateurs, le contrôle et la supervision des équipements IoT, la surveillance en temps réel des capteurs, l'envoi de notifications en cas d'anomalie ainsi que la conservation des données pour la génération de statistiques et de rapports. Ce diagramme permet de structurer les besoins du système de manière claire et cohérente, tout en servant de référence pour les phases de conception et de développement de la solution.

2.1.2 Exigences non fonctionnelles

Les exigences non fonctionnelles décrivent les critères de qualité attendus du système Smart House :

- **Sécurité** : l'application doit protéger les données des utilisateurs et empêcher tout accès non autorisé.
- **Fiabilité** : le système doit fonctionner de manière stable et cohérente.
- **Performance** : l'application doit garantir une navigation fluide et un temps de réponse satisfaisant.
- **Disponibilité** : le service doit rester accessible lorsque l'utilisateur en a besoin.
- **Utilisabilité** : l'interface doit être simple, claire et intuitive.

2.2 Modélisation des besoins

La modélisation des besoins constitue une étape essentielle dans l'analyse du système Smart House. Elle permet d'identifier les différents acteurs interagissant avec la plateforme ainsi que les principales fonctionnalités offertes par le système. Cette modélisation facilite la compréhension des besoins des utilisateurs et définit clairement les interactions entre les acteurs et les services proposés par la solution.

2.2.1 Identification des acteurs

Les acteurs sont des entités externes qui interagissent avec le système. Dans le cadre de notre système Smart House, quatre acteurs principaux ont été identifiés.

TABLE 2.1 – Fonctionnalités selon les acteurs du système

Acteur	Fonctionnalités
Visiteur	— Consulter les informations des services — Acheter un abonnement — Créer un compte — S'authentifier — Payer une offre (Basic, Premium Plus, Extended Pro)
Membre	— S'authentifier — Gérer les équipements — Gérer le profil — Consulter les notifications — Consulter l'historique — Soumettre une réclamation
Résident	— Même fonctionnalités que le membre + — Gérer les abonnements — Gérer les membres de la famille
Administrateur	— S'authentifier — Gérer les utilisateurs — Traiter les réclamations — Consulter les relevés des capteurs par maison — Exporter les données en PDF — Gérer les abonnements

2.2.2 Diagramme de cas d'utilisation global

La figure 2.2 présente le diagramme de cas d'utilisation global du système Smart House. Ce diagramme met en évidence les interactions entre les acteurs externes (visiteur, résident, administrateur et carte électronique) et les principales fonctionnalités du système. Les acteurs doivent être représentés par des symboles d'acteurs UML placés à l'extérieur de la frontière du système, tandis que les cas d'utilisation doivent être placés à l'intérieur du rectangle représentant le système.

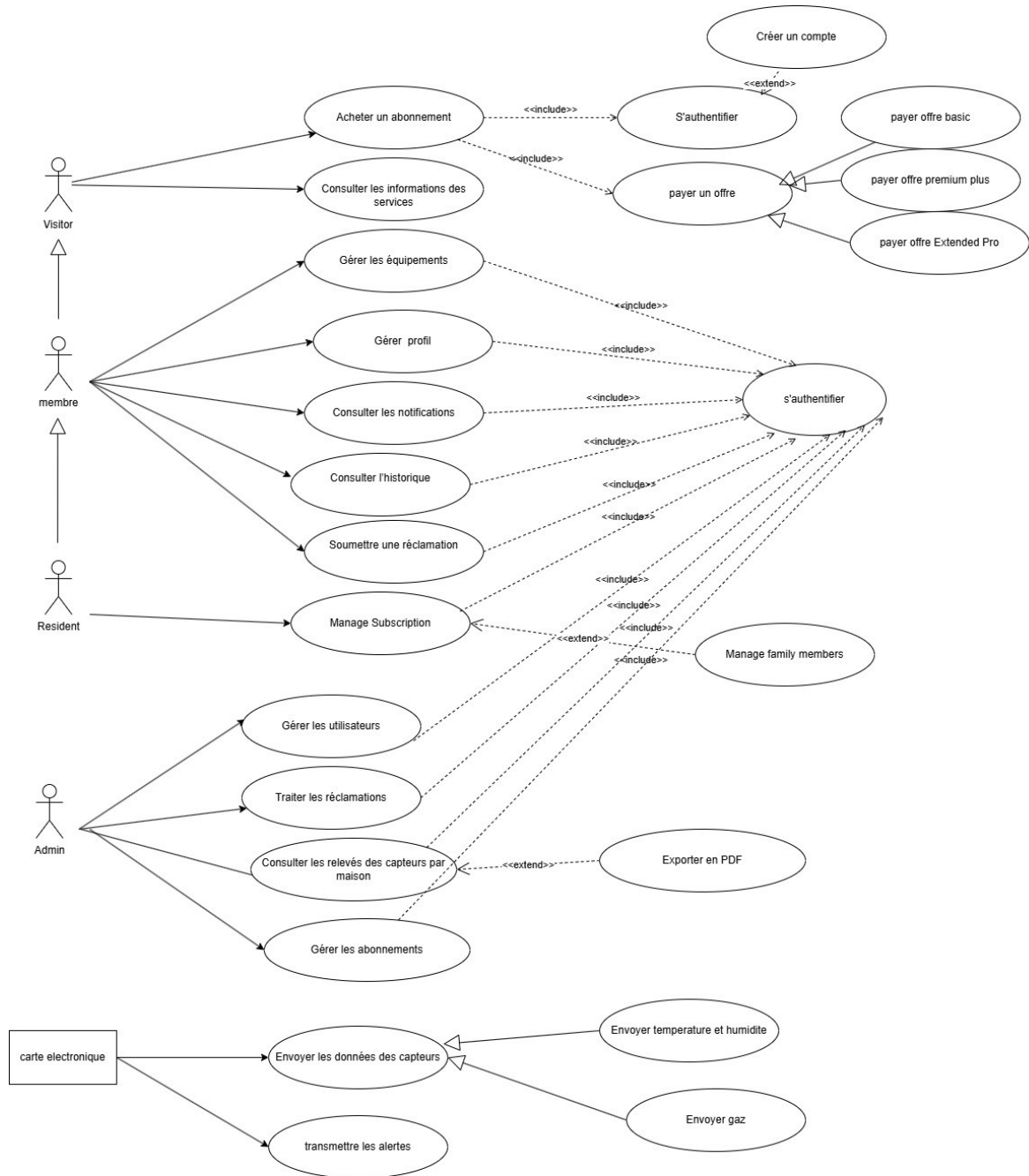


FIGURE 2.2 – Diagramme de cas d'utilisation du système

2.2.3 Diagramme de classes global

La figure 2.3 présente le diagramme de classes du système Smart House. Ce diagramme illustre les différentes classes du système ainsi que les relations entre elles. Il permet de mieux comprendre la structure de la base de données et l'organisation des fonctionnalités de l'application.

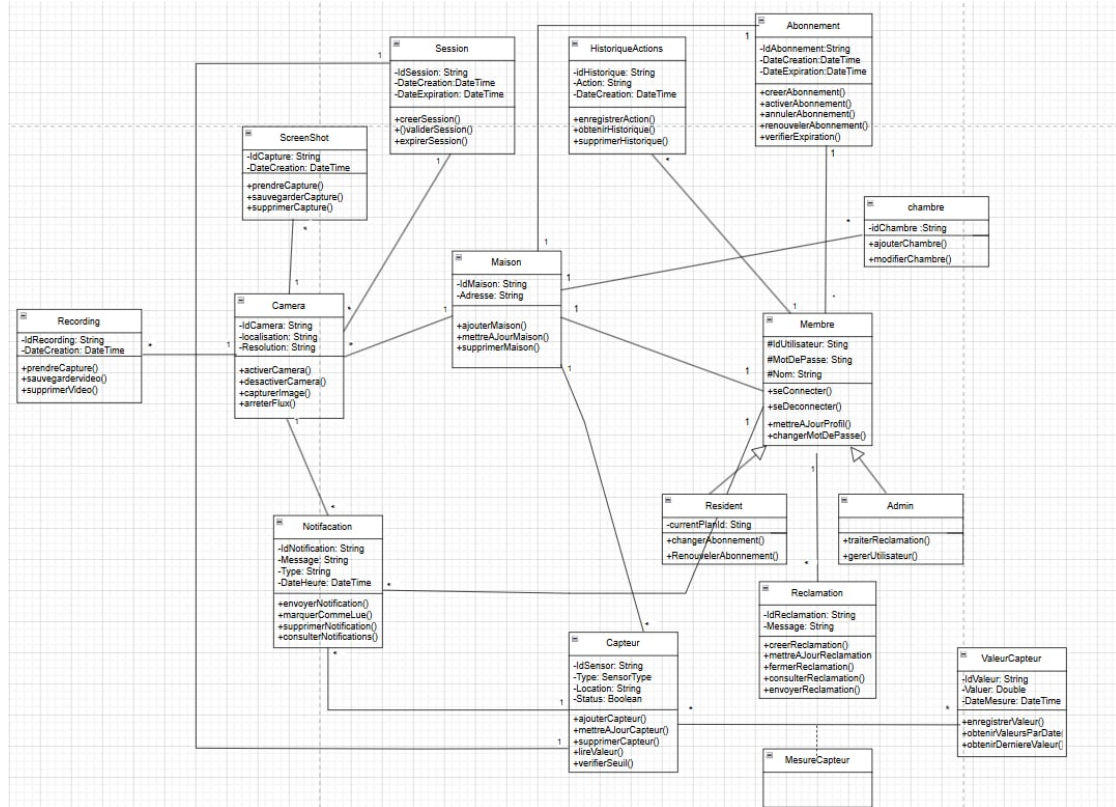


FIGURE 2.3 – Diagramme de classes

Ce diagramme met en évidence les principales classes utilisées dans le système, notamment les utilisateurs, les capteurs, les caméras, les alertes et les abonnements. Les relations entre ces classes assurent la gestion des données et la communication entre les différents composants de la plateforme Smart House.

2.3 Pilotage du projet avec Scrum

Afin d'assurer une organisation efficace du projet Smart House, nous avons adopté le framework Scrum. Cette approche permet de structurer le développement en plusieurs sprints tout en favorisant la collaboration entre les membres de l'équipe, le suivi continu des tâches et l'adaptation progressive aux besoins du projet.

2.3.1 Équipe et rôles

La figure ci-dessous montre l'organisation de l'équipe SCRUM et la répartition des responsabilités dans le cadre du projet :

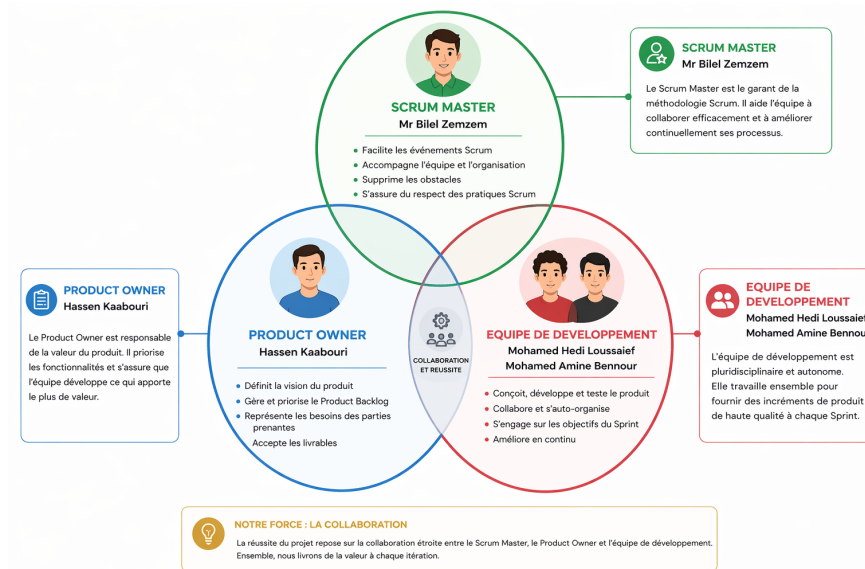


FIGURE 2.4 – Présentation de l'équipe SCRUM

2.3.2 Le Backlog du produit

Le tableau 2.2 présente le backlog du produit. Il regroupe les principales user stories identifiées, leur priorité, leur durée estimée et leur affectation aux différents sprints. Cette organisation permet de planifier le développement de manière progressive selon le framework Scrum.

TABLE 2.2 – Backlog du produit

ID	Acteur	User Story	Priorité	Durée	Sprint / Objectif
1	Visiteur	En tant que visiteur, je peux consulter les informations sur les services.	Haute	5 J	Sprint 1 : Découverte & Accès
2	Visiteur	En tant que visiteur, je peux créer un compte.	Haute	5 J	Sprint 1 : Découverte & Accès
3	Visiteur	En tant que visiteur, je peux acheter un abonnement.	Haute	6 J	Sprint 1 : Découverte & Accès
4	Résident	En tant que résident, je peux m'authentifier.	Haute	5 J	Sprint 1 : Découverte & Accès

ID	Acteur	User Story	Priorité	Durée	Sprint / Objectif
5	Résident	En tant que résident, je peux gérer les capteurs.	Haute	7 J	Sprint 2 : Gestion des équipements IoT
6	Résident	En tant que résident, je peux gérer les caméras.	Haute	7 J	Sprint 2 : Gestion des équipements IoT
7	Résident	En tant que résident, je peux consulter les captures vidéo et les images enregistrées par mes caméras.	Haute	7 J	Sprint 2 : Gestion des équipements IoT
8	Résident	En tant que résident, je peux consulter les notifications.	Haute	7 J	Sprint 3 : Notifications & Historique
9	Résident	En tant que résident, je peux consulter l'historique des actions et l'évolution des données collectées.	Moyenne	7 J	Sprint 3 : Notifications & Historique
10	Résident	En tant que résident, je peux gérer les membres de ma famille.	Haute	7 J	Sprint 3 : Notifications & Historique
11	Résident	En tant que résident, je peux gérer mon profil.	Moyenne	7 J	Sprint 4 : Profil & Abonnement
12	Résident	En tant que résident, je peux gérer mon abonnement.	Haute	7 J	Sprint 4 : Profil & Abonnement
13	Résident	En tant que résident, je peux soumettre une réclamation afin de signaler un problème.	Moyenne	7 J	Sprint 4 : Profil & Abonnement
14	Admin	En tant qu'administrateur, je peux gérer les abonnements.	Haute	5 J	Sprint 5 : Administration
15	Admin	En tant qu'administrateur, je peux gérer les réclamations.	Moyenne	4 J	Sprint 5 : Administration
16	Admin	En tant qu'administrateur, je peux consulter l'historique des événements et des données des maisons.	Moyenne	7 J	Sprint 5 : Administration
17	Admin	En tant qu'administrateur, je peux gérer les utilisateurs.	Haute	7 J	Sprint 5 : Administration

2.3.3 Planification des sprints

La figure 2.5 présente la planification prévisionnelle des différents sprints du projet sous forme d'un diagramme de Gantt. Cette planification permet de visualiser l'ordre d'exécution des sprints ainsi que leur durée tout au long de la période de réalisation du projet. Chaque Sprint est consacré à un ensemble de fonctionnalités spécifiques afin d'assurer une progression organisée et continue du système Smart House.

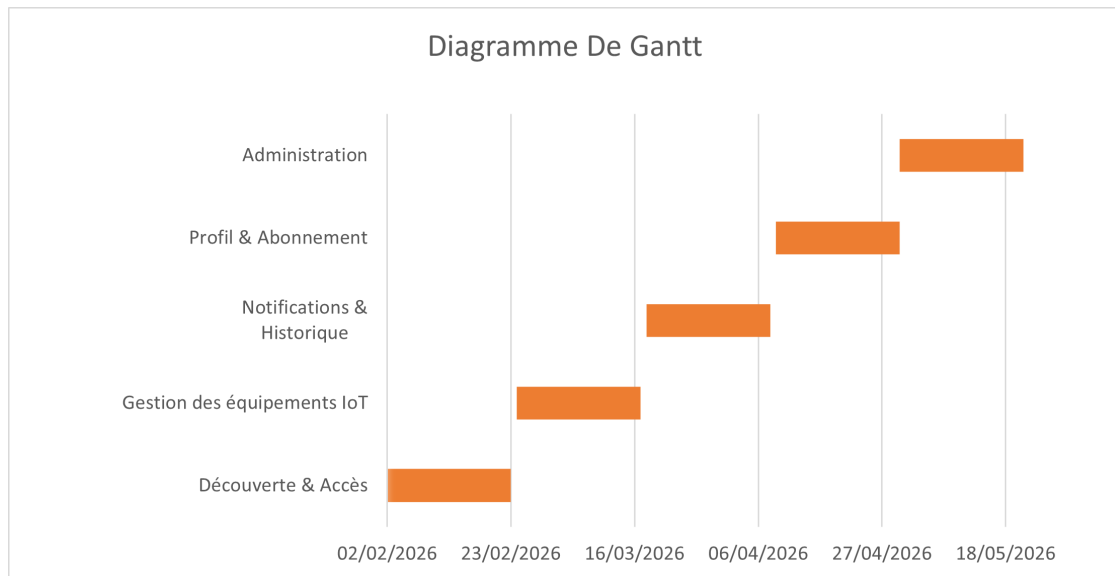


FIGURE 2.5 – Diagramme de Gantt des sprints du projet

Le diagramme de Gantt présenté ci-dessus offre une vue globale sur l'organisation temporelle des différentes phases du projet. Il met en évidence la répartition des tâches et la succession des sprints afin de garantir un suivi efficace de l'avancement du système Smart House durant toute la période de développement.

Après avoir présenté l'organisation et la planification des sprints, la section suivante sera consacrée à la description de l'environnement matériel et logiciel utilisé pour la réalisation du projet.

2.4 Environnement de travail

Le développement du système a été réalisé dans un environnement de travail combinant des outils matériels et logiciels soigneusement sélectionnés.

2.4.1 Architecture IoT du système

L'architecture présentée correspond à la solution IoT embarquée dédiée à la surveillance et au contrôle d'une maison intelligente. Elle repose sur une alimentation électrique adaptée permettant d'alimenter la carte ESP32, qui constitue l'unité de traitement principale du système. Ce microcontrôleur assure la collecte, le traitement et la transmission des données provenant des différents capteurs connectés au système.

Dans le cadre de notre projet Smart House, nous avons mis en place une architecture IoT moderne permettant la supervision de l'environnement domestique en temps réel.[22] Cette architecture facilite la communication entre les équipements intelligents, le serveur backend et l'application web. Elle garantit également une meilleure organisation du système, une bonne évolutivité ainsi qu'une gestion efficace des données.

Le système utilise plusieurs capteurs et modules connectés afin d'assurer la sécurité et le confort des utilisateurs. Parmi les composants intégrés, on retrouve :

- Le capteur **DHT22** pour mesurer la température et l'humidité.
- Le capteur **MQ2** pour détecter la présence de gaz et de fumée.
- Le capteur **PIR** pour détecter les mouvements et les intrusions.
- Le capteur de détection de feu (*Flame Sensor*) pour identifier les risques d'incendie.
- Le module **ESP32-CAM** permettant la capture d'images et la surveillance vidéo.

L'architecture IoT du système repose sur plusieurs couches fonctionnelles permettant d'assurer le bon fonctionnement de la solution :

1. **Couche perception** : Elle représente la couche matérielle contenant les capteurs et les dispositifs de collecte des données environnementales.
2. **Couche réseau** : Cette couche assure la communication entre les équipements IoT et le serveur via le réseau WiFi en utilisant des protocoles comme HTTP ou MQTT.
3. **Couche traitement** : Les données reçues sont traitées par le serveur backend développé avec Node.js/NestJS puis stockées dans une base de données MongoDB.
4. **Couche application** : Elle correspond à l'interface web permettant au résident de surveiller les équipements, consulter les alertes et contrôler certains appareils à distance.

5. **Couche métier** : Cette couche gère la logique fonctionnelle du système comme la gestion des utilisateurs, les notifications et les règles de sécurité.

Grâce à cette architecture, le système Smart House peut détecter les événements dangereux tels que les fuites de gaz, les incendies ou les intrusions, puis envoyer des alertes en temps réel à l'utilisateur. Cette organisation modulaire facilite également l'ajout de nouveaux équipements et l'évolution future du projet.

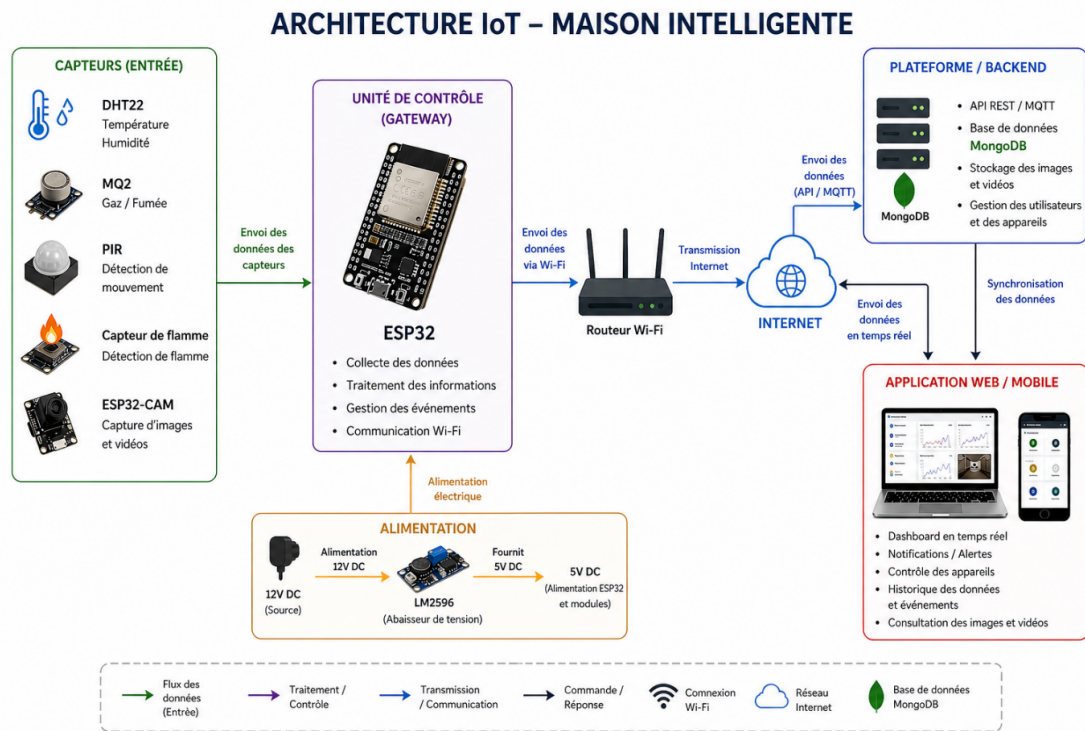


FIGURE 2.6 – Architecture IoT du système Smart House

La figure ci-dessus illustre l'architecture IoT retenue pour le système Smart House. Cette architecture est organisée en cinq couches distinctes, permettant d'assurer une communication fiable, sécurisée et performante entre les différents composants matériels et logiciels de la plateforme. Le choix de cette approche multicouche vise à garantir une meilleure modularité du système, à simplifier les opérations de maintenance et à favoriser l'évolutivité de la solution en fonction des futurs besoins.

La première couche, appelée couche de perception, regroupe l'ensemble des capteurs IoT chargés de collecter les données de l'environnement, telles que la température, l'humidité, la détection de mouvement et la présence de gaz. Ces informations constituent la base du fonctionnement du système de surveillance et d'automatisation de la maison intelligente.

La deuxième couche correspond à la couche de contrôle, assurée par la carte ESP32. Cette dernière a été choisie en raison de sa puissance de traitement, de sa connectivité Wi-Fi intégrée, de sa faible consommation énergétique et de sa capacité à gérer simulta-

nément plusieurs capteurs. Elle assure l'acquisition des données, leur prétraitement et leur transmission vers les couches supérieures.

La troisième couche représente la couche de communication. Elle est responsable de l'échange des données entre l'ESP32 et le serveur backend à travers le réseau Wi-Fi en utilisant le protocole HTTP. Cette couche garantit une transmission rapide et continue des informations collectées, assurant ainsi la réactivité du système.

La quatrième couche est dédiée au traitement et au stockage des données. Elle comprend le serveur backend développé avec NestJS ainsi que la base de données MongoDB. Cette couche assure la réception, le traitement, la sécurisation et la persistance des données provenant des équipements IoT, tout en fournissant les services nécessaires aux applications clientes.

Enfin, la cinquième couche correspond à la couche applicative, développée avec Angular. Elle offre aux utilisateurs et aux administrateurs une interface intuitive permettant de consulter les données en temps réel, de recevoir des notifications en cas d'anomalie et d'interagir avec les différents services de la plateforme Smart House.

L'adoption de cette architecture en cinq couches permet de séparer clairement les responsabilités de chaque composant, d'améliorer les performances globales du système et de faciliter l'intégration de nouvelles fonctionnalités ou de nouveaux équipements IoT à l'avenir.

2.4.2 Environnement matériel

Le développement de la plateforme a été réalisé sur deux postes de travail dotés des caractéristiques suivantes :

Processeur :	Intel Core i5, 11 ^e génération
Mémoire vive :	16 Go RAM
Stockage :	SSD 512 Go
Système d'exploitation :	Windows 11

Microcontrôleur : ESP32-WROOM-32 CP2102

Dans le cadre de la conception de la solution IoT, la carte **ESP32 WROOM-32 intégrant le convertisseur USB-série CP2102** a été retenue comme unité de contrôle principale. Cette carte se distingue par ses capacités avancées de connectivité Wi-Fi et Bluetooth, sa puissance de traitement, ainsi que sa compatibilité avec une large variété de capteurs et modules IoT.

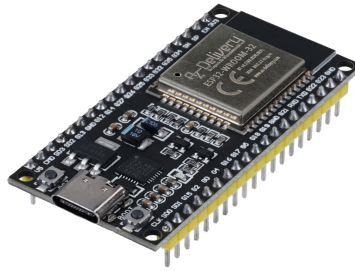


FIGURE 2.7 – La carte ESP32-WROOM-32 CP2102

Avantages de l'ESP32 :

- **Connectivité intégrée** : Wi-Fi et Bluetooth natifs, sans modules externes.
- **Puissance de traitement élevée** : processeur double cœur gérant simultanément la communication réseau et le pilotage des capteurs.
- **Compatibilité avec les systèmes de sécurité** : support des protocoles HTTPS et MQTT sécurisé.[7]

TABLE 2.3 – Spécifications techniques de la carte ESP32-WROOM-32 CP2102

Caractéristique	Description
Microcontrôleur	ESP32-WROOM-32
Convertisseur USB-Série	CP2102 (Silicon Labs)
Processeur	Dual-Core Xtensa LX6
Fréquence d'horloge	Jusqu'à 240 MHz
Mémoire Flash	4 MB
Mémoire SRAM	520 KB
Connectivité	Wi-Fi 802.11 b/g/n, Bluetooth v4.2 (BR/EDR + BLE)
GPIO	Jusqu'à 34 GPIO
Interfaces supportées	UART, SPI, I2C, I2S, PWM, ADC, DAC
Tension de fonctionnement	3.3 V
Tension d'alimentation	5 V via USB Type-C
Consommation énergétique	Faible avec modes veille (Deep Sleep)
Température de fonctionnement	-40°C à +85°C

Capteur DHT22

Le capteur **DHT22** assure la surveillance des conditions environnementales au sein de la maison intelligente, mesurant avec précision la température et l'humidité ambiantes.

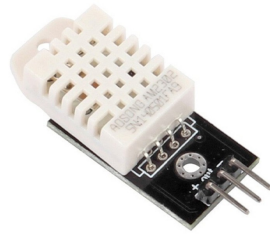


FIGURE 2.8 – Capteur DHT22

Le capteur DHT22 (également référencé AM2302) est un capteur numérique de température et d'humidité offrant une précision de $\pm 0,5$ °C et $\pm 2\%$ HR [9]. Il communique via une interface série sur un seul fil et est interrogé périodiquement par la carte ESP32 afin de transmettre les mesures environnementales vers le serveur backend [9].

Capteur MQ2

Le capteur **MQ2** assure la détection des gaz dangereux et de la fumée, constituant un élément essentiel pour la sécurité des occupants de la maison intelligente.

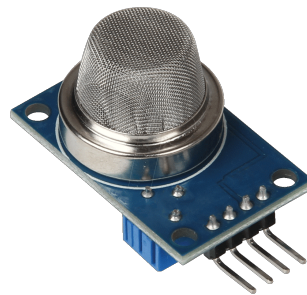


FIGURE 2.9 – Capteur MQ2

Le capteur MQ2 est un détecteur de gaz combustibles et de fumée sensible au GPL, au méthane, au propane, à l'hydrogène et à la fumée [10]. Il produit un signal analogique proportionnel à la concentration des gaz détectés, permettant à la carte ESP32 d'évaluer le niveau de danger et de déclencher une alerte si un seuil critique est dépassé [10].

Capteur de flamme

Le capteur de flamme détecte les rayonnements infrarouges émis par les flammes, permettant une identification précoce des incendies.

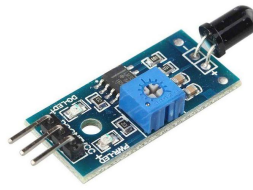


FIGURE 2.10 – Capteur de flamme

Capteur PIR

Le capteur PIR est un appareil qui détecte le mouvement en recevant un rayonnement infrarouge. Il détecte un changement rapide de l'énergie infrarouge et émet un signal lorsque quelqu'un passe devant le capteur[11].



FIGURE 2.11 – Capteur PIR

ESP32-CAM

L'ESP32-CAM est un module combinant un microcontrôleur ESP32 et un capteur d'images OV2640, capable de capturer des images jusqu'à 2 mégapixels et de transmettre un flux vidéo via le réseau Wi-Fi [8]. Ce module permet d'assurer la surveillance visuelle de la maison intelligente à moindre coût tout en offrant une intégration simple avec le backend NestJS [8].

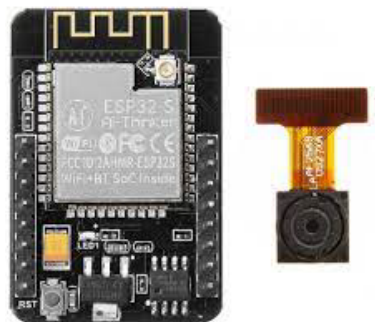


FIGURE 2.12 – ESP32-CAM OV2640

Avantages de l'ESP32-CAM :

- **Intégration complète** : combine le microcontrôleur ESP32 et le capteur d'image OV2640 dans un seul module.
- **Flexibilité du microcontrôleur** : prise en charge de plusieurs protocoles de communication.
- **Performances du capteur OV2640** : haute résolution et bonne qualité d'image.

ESP32-CAM-MB

Le programmeur **ESP32-CAM-MB** est une carte conçue pour programmer le module ESP32-CAM. Il est équipé d'un convertisseur USB/TTL CH340C et d'un connecteur USB Type Micro-B.

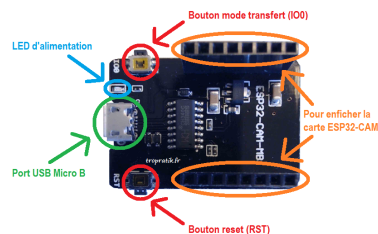


FIGURE 2.13 – ESP32-CAM-MB

2.4.3 Environnement logiciel

L'environnement logiciel regroupe aussi l'ensemble des langages et technologies utilisés pour la conception et le développement du système. Ces outils ont été sélectionnés afin d'assurer la performance, la sécurité et la fiabilité du système.

Angular

Angular est un framework frontend développé et maintenu par Google, utilisé pour la conception d'applications web dynamiques, modulaires et performantes . Il repose sur le langage TypeScript et propose une architecture orientée composants facilitant la séparation des responsabilités, la gestion des services et la maintenance du code à grande échelle . Ce choix a été retenu pour notre projet car Angular offre une structure complète et adaptée aux applications web complexes et évolutives.[3]



FIGURE 2.14 – Logo Angular

NestJS

NestJS est un framework backend basé sur Node.js , conçu pour le développement d'API REST robustes et évolutives. Il s'appuie fortement sur TypeScript et adopte une architecture modulaire inspirée d'Angular, garantissant une meilleure maintenabilité et une séparation claire des modules . NestJS a été retenu dans notre projet pour la structuration du backend, la sécurisation des routes et la facilité d'évolution des services exposés.[4]



FIGURE 2.15 – Logo NestJS

C++ (Arduino)

Le langage C++ est utilisé pour la programmation de la carte ESP32 via l'environnement Arduino IDE . Il permet un contrôle fin des ressources matérielles, la gestion des capteurs et la communication réseau avec le backend . Ce langage a été choisi pour sa compatibilité native avec l'écosystème Arduino et sa capacité à gérer efficacement les contraintes temps réel des systèmes embarqués.



FIGURE 2.16 – Logo C++

MongoDB

MongoDB est une base de données NoSQL orientée documents, utilisée dans notre projet pour stocker les informations des utilisateurs, les données des capteurs IoT et les journaux d'événements du système . Sa structure flexible, basée sur des documents JSON, la rend particulièrement adaptée aux données hétérogènes générées par les équipements connectés . MongoDB a été choisi pour sa scalabilité et sa compatibilité directe avec l'écosystème Node.js/NestJS.[5]



FIGURE 2.17 – Logo MongoDB

2.4.4 Outils de développement

Cette section présente les principaux outils de développement mobilisés pour concevoir, implémenter et tester le système :

Visual Studio Code

Visual Studio Code est un éditeur de code source léger et extensible développé par Microsoft, offrant un support natif pour TypeScript, Angular, NestJS et Docker grâce à un écosystème riche d'extensions . Il a été l'environnement de développement principal utilisé tout au long du projet.[16]

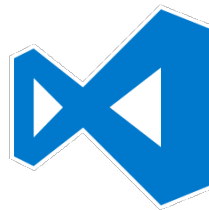


FIGURE 2.18 – Logo Visual Studio Code

Postman

Postman est une plateforme collaborative de développement et de test d'API REST permettant de simuler des requêtes HTTP, de valider les réponses du serveur et de déboguer les endpoints backend . Il a été utilisé tout au long du projet pour valider les API développées avec NestJS avant leur intégration dans le frontend Angular.[17]



FIGURE 2.19 – Logo Postman

Arduino IDE

L'Arduino IDE est un environnement de développement intégré open source permettant d'écrire, compiler et téléverser du code sur des microcontrôleurs compatibles, dont l'ESP32 . Il fournit une bibliothèque étendue de pilotes pour les capteurs utilisés dans le projet et simplifie significativement le développement embarqué .[15]



FIGURE 2.20 – Logo Arduino IDE

GitLab

GitLab est une plateforme DevSecOps intégrée permettant la gestion du code source, le suivi des versions, la revue de code et l'intégration continue . Elle a été utilisée dans notre projet pour centraliser le dépôt de code, gérer les branches de développement et faciliter la collaboration entre les membres de l'équipe .[14]



FIGURE 2.21 – Logo GitLab

Docker

Docker est une plateforme de conteneurisation permettant d'empaqueter une application et ses dépendances dans des conteneurs isolés et portables . Dans notre projet, Docker orchestre via un fichier `docker-compose` les quatre services principaux du système : le frontend Angular, le backend NestJS, la base de données MongoDB et le broker MQTT Mosquitto.[13]



FIGURE 2.22 – Logo Docker

Conclusion

Dans ce chapitre, nous avons présenté l'analyse et la spécification des Dans ce chapitre, nous avons présenté l'analyse ainsi que la spécification des besoins du système Smart House. Cette étude nous a permis d'identifier les différents acteurs intervenant dans le système, de définir les besoins fonctionnels et non fonctionnels, puis de les structurer à travers le diagramme global des cas d'utilisation ainsi que le backlog produit.

Nous avons également décrit la planification des différents sprints et présenté l'environnement matériel et logiciel adopté pour la réalisation du projet. Ces éléments constituent une base essentielle pour organiser le développement du système et assurer une progression cohérente conformément au framework Scrum.

Après avoir défini les besoins, les outils et l'organisation du travail, le chapitre suivant sera consacré à la réalisation du premier sprint, portant principalement sur la découverte de la plateforme, l'accès au système ainsi que les premières fonctionnalités développées.

Chapitre 3

Sprint 1 : Découverte et accès à la plateforme

Introduction

Dans ce chapitre, nous décrivons les étapes réalisées durant le Sprint 1 du projet Smart House, dédié à la découverte de la plateforme et à la gestion de l'accès utilisateur. Nous commençons par l'analyse des besoins fonctionnels concernant l'authentification et la souscription aux abonnements, puis présentons la conception des différentes composantes du système à l'aide de diagrammes UML et de l'architecture choisie. Enfin, nous illustrons le parcours utilisateur en présentant les principales interfaces développées dans l'application.

3.1 Sprint Goal

L'objectif principal du Sprint 1 est de développer les fonctionnalités de base permettant au visiteur de découvrir la plateforme Smart House, de créer un compte, de s'authentifier de manière sécurisée et de souscrire à une offre d'abonnement. Ce Sprint vise également à garantir une expérience utilisateur fluide grâce au développement des interfaces essentielles du système.

3.2 Sprint Backlog

Le tableau suivant représente les user stories du Sprint 1 :

TABLE 3.1 – Sprint Backlog - Authentification et Abonnement

ID	Acteur	User Story	Priorité	Estimation	Fonctionnalité
1	Visiteur	En tant que visiteur, je peux consulter les informations sur les services afin de découvrir les fonctionnalités proposées par le système.	Moyenne	2 J	Consultation des services
2	Visiteur	En tant que visiteur, je peux créer un compte afin d'accéder aux fonctionnalités du système.	Haute	4 J	Inscription
3	Résident	En tant que résident, je peux m'authentifier afin d'accéder à mon espace personnel.	Haute	3 J	Authentification
4	Visiteur	En tant que visiteur, je peux souscrire à un abonnement afin d'activer les services du système.	Haute	5 J	Gestion des abonnements

Le tableau 3.1 présente les différentes user stories retenues pour le Sprint 1, ainsi que les fonctionnalités associées à chaque besoin fonctionnel. Ce Sprint se concentre principalement sur la découverte de la plateforme, la gestion des accès utilisateurs et la mise en place du processus de souscription aux abonnements. Les fonctionnalités développées permettent au visiteur de consulter les services proposés par Smart House, de créer un compte personnel et de s'authentifier de manière sécurisée pour accéder à son espace utilisateur. Le système offre également la possibilité de consulter les différentes offres d'abonnement et de choisir celle qui correspond le mieux aux besoins de l'utilisateur. Par ailleurs, ce Sprint vise à établir une base fonctionnelle solide pour la suite du projet, notamment via le développement des interfaces essentielles, la validation des données utilisateurs et l'intégration d'un mécanisme d'authentification sécurisé. Ces éléments constituent une étape clé pour garantir une expérience utilisateur fluide et assurer un accès sécurisé aux services de la plateforme.

3.3 Analyse

Cette section présente l'analyse des fonctionnalités réalisées durant le Sprint 1 du projet Smart House. Elle met en avant les principales interactions entre l'utilisateur et le système à travers le diagramme de cas d'utilisation et la description textuelle du processus de souscription à un abonnement. Cette analyse facilite la compréhension du fonctionnement du système et la définition des scénarios liés à l'authentification, à la consultation des services et à la gestion des abonnements.

3.3.1 Diagramme de cas d'utilisation du Sprint 1

La figure ci-dessous représente le diagramme des cas d'utilisation du Sprint 1 :

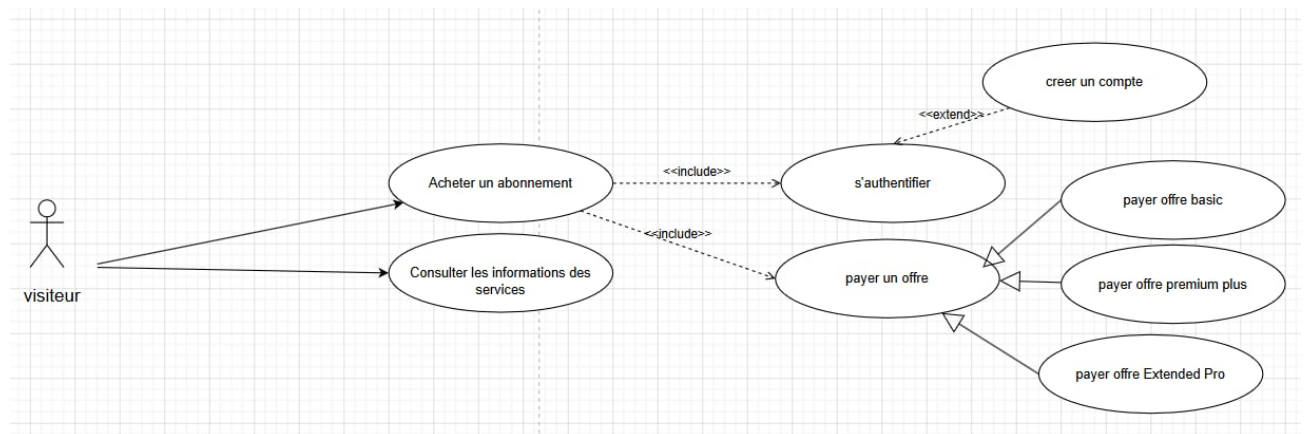


FIGURE 3.1 – Diagramme de cas d'utilisation du Sprint 1

Le diagramme de cas d'utilisation du Sprint 1, présenté ci-dessus, illustre les principales interactions entre le visiteur et le système Smart House lors de la phase de découverte et d'accès à la plateforme. Il met en lumière les fonctionnalités essentielles proposées au visiteur, notamment la consultation des informations sur les services et la souscription à un abonnement. Pour acheter un abonnement, le visiteur doit d'abord s'authentifier. S'il ne possède pas encore de compte, il peut utiliser la fonctionnalité de création de compte. Une fois authentifié, l'utilisateur peut procéder au paiement de l'offre choisie parmi les différentes options disponibles, telles que Basic, Premium Plus ou Extended Pro. Ce diagramme offre ainsi une représentation claire du parcours utilisateur et des relations entre les différents cas d'utilisation du Sprint 1.

3.3.2 Description textuelle

Le tableau suivant présente la description textuelle du cas d'utilisation de l'item « Permettre la sélection et la gestion des offres d'abonnement ».

TABLE 3.2 – Description textuelle du cas d'utilisation numéro 1

Élément	Description
Titre	Souscrire à un abonnement
Acteur	Visiteur
Description	Permettre à l'utilisateur de choisir une offre d'abonnement et de l'activer afin d'accéder aux fonctionnalités du système.
Pré-condition	Le visiteur doit être authentifié dans le système.
Postconditions	L'abonnement est activé et les services deviennent accessibles à l'utilisateur.
Scénario nominal	1. Le visiteur accède à la page des abonnements. 2. Le système affiche les différentes offres disponibles. 3. Le visiteur sélectionne une offre d'abonnement. 4. Le système affiche les informations de paiement. 5. Le visiteur valide le paiement. 6. Le système confirme l'activation de l'abonnement.
Scénario alternatif	1. À l'étape 5 du scénario nominal, le paiement échoue ou les informations saisies sont invalides. 2. Le système affiche un message d'erreur. 3. L'utilisateur peut réessayer le paiement ou choisir une autre méthode de paiement. 4. Le cas d'utilisation reprend à l'étape 5 du scénario nominal.

Ce tableau présente une description détaillée du cas d'utilisation « Souscrire à un abonnement ». Il met en avant les différentes étapes permettant au résident de choisir une offre et d'activer les services du système Smart House. Le tableau précise aussi les pré-conditions, les résultats attendus après le processus, ainsi que le scénario nominal décrivant le déroulement normal de la souscription. Par ailleurs, il inclut les scénarios alternatifs liés aux erreurs de paiement pour garantir une gestion fiable et sécurisée du processus d'abonnement.

3.4 Conception : Diagramme de classes du Sprint 1

Cette section présente la phase de conception du Sprint 1 à travers le diagramme de classes. Cette modélisation illustre la structure statique du système en identifiant les

principales classes, leurs attributs, leurs méthodes, ainsi que les relations entre elles. Elle constitue une étape clé pour organiser efficacement le développement des fonctionnalités liées à l'accès à la plateforme et à la gestion des abonnements.

Diagramme de classes du Sprint 1

Le diagramme de classes présenté ci-dessous illustre les principales entités du Sprint 1, à savoir les classes **Membre**, **Résident** et **Abonnement**. La classe **Membre** regroupe les informations nécessaires à l'authentification et à la gestion du profil, ainsi que les opérations de connexion, de déconnexion et de mise à jour des informations personnelles.

La classe **Résident** hérite de la classe **Membre** et dispose de fonctionnalités supplémentaires lui permettant de gérer son abonnement, notamment son changement et son renouvellement.

La classe **Abonnement** assure la gestion du cycle de vie des abonnements à travers les opérations de création, d'activation, de renouvellement, d'annulation et de vérification de l'expiration. Une association est établie entre les membres et l'abonnement, permettant de représenter les utilisateurs rattachés à un même abonnement.

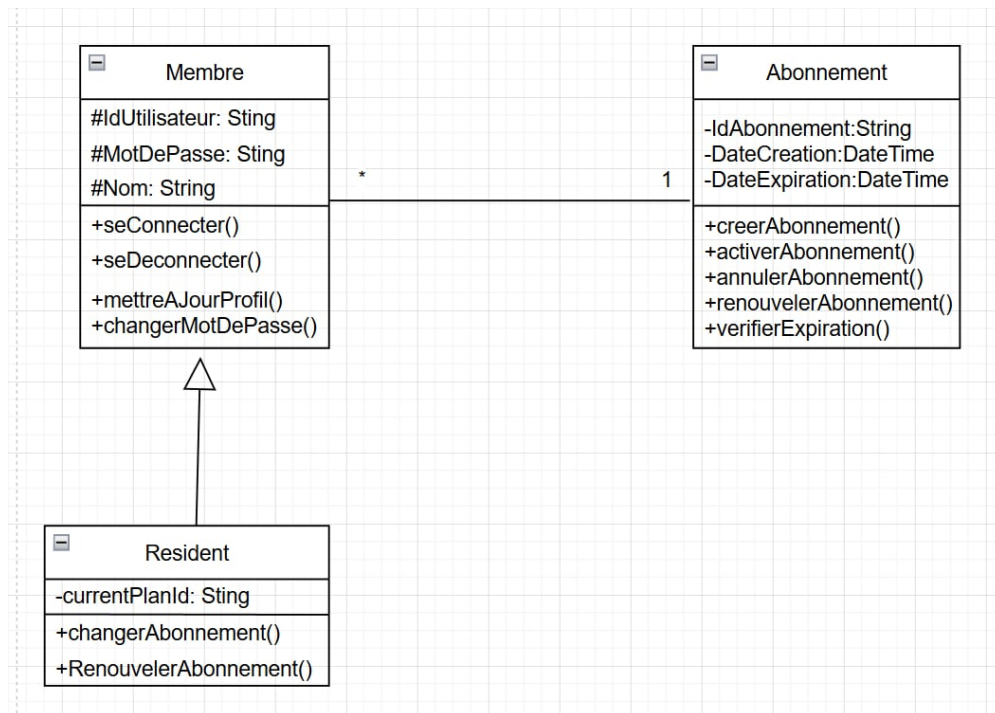


FIGURE 3.2 – Diagramme de classes du Sprint1

Cette modélisation permet de structurer les données relatives à la gestion des utilisateurs et des abonnements de manière claire et cohérente. Elle facilite l'implémentation des fonctionnalités développées au cours du Sprint 1 et constitue une base solide pour l'intégration des autres composantes du système Smart House.

3.5 Réalisation et tests

Cette section présente les différentes phases de réalisation du Sprint 1 ainsi que les tests menés pour vérifier le bon fonctionnement des fonctionnalités développées. Elle décrit l'implémentation de l'architecture de la plateforme, la mise en œuvre de la structure de la base de données MongoDB à partir du modèle de conception et l'intégration des mécanismes d'authentification et de sécurité. Cette partie met également en avant les principales interfaces développées, conçues pour offrir aux utilisateurs une navigation intuitive, sécurisée et adaptée aux besoins de la plateforme Smart House.

3.5.1 Planification des sprints

Afin d'assurer une gestion efficace du projet Smart House, nous avons adopté le framework Scrum. La planification des différents sprints a été réalisée à l'aide de l'outil Jira, permettant de structurer le projet en plusieurs fonctionnalités majeures regroupées sous forme d'épics.

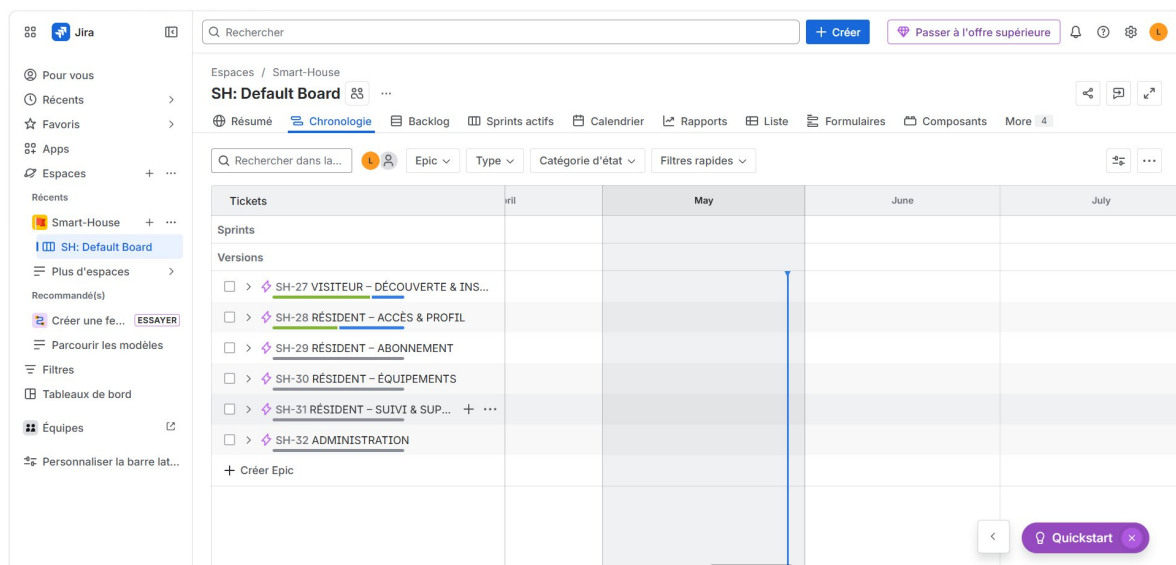


FIGURE 3.3 – Planification des sprints sur Jira

La figure 3.3 présente la planification globale du projet. Celui-ci a été organisé en six épics principaux : *Visiteur – Découverte et Inscription*, *Résident – Accès et Profil*, *Résident – Abonnement*, *Résident – Équipements*, *Résident – Suivi et Supervision* ainsi que *Administration*. Chaque épique regroupe un ensemble cohérent de fonctionnalités répondant aux besoins des différents acteurs du système.

Cette organisation a permis de découper le projet en plusieurs sprints successifs, facilitant ainsi la gestion des tâches, la définition des priorités et le suivi de l'avancement des développements. Grâce à l'utilisation de Jira, il a été possible de visualiser l'évolution

du projet au fil du temps, d'assurer une meilleure coordination des activités et de valider progressivement les fonctionnalités développées.

Cette planification a contribué à garantir le respect des objectifs fixés tout en favorisant une intégration progressive et maîtrisée des différentes composantes de la plateforme Smart House.

3.5.2 Préparation de l'environnement de développement

Avant de commencer l'implémentation des fonctionnalités du Sprint 1, nous avons préparé l'environnement de développement nécessaire à la réalisation de la plateforme Smart House. Cette étape a permis d'organiser le code source, de configurer les services techniques du projet et de garantir un environnement stable pour le développement frontend, backend et base de données.

La figure 3.4 présente l'organisation des différents dépôts GitLab utilisés dans le cadre du projet *Smart House*. Afin de garantir une meilleure séparation des responsabilités et de faciliter le travail collaboratif, le projet a été réparti en trois dépôts distincts. Le dépôt *sh-api* regroupe l'ensemble du code source du backend développé avec NestJS, responsable de la gestion des API et de la logique métier. Le dépôt *sh-client* contient l'application frontend développée avec Angular, permettant aux utilisateurs d'interagir avec le système à travers une interface web moderne. Enfin, le dépôt *sh-dockers* rassemble les fichiers de configuration Docker nécessaires au déploiement et à l'orchestration des différents services du projet. Cette organisation favorise la maintenabilité, la modularité et la gestion efficace des différentes composantes du système.

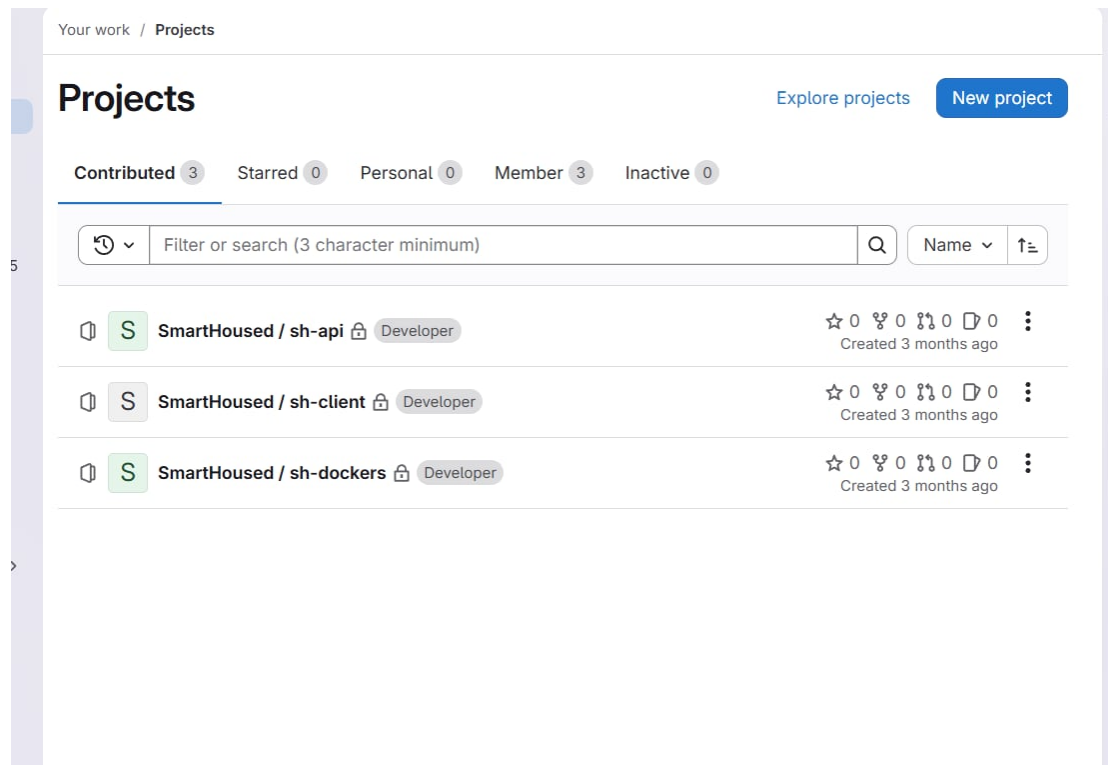


FIGURE 3.4 – Organisation des dépôts GitLab du projet Smart House

Afin d'assurer un déploiement cohérent et reproductible de l'ensemble des services, nous avons adopté la technologie Docker. La figure suivante présente l'organisation du projet Docker ainsi que les différents conteneurs utilisés pour exécuter les composants de la plateforme Smart House.

La figure 3.5 illustre l'exécution de l'application Smart House dans un environnement conteneurisé à l'aide de Docker. Cette approche permet d'isoler les différents services du système dans des conteneurs indépendants tout en facilitant leur déploiement et leur maintenance. L'architecture est composée de trois conteneurs principaux : *sh-client*, qui héberge l'application frontend développée avec Angular, *sh-api*, qui exécute les services backend développés avec NestJS, et *sh-db*, qui assure le stockage des données à travers MongoDB. Grâce à Docker, l'ensemble des composants peut être lancé, configuré et supervisé de manière uniforme, garantissant ainsi la portabilité et la cohérence de l'environnement d'exécution.

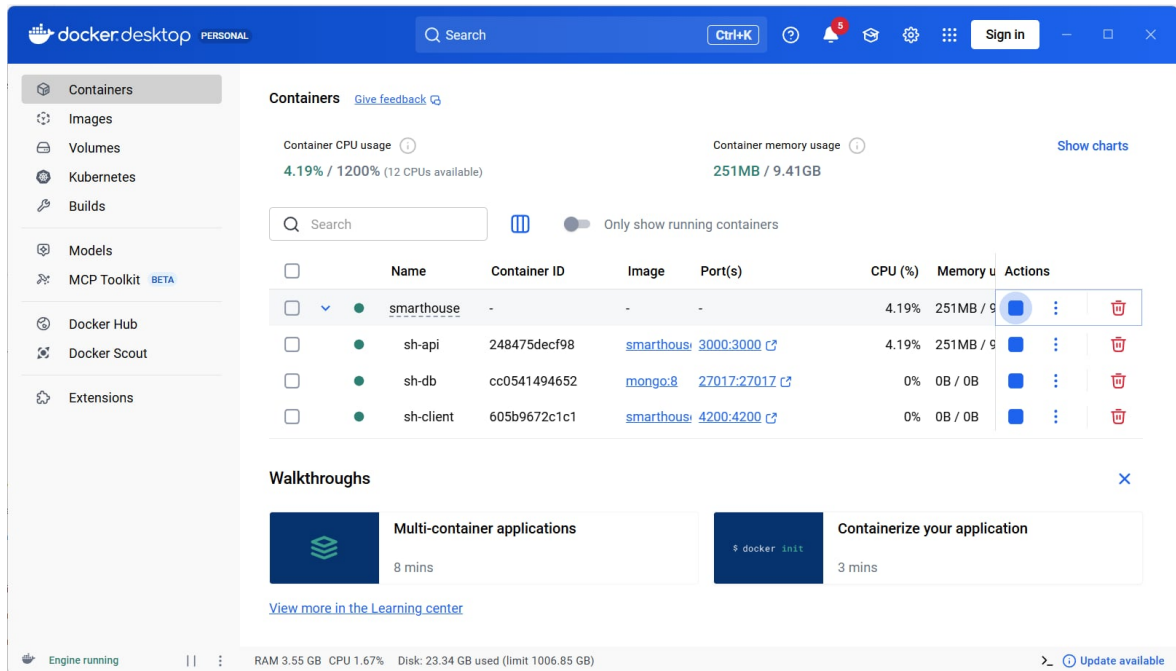


FIGURE 3.5 – Exécution des conteneurs Docker du projet Smart House

Cette conteneurisation constitue une étape essentielle dans la mise en œuvre de l'architecture du projet. Afin de mieux comprendre les interactions entre les différents services déployés, la figure suivante présente l'architecture globale du système Docker ainsi que les relations entre les conteneurs composant la plateforme Smart House.

3.5.3 Passage du diagramme de classes vers la base de données MongoDB

Après la conception du diagramme de classes du système Smart House, une étape importante consiste à transformer le modèle relationnel orienté objet vers une base de données NoSQL MongoDB. Contrairement aux bases de données relationnelles classiques qui utilisent des tables liées par des clés étrangères, MongoDB repose sur une structure orientée documents stockés au format BSON (Binary JSON).

Dans notre système, les principales entités telles que **Users**, **Subscriptions**, **Complaints**, **Devices** et **Notifications** ont été transformées en collections MongoDB. Chaque document représente une instance d'une classe du diagramme UML.

a. Transformation des classes en collections MongoDB

La classe **Resident** du diagramme de classes a été convertie vers la collection **users**. Cette collection contient les informations relatives aux utilisateurs telles que le nom, l'email, le mot de passe chiffré, le rôle ainsi que les informations liées à l'abonnement Stripe.

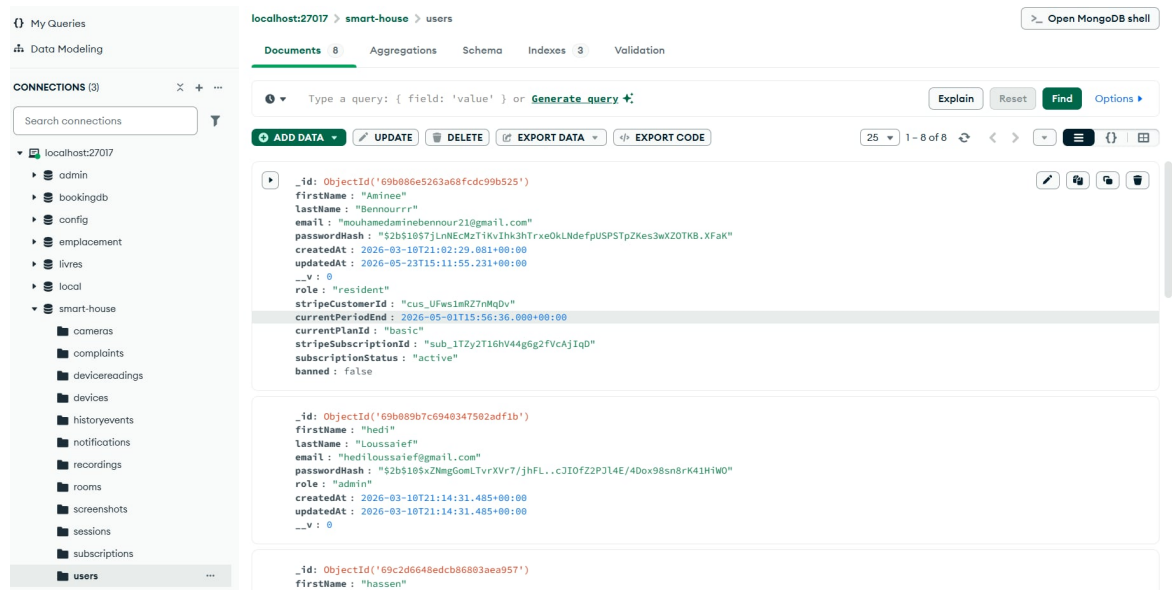


FIGURE 3.6 – Collection MongoDB des utilisateurs

De même, la classe **Abonnement** a été transformée vers la collection **subscriptions** contenant les informations des abonnements comme l'identifiant Stripe, le plan choisi, le statut de l'abonnement, la devise, le montant ainsi que les dates de création et de mise à jour.

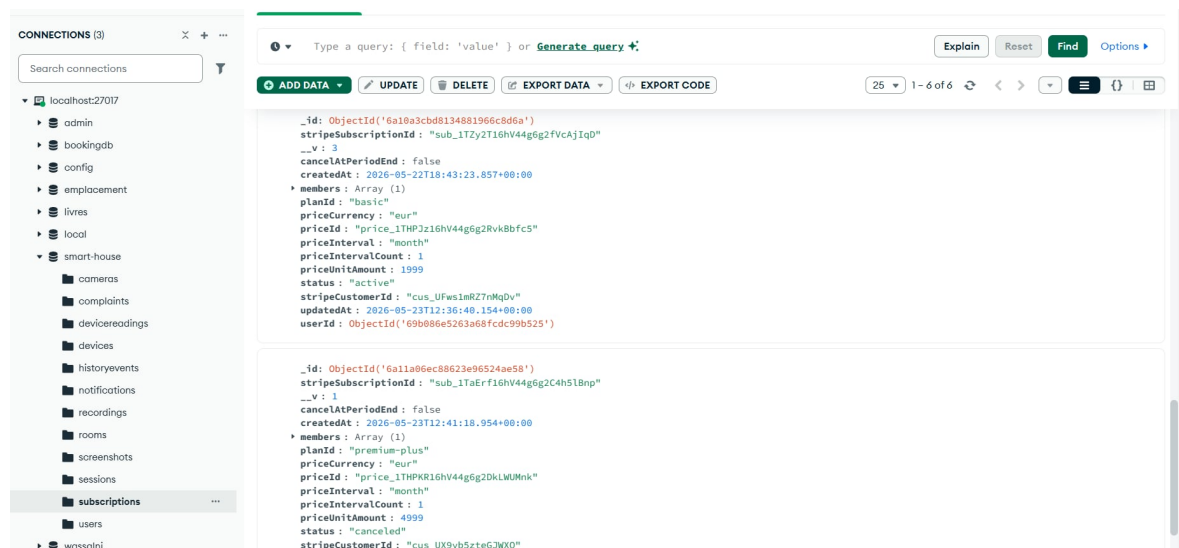


FIGURE 3.7 – Collection MongoDB des abonnements

b.Normalisation et dénormalisation dans MongoDB

Lors de la conception de la base de données MongoDB, deux approches principales peuvent être utilisées : la **normalisation** et la **dénormalisation**.

La normalisation La normalisation consiste à séparer les données dans plusieurs collections afin d'éviter la duplication des informations. Dans cette approche, les relations

entre les documents sont réalisées à l'aide des identifiants (*ObjectId*).

Dans notre projet, les utilisateurs et les abonnements sont stockés dans deux collections séparées :

- Collection **users**
- Collection **subscriptions**

Chaque abonnement contient une référence vers l'utilisateur à travers le champ **userId**. Cette approche permet :

- de réduire la redondance des données ;
- d'améliorer la cohérence des informations ;
- de faciliter la mise à jour des documents.

Cependant, cette méthode nécessite parfois plusieurs requêtes pour récupérer les données liées, ce qui peut augmenter le temps d'accès aux informations.

La dénormalisation La dénormalisation consiste à intégrer certaines données directement dans un même document afin d'optimiser les performances de lecture. MongoDB favorise souvent cette approche pour améliorer la rapidité des accès.

Dans notre système Smart House, certaines informations de l'abonnement sont également stockées dans la collection **users**, comme :

- **currentPlanId**
- **subscriptionStatus**
- **currentPeriodEnd**

Cette stratégie permet :

- un accès rapide aux informations essentielles ;
- une réduction du nombre de requêtes ;
- une amélioration des performances de consultation.

Néanmoins, la dénormalisation peut provoquer une duplication des données et rendre les mises à jour plus complexes.

c.Choix adopté dans le projet

Dans ce projet Smart House, une approche hybride combinant la normalisation et la dénormalisation a été adoptée. Les données principales des abonnements sont stockées dans la collection **subscriptions**, tandis que certaines informations importantes sont copiées dans la collection **users** afin d'accélérer les opérations les plus fréquentes.

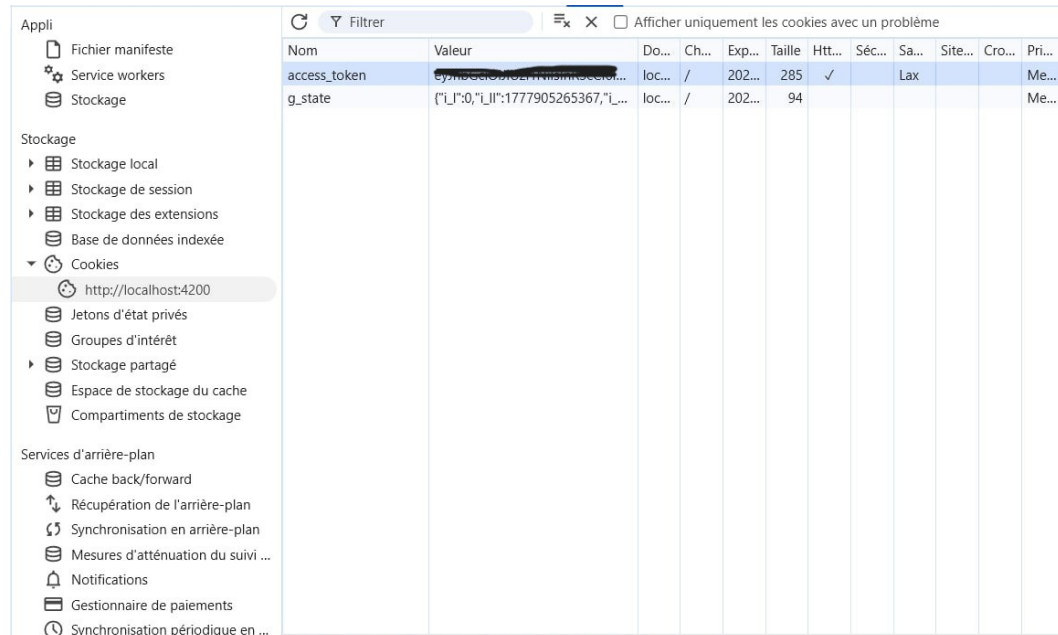
Cette solution permet d'obtenir un bon compromis entre :

- la cohérence des données ;

- les performances du système ;
- la flexibilité offerte par MongoDB.

3.5.4 Architecture de sécurité et gestion de l'authentification

La réalisation du Sprint 1 repose sur une architecture client–serveur garantissant une gestion sécurisée et organisée des fonctionnalités d'accès au système. Le frontend, développé avec Angular, offre les interfaces utilisateur pour la consultation des services, l'inscription, l'authentification et la gestion des abonnements. Le backend, implémenté avec NestJS, expose des API REST pour traiter les requêtes client, incluant la création de comptes, la validation des données, l'authentification et la gestion des abonnements. Pour sécuriser les échanges entre client et serveur, le système utilise le mécanisme JWT (JSON Web Token). Lors d'une authentification réussie, le serveur génère un jeton JWT stocké dans un cookie sécurisé côté client, permettant de maintenir la session et d'envoyer automatiquement les informations d'authentification pour les requêtes suivantes. Les mots de passe sont stockés de manière sécurisée dans une base de données MongoDB après un chiffrement par hachage. Cette architecture assure une séparation claire des couches du système, facilitant la maintenance, l'évolutivité et la sécurisation de l'application.



Nom	Valeur	Do...	Ch...	Exp...	Taille	Http...	Séc...	Sa...	Site...	Cro...	Pri...
access_token	[redacted]	loc...	/	202...	285	✓		Lax			Me...
g_state	[{"i_l":0,"i_ll":"1777905265367","i_...	loc...	/	202...	94						Me...

FIGURE 3.8 – Stockage du jeton JWT dans un cookie côté client

La figure ci-dessus illustre le stockage du jeton JWT dans un cookie au niveau du navigateur. Ce mécanisme permet de conserver la session utilisateur et d'envoyer automatiquement le jeton lors des requêtes vers le serveur, assurant ainsi une authentification sécurisée.

3.5.5 Architecture du système

La figure ?? présente l'architecture globale adoptée pour le Sprint 1. Elle montre la communication entre le frontend développé avec Angular, le backend implémenté avec NestJS, et la base de données MongoDB. L'architecture illustre également le mécanisme d'authentification sécurisé basé sur JWT ainsi que la gestion des sessions utilisateur.

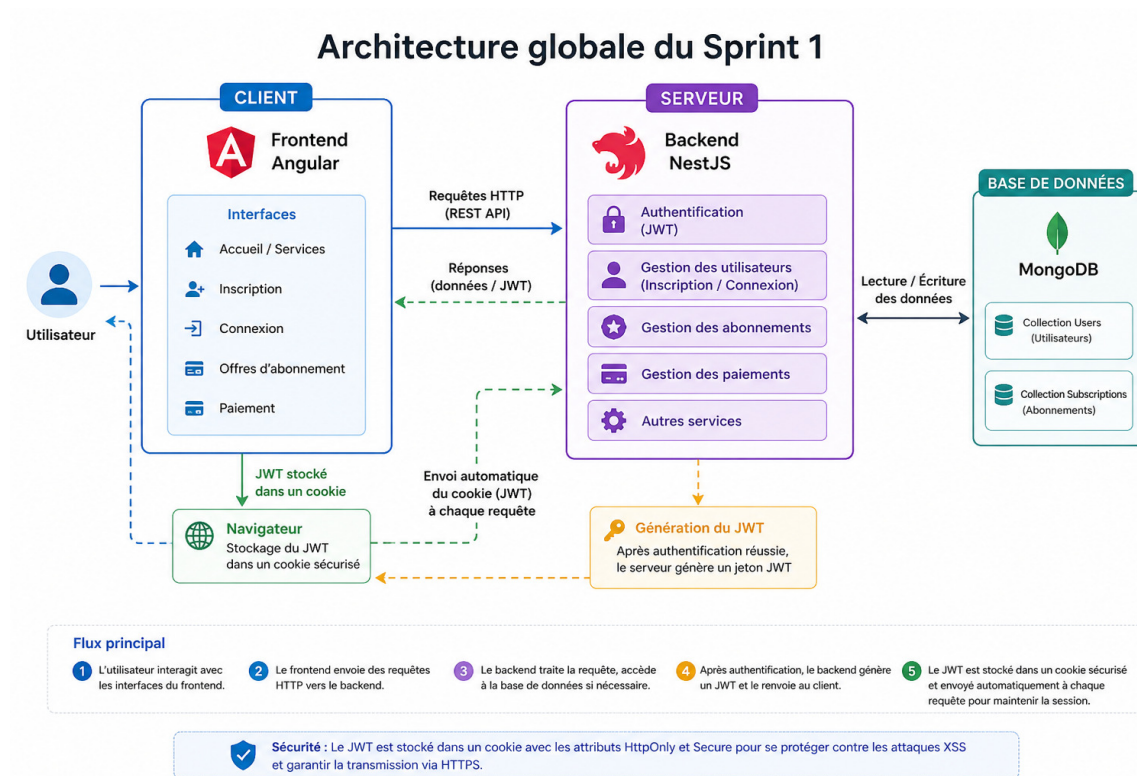


FIGURE 3.9 – Architecture globale du Sprint 1

L'architecture adoptée repose sur un modèle client-serveur utilisant des API REST. Le système combine un mécanisme d'authentification basé sur JWT et la gestion de sessions. Lors de l'authentification, un jeton JWT est généré et stocké dans un cookie sécurisé côté client. Parallèlement, une session est enregistrée dans la base de données MongoDB pour gérer l'état de connexion de l'utilisateur et renforcer le contrôle des accès. Cette approche hybride offre la flexibilité du JWT tout en conservant un contrôle serveur via les sessions, notamment pour la gestion de la déconnexion et la sécurité.

3.5.6 Présentation des Interfaces réalisées

Cette section présente les principales interfaces développées dans le cadre du Sprint 1. Elles illustrent le parcours suivi par le visiteur, depuis la découverte de la plateforme jusqu'à la souscription à une offre d'abonnement.

a. Interface d'accueil

Cette interface constitue l'entrée principale vers le système Smart House. Elle offre aux utilisateurs autorisés un accès sûr à leurs sections privées ainsi qu'aux diverses options disponibles dans le dispositif.

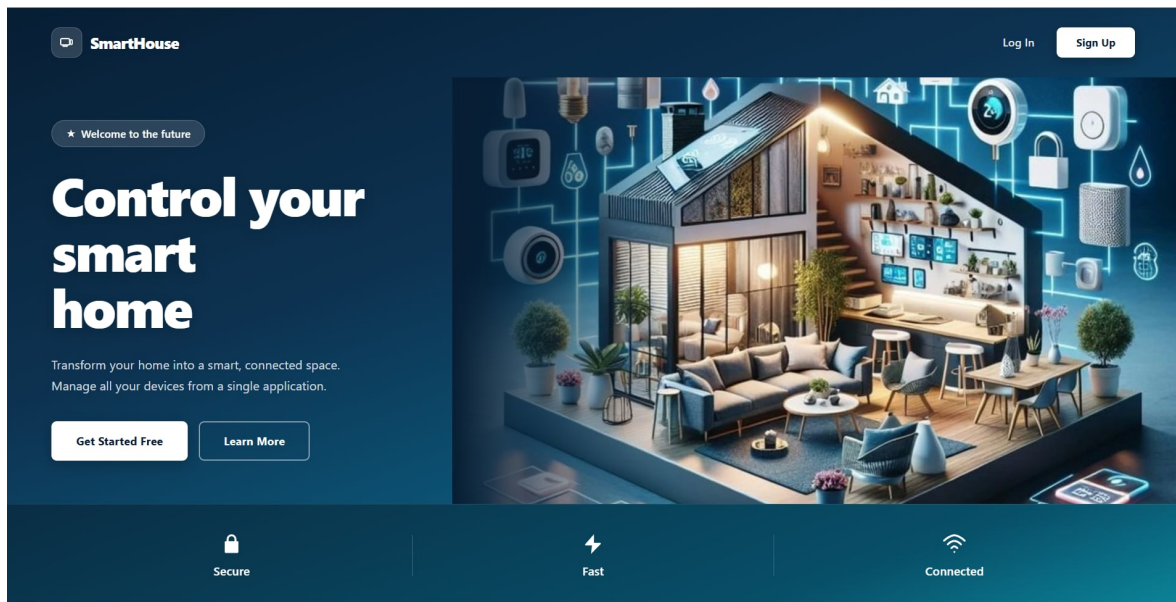


FIGURE 3.10 – Interface d'accueil de l'application Smart House

L'interface d'accueil est le point d'entrée principal de l'application Smart House. Elle offre au visiteur une présentation claire et structurée de la plateforme, mettant en avant ses fonctionnalités et services. Cette page joue un rôle central dans l'expérience utilisateur en orientant le visiteur vers les différentes sections de l'application. Elle donne accès aux fonctionnalités clés telles que la connexion, l'inscription et la consultation des offres disponibles. La conception privilégie une navigation intuitive, guidant l'utilisateur de manière progressive à travers les fonctionnalités du système.

b.Interface d'inscription

Cette page détaille les divers forfaits d'abonnement offerts par la plateforme Smart House. Elle guide l'utilisateur dans la sélection de l'offre qui correspond le mieux à ses exigences, selon les prestations et options incluses.

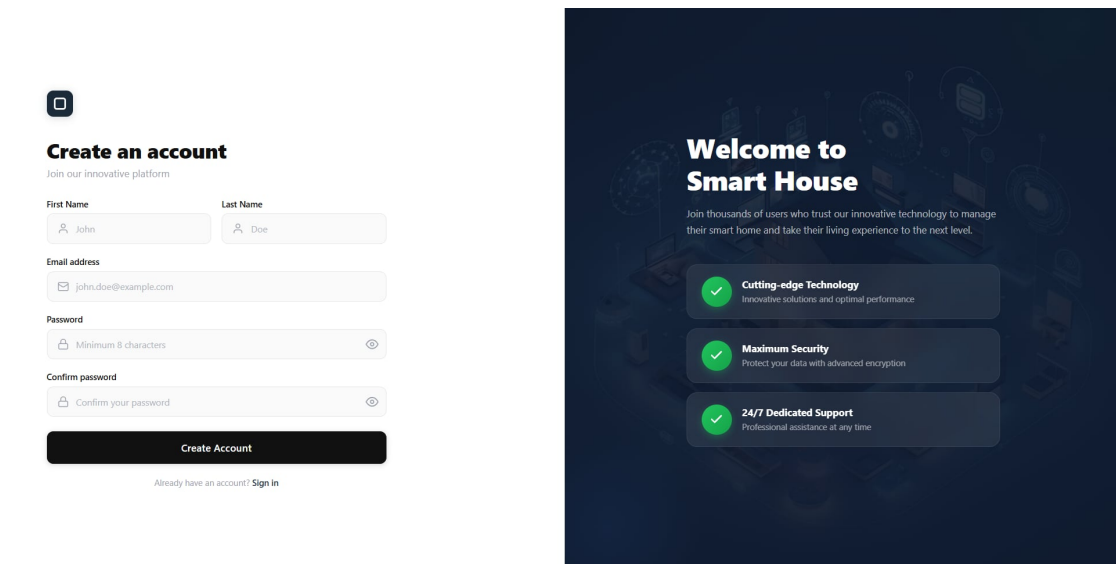


FIGURE 3.11 – Interface de création de compte

L'interface d'inscription permet aux nouveaux visiteurs de créer un compte en saisissant leurs informations personnelles : prénom, nom, adresse e-mail et mot de passe. Un mécanisme de validation des données intégré au formulaire garantit l'intégrité et la conformité des informations fournies. Cette étape est essentielle pour accéder à toutes les fonctionnalités de la plateforme Smart House. Une fois l'inscription réussie, l'utilisateur est redirigé vers l'interface de connexion, où il peut accéder à son espace personnel et souscrire à un abonnement.

c.Interface de connexion

Ce portail autorise les nouveaux venus à intégrer l'univers Smart House par la création d'un profil individuel. C'est une démarche primordiale pour profiter des prestations connectées et faire usage des outils de pilotage de l'habitation intelligente.

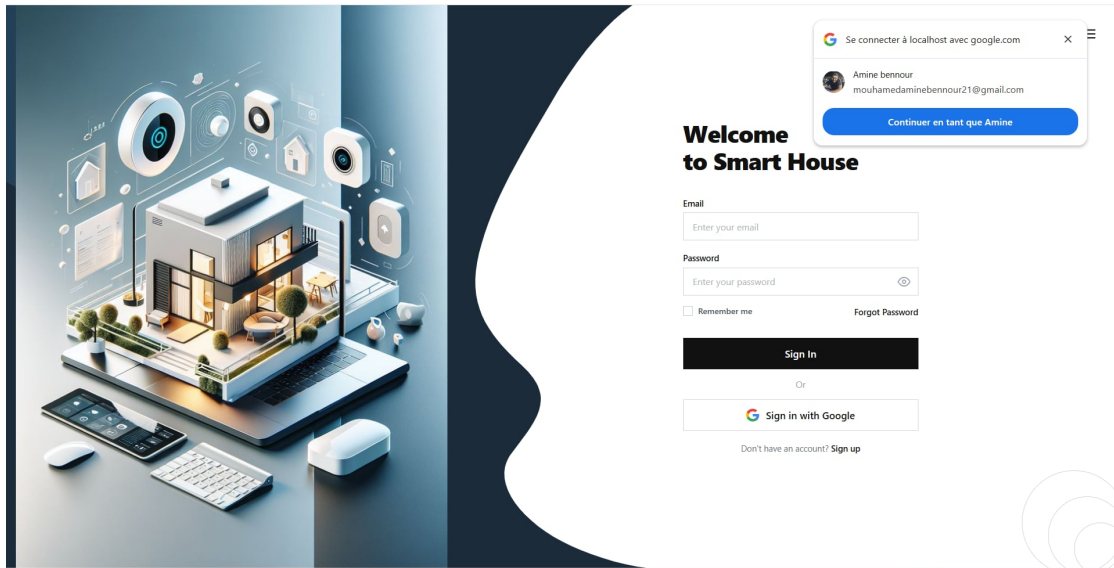


FIGURE 3.12 – Interface de connexion

L'interface de connexion permet à l'utilisateur de s'authentifier sur la plateforme en saisissant son adresse e-mail et son mot de passe. Cette étape assure un accès sécurisé à son espace personnel, condition indispensable à l'utilisation des fonctionnalités du système. Pour améliorer l'expérience utilisateur, l'application propose également une authentification rapide via Google, utilisant le protocole OAuth 2.0. Ce mécanisme permet de se connecter directement avec un compte Gmail, sans créer de compte distinct, tout en garantissant un haut niveau de sécurité. Un lien de redirection vers l'interface d'inscription est aussi disponible pour les visiteurs ne disposant pas encore d'un compte.

Après la tentative d'authentification, le système vérifie les informations saisies par l'utilisateur. En cas d'erreur dans l'adresse e-mail ou le mot de passe, un message d'alerte est affiché afin d'informer l'utilisateur de l'échec de la connexion et de l'inviter à corriger ses identifiants.

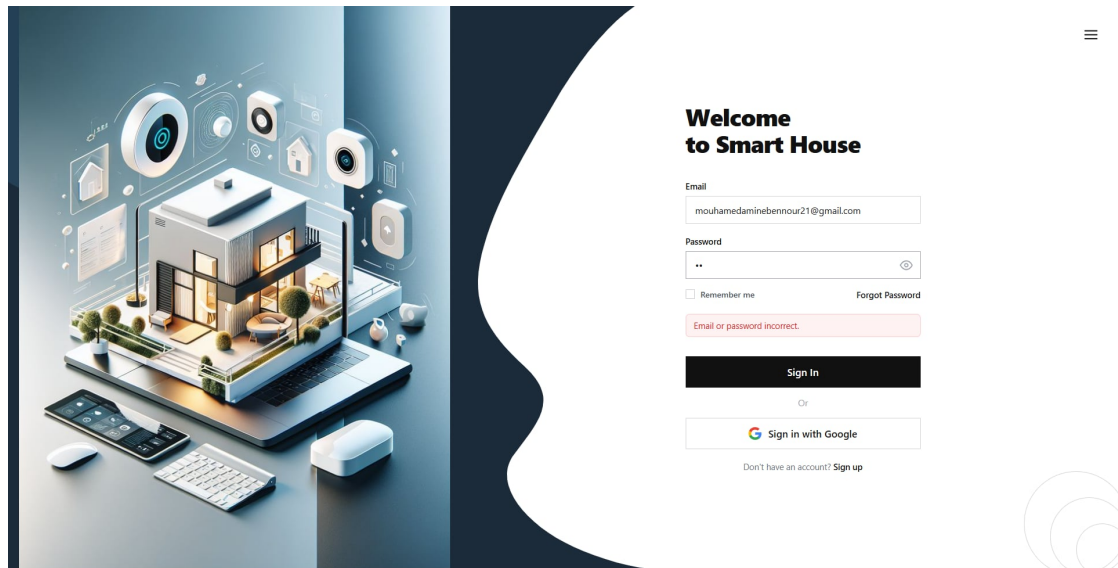


FIGURE 3.13 – Échec de connexion

La figure 3.13 illustre un scénario d'échec de connexion. Lorsque les informations d'authentification saisies ne correspondent à aucun compte enregistré dans la base de données, l'application affiche un message d'erreur indiquant que l'adresse e-mail ou le mot de passe est incorrect. Ce mécanisme permet de renforcer la sécurité du système en empêchant les accès non autorisés tout en guidant l'utilisateur vers une nouvelle tentative de connexion. L'utilisateur peut alors vérifier ses informations, réinitialiser son mot de passe via l'option *Forgot Password* ou créer un nouveau compte s'il n'en possède pas.

d. Interface des offres d'abonnement

Cette interface intervient au moment de clore la procédure d'adhésion. Elle offre à l'utilisateur la possibilité de régler ses achats en toute sécurité pour ainsi activer l'abonnement choisi et bénéficier des prestations de la plateforme Smart House.

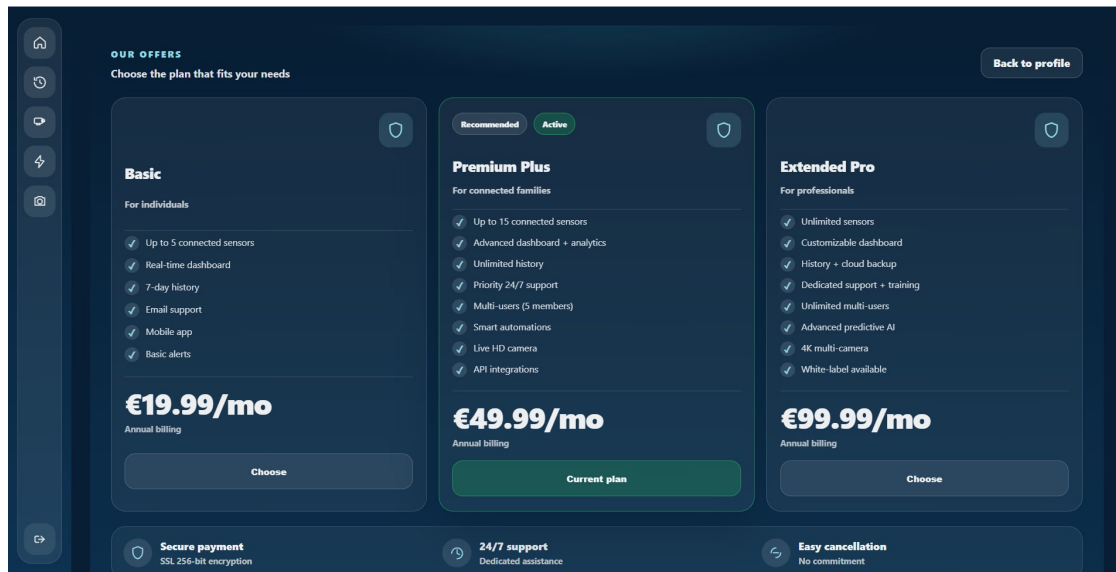


FIGURE 3.14 – Interface des offres d’abonnement

L’interface des offres d’abonnement présente l’ensemble des formules disponibles au sein de la plateforme Smart House. Elle permet au visiteur de consulter en détail les caractéristiques de chaque offre, de comparer les fonctionnalités incluses et d’évaluer les tarifs correspondants. Cette interface est conçue pour faciliter la prise de décision de visiteur en lui offrant une vue comparative claire et structurée des différentes formules. Une fois l’offre sélectionnée, le visiteur est redirigé vers l’interface de paiement pour finaliser sa souscription.

e.Interface de paiement

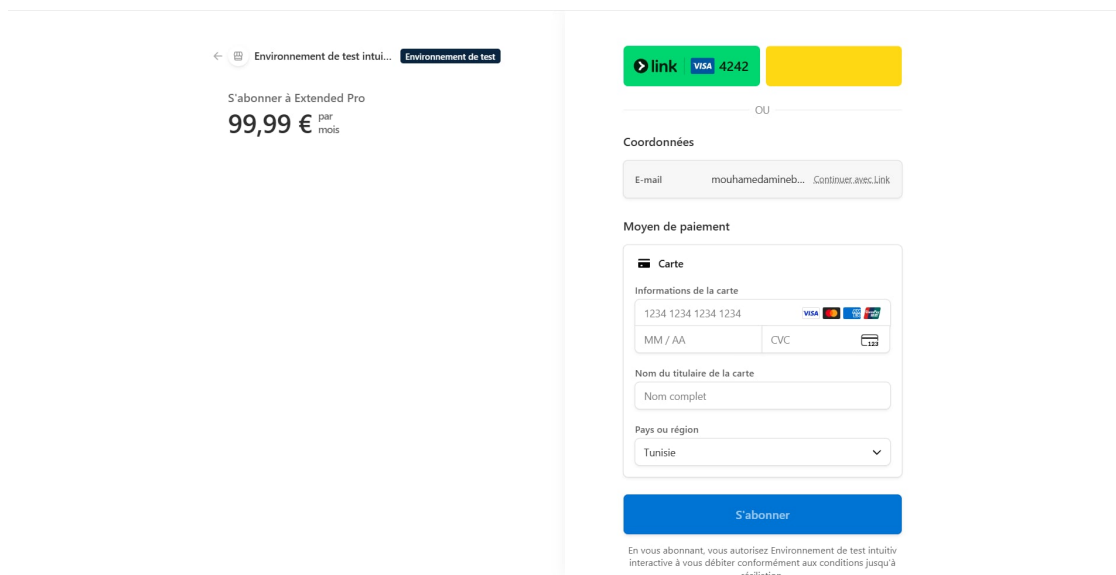


FIGURE 3.15 – Interface de paiement

L’interface de paiement constitue la dernière étape du processus de souscription.

Elle permet à l'utilisateur de finaliser son abonnement en renseignant les informations nécessaires au traitement du paiement, notamment ses coordonnées et le moyen de paiement choisi. À l'issue de cette étape, l'abonnement sélectionné est activé et l'utilisateur dispose d'un accès complet aux services Smart House correspondant à sa formule. L'ensemble de ces interfaces illustre ainsi le parcours utilisateur dans sa globalité, depuis la découverte de la plateforme jusqu'à l'activation des services, en passant par l'authentification, la création de compte et la souscription à une offre.

Toutefois, dans certains cas, le paiement peut échouer lorsque les informations bancaires saisies sont incorrectes ou lorsque la transaction est refusée par l'établissement bancaire. Afin d'informer l'utilisateur de cette situation, le système affiche un message d'erreur explicite lui permettant de corriger les données saisies et de réessayer l'opération.

f. Interface d'échec de paiement

← Environnement de test intuitif... Environnement de test

S'abonner à Premium Plus
49,99 € par mois

link VISA 4242 amazon pay

OU

Coordonnées

E-mail mouhamedamineb... Continuer avec Link

Moyen de paiement

Carte

Informations de la carte

1234 1234 1234 1234

03 / 29 123

Votre numéro de carte n'est pas valide.

Nom du titulaire de la carte

Amine

Pays ou région

Tunisie

S'abonner

En vous abonnant, vous autorisez Environnement de test intuitif

FIGURE 3.16 – Interface d'échec de paiement

Cette interface apparaît lorsqu'une tentative de paiement ne peut pas être validée. Dans l'exemple illustré, le numéro de carte bancaire saisi est invalide, ce qui empêche la finalisation de la transaction. Un message d'erreur est affiché directement sous le champ concerné afin d'indiquer clairement la cause du problème à l'utilisateur. Cette validation permet d'améliorer l'expérience utilisateur en facilitant l'identification et la correction des erreurs avant de soumettre une nouvelle demande de paiement.

Conclusion

Dans ce chapitre, nous avons présenté les différentes étapes du Sprint 1, dédié à la découverte et à l'accès à la plateforme. Nous avons d'abord exposé le Sprint backlog et les besoins fonctionnels retenus, puis analysé ces besoins à travers le diagramme de cas d'utilisation et la description du cas d'utilisation relatif à la souscription à un abonnement. Nous avons ensuite présenté la conception du Sprint via le diagramme de classes, ainsi que l'architecture adoptée pour assurer une gestion sécurisée de l'authentification et des sessions utilisateur. Enfin, nous avons illustré la réalisation des principales fonctionnalités à travers les interfaces développées, de la page d'accueil au paiement de l'abonnement. Ce Sprint a ainsi permis de mettre en place les fonctionnalités de base du système Smart House, constituant une étape essentielle avant d'aborder, dans le chapitre suivant, la gestion des équipements IoT.

Chapitre 4

Sprint 2 : Gestion des équipements IoT

Introduction

Ce chapitre porte sur le Sprint2 du projet Smart House, dont le but est de donner au résident la capacité de contrôler les appareils connectés de son logement intelligent. Il englobe la surveillance des capteurs ambiants et des caméras, ainsi que la visualisation des flux vidéo instantanés et des clichés capturés par ces systèmes, afin d'accroître la sûreté et la vigilance de l'habitation.

4.1 Sprint Goal

Pour ce sprint, notre mission première est la création d'une plateforme de pilotage pour vos dispositifs IoT, offrant au résident la possibilité de surveiller les capteurs et les caméras de sa demeure connectée. Cette période de travail se focalise aussi sur la capacité à visionner les flux vidéo en direct et à accéder aux enregistrements vidéo et aux clichés, dans le but d'améliorer la sûreté, l'observation et l'automatisation de la résidence intelligente.

4.2 Sprint Backlog

Durant la deuxième phase de développement (Sprint2), nous mettons l'accent sur le contrôle des dispositifs connectés de l'Internet des Objets intégrés à notre solution de maison intelligente. L'objectif central de ce Sprint est de donner à l'habitant la capacité de suivre de près les différents appareils connectés, tels que les capteurs et les dispositifs de vidéosurveillance. Les fonctionnalités que nous élaborons ont aussi pour but d'améliorer l'expérience utilisateur en permettant la visualisation des flux vidéo, tout comme l'accès aux séquences et enregistrements déjà réalisés.

Le tableau suivant représente les user stories du Sprint2 :

TABLE 4.1 – Sprint Backlog - Gestion des équipements IoT

ID	Acteur	User Story	Priorité	Estimation	Fonctionnalité
5	Résident ou Membre	En tant que résident, je peux gérer les capteurs.	Haute	7 J	Gestion des équipements IoT
6	Résident ou Membre	En tant que résident, je peux consulter les flux vidéo.	Haute	7 J	Gestion des caméras
7	Résident ou Membre	En tant que résident, je peux consulter les images enregistrées par mes caméras.	Haute	7 J	Consultation des médias

Le tableau ci-dessus liste les diverses user stories retenues pour le Sprint2, ainsi que les fonctionnalités qui correspondent à chaque besoin. Ce Sprint se concentre essentiellement sur la gestion des objets connectés et la surveillance avancée de la maison intelligente. Les fonctionnalités mises en place autorisent le résident à contrôler les capteurs et les caméras, à visionner les flux vidéo instantanément et à consulter les enregistrements visuels et audiovisuels. L’attribution d’une haute priorité à ces aspects démontre leur rôle central dans le fonctionnement d’ensemble du système de maison connectée, car ils garantissent la collecte des données, le suivi constant de l’environnement et l’interaction avec les dispositifs connectés.

4.3 Analyse

Ce chapitre détaille l’examen des fonctionnalités créées au fil du Sprint2 du projet Smart House. Il illustre les interactions variées entre l’occupant et la plateforme à l’aide de schémas de cas d’utilisation, de diagrammes bloc et de leurs descriptions textuelles. Cette examination sert à appréhender plus aisément le mode opératoire du système IoT, ainsi que les diverses tâches relatives au pilotage des appareils connectés et à la supervision intelligente de l’habitat.

4.3.1 Diagramme de cas d’utilisation

La figure suivante présente le diagramme de cas d’utilisation du Sprint2, consacré à la gestion des équipements IoT. Ce diagramme illustre les principales interactions entre le résident et le système, notamment la gestion des équipements, des caméras et des capteurs, ainsi que la consultation de la diffusion des caméras et des images enregistrées.

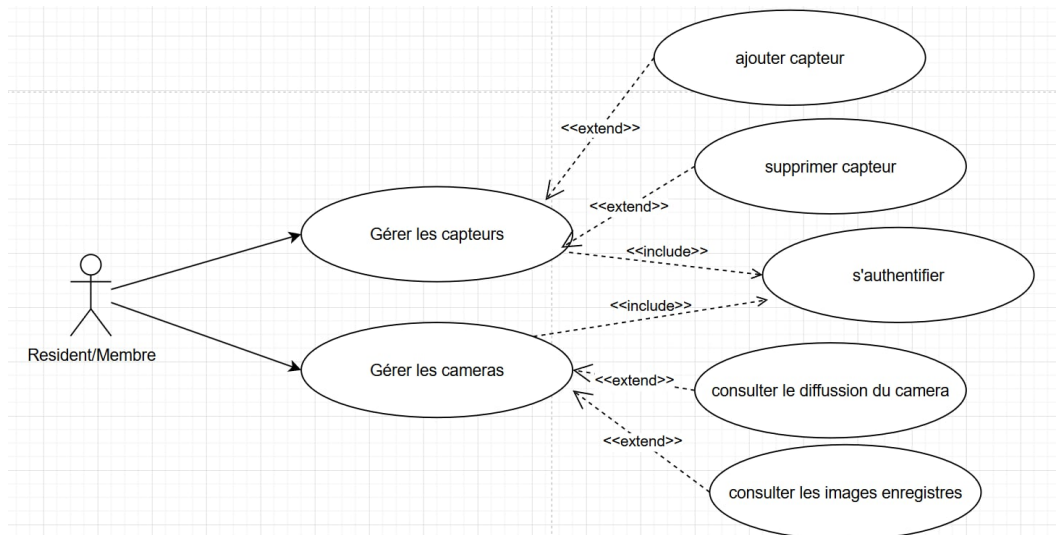


FIGURE 4.1 – Diagramme de cas d'utilisation du Sprint2

La figure ci-dessus illustre le diagramme de cas d'utilisation du Sprint2, consacré à la gestion des équipements IoT dans le système Smart House. Ce diagramme met en évidence les différentes interactions entre le résident et les fonctionnalités proposées par la plateforme afin d'assurer une surveillance efficace et un contrôle intelligent de l'habitation connectée.

À travers ce système, le résident peut gérer les capteurs installés dans la maison en effectuant différentes opérations telles que l'ajout et la suppression des capteurs. Il peut également gérer les caméras de surveillance afin d'assurer le suivi de son domicile à distance. Le système offre aussi la possibilité de consulter la diffusion en direct des caméras ainsi que les images enregistrées pour visualiser les événements précédemment capturés.

Le diagramme montre également que l'accès aux fonctionnalités de gestion des capteurs et des caméras nécessite une authentification préalable de l'utilisateur. De plus, les fonctionnalités « ajouter capteur » et « supprimer capteur » représentent des extensions du cas d'utilisation principal « Gérer les capteurs », tandis que les fonctionnalités « consulter la diffusion du caméra » et « consulter les images enregistrées » étendent le cas d'utilisation « Gérer les caméras ». Cette organisation permet de structurer clairement les différentes opérations liées à la gestion des équipements IoT au sein du système Smart House.

4.3.2 Description textuelle des cas d'utilisation

Ce chapitre rend compte des scénarios d'usage principaux du Sprint2. Il permet de spécifier les échanges entre l'habitant et l'environnement Domotique, en mentionnant les

prérequis, les déroulements courants et les diverses issues envisageables. Ces narrations clarifient le fonctionnement de l'intelligence de la maison et les aptitudes relatives à la supervision des dispositifs connectés et à l'observation avisée des lieux.

a. Gérer les équipements IoT

Le graphique ci-après récapitule la description textuelle du scénario d'utilisation « Administrer les dispositifs IoT ». Ce scénario détaille les échanges fondamentaux entre l'habitant et le système afin de garantir le contrôle des capteurs et des caméras de la demeure connectée.

TABLE 4.2 – Cas d'utilisation : Gérer les capteurs

Titre	Gérer les capteurs
Acteurs	Résident, Système embarqué
Pré-condition	Le résident doit être authentifié dans l'application
Post-condition	Les capteurs sont ajoutés, modifiés ou supprimés avec succès
Description	Le résident accède à son espace personnel afin de gérer les équipements IoT de sa maison intelligente. Il peut ajouter, modifier ou supprimer des capteurs ainsi que des caméras. Le système enregistre les modifications et met à jour les données en temps réel.
Scénario nominal	1. Le résident s'authentifie dans l'application. 2. Le système affiche la liste des équipements IoT enregistrés. 3. Le résident choisit d'ajouter, modifier ou supprimer un équipement. 4. Le résident saisit ou modifie les informations nécessaires. 5. Le système vérifie les données saisies. 6. Le système enregistre les modifications dans la base de données. 7. Le système affiche un message de confirmation.

Cette description textuelle expose avec précision le fonctionnement de la consultation des médias au sein de la maison intelligente. Elle souligne les divers critères requis

pour que le résident puisse visualiser les flux vidéo et les enregistrements provenant des caméras de surveillance, en particulier la vérification de l'identité de l'utilisateur et la bonne connexion des appareils. Elle détaille en outre le rôle du système dans l'extraction des images et des vidéos stockées, leur traitement, puis leur présentation dans l'interface utilisateur. Cette capacité assure ainsi une surveillance constante de l'environnement de vie et procure au résident une approche sécurisée et unifiée des informations de supervision de sa demeure connectée.

b. Consulter la diffusion vidéo et images

Le tableau ci-après détaille le cas d'usage « Voir la vidéo et les images ». Ce scénario permet de retracer les échanges entre le résident et le système lorsqu'il visualise les flux vidéo et les instantanés capturés par les caméras de surveillance.

TABLE 4.3 – Cas d'utilisation : Consulter la diffusion vidéo et images

Titre	Consulter la diffusion vidéo et images
Acteurs	Résident, Système
Pré-condition	Le résident est authentifié et possède au moins une caméra enregistrée
Post-condition	Les images et vidéos sont affichées correctement
Description	Le résident accède à la section des caméras pour consulter les captures enregistrées. Le système récupère les images et vidéos stockées et les affiche dans l'interface.
Scénario nominal	1. Le résident s'authentifie dans l'application. 2. Le résident accède à la section des caméras. 3. Le système récupère les images et vidéos enregistrées. 4. Le système affiche les captures dans l'interface utilisateur. 5. Le résident consulte les images et les vidéos disponibles.

Le présent descriptif expose en détail le mode de fonctionnement du dispositif de visualisation multimédia intégré au domicile connecté. Il souligne les divers prérequis requis pour qu'un habitant puisse visionner les flux vidéo instantanés ainsi que les clichés et enregistrements effectués par les caméras de surveillance, notamment la vérification de l'identité de l'usager et la présence opérationnelle des appareils rattachés au programme.

Il aborde pareillement la fonction du programme quant à la réquisition des contenus multimédias archivés, leur transformation ainsi que leur présentation sur l'interface de l'utilisateur pour assurer une consultation aisée et protégée. Cette capacité permet ainsi à l'habitant d'accomplir le suivi permanent de son cadre résidentiel et de consulter le répertoire des enregistrements produits par le système de Maison Intelligente.

4.4 Conception

L'étape de conception représente une phase cruciale dans la création du système Smart House. Elle vise à établir la structure des divers éléments matériels et logiciels, ainsi que les échanges entre les appareils IoT, le système d'arrière-plan et l'interface web. Ce segment expose les schémas majeurs conçus pour représenter le fonctionnement général du système et simplifier sa réalisation.

4.4.1 Diagramme de blocs du système Smart House

Avant de présenter les détails techniques du système, il est important d'illustrer l'organisation des différents composants matériels utilisés dans le projet. La figure 4.2 présente le diagramme de blocs du système Smart House ainsi que les connexions entre les capteurs, les cartes électroniques et les modules de communication.

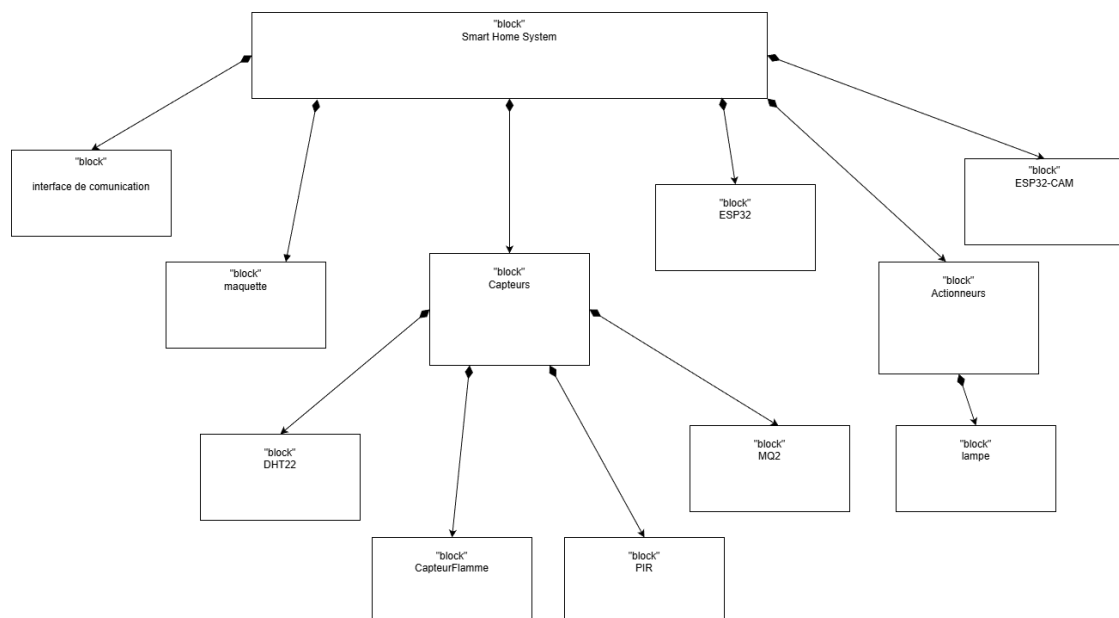


FIGURE 4.2 – Diagramme de blocs du système Smart House

La figure 4.2 illustre la structure matérielle globale du système Smart House développé dans le cadre du projet. La carte ESP32 représente l'unité centrale chargée de recevoir et traiter les données provenant des différents capteurs, notamment le capteur DHT22 pour la température et l'humidité, le capteur MQ2 pour la détection de gaz

et de fumée, le capteur de flamme pour la détection d'incendie ainsi que le capteur PIR permettant la détection de mouvement. Le diagramme met également en évidence le rôle de l'ESP32-CAM dans la capture d'images et l'enregistrement des vidéos pour assurer la surveillance de la maison. Enfin, le module FTDI USB permet l'alimentation et la programmation des cartes électroniques afin d'assurer le bon fonctionnement du système IoT.

4.4.2 Diagramme de classes

Le diagramme de classes suivant représente l'architecture statique du système de maison intelligente. Il décrit les différentes entités principales du système ainsi que les relations entre elles. Les classes présentées permettent la gestion des maisons, des caméras, des capteurs, des enregistrements vidéo, des captures d'écran et des membres utilisateurs.

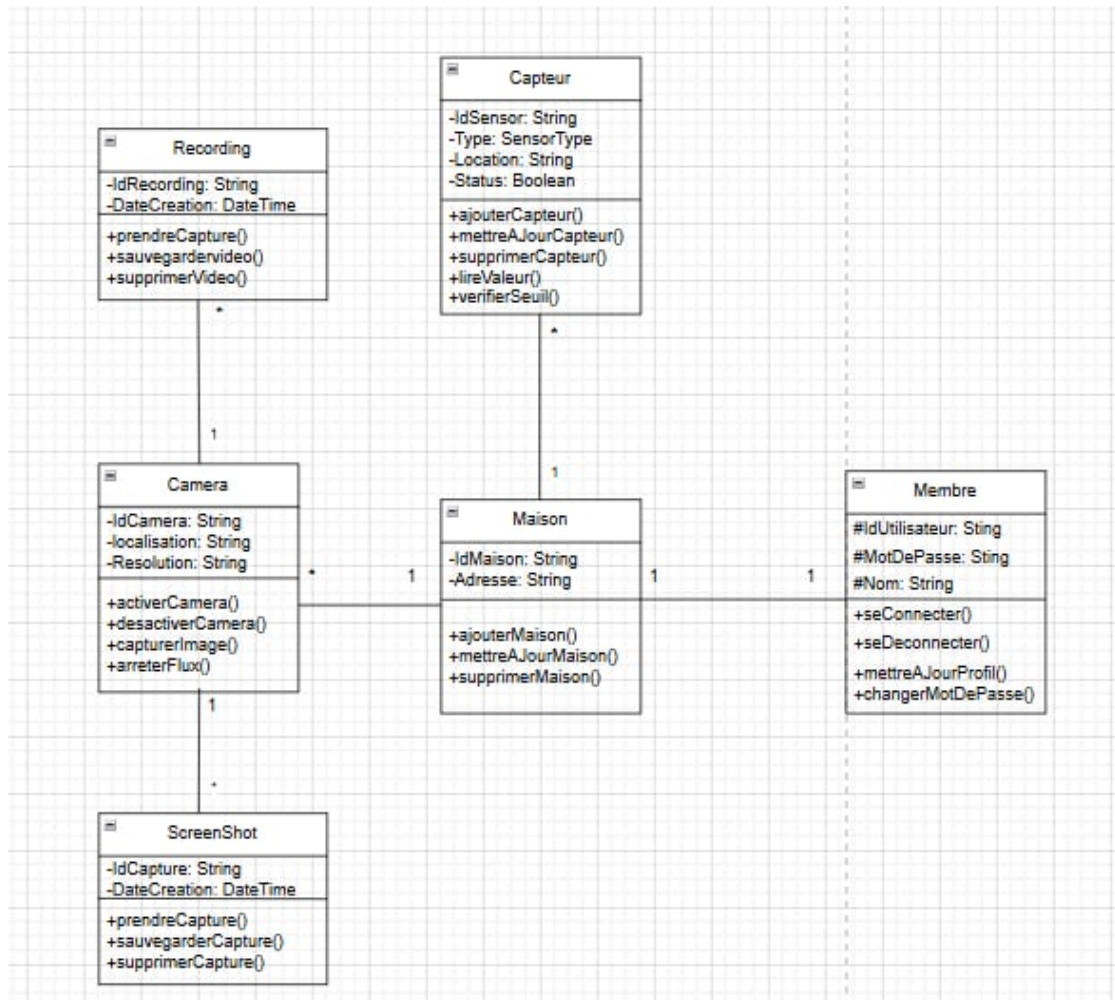


FIGURE 4.3 – Diagramme de classes du système de maison intelligente

Ce diagramme montre qu'une maison constitue l'élément central du système et peut être associée à plusieurs composants intelligents tels que les caméras et les capteurs. Les

caméras permettent la capture d'images, l'enregistrement de vidéos et la gestion du flux vidéo, tandis que les capteurs assurent la surveillance et la vérification des seuils définis. Les classes *Recording* et *ScreenShot* sont utilisées pour stocker les données multimédias générées par les caméras. Enfin, la classe *Membre* représente les utilisateurs du système avec des fonctionnalités d'authentification et de gestion du profil.

4.4.3 Diagramme de blocs interne

La figure suivante présente le diagramme de blocs interne du système Smart House. Ce diagramme met en évidence les principaux composants matériels et fonctionnels qui constituent l'architecture du système, ainsi que les relations entre eux. Il illustre notamment l'intégration de l'ESP32, de l'ESP32-CAM, des différents capteurs, des actionneurs et de l'interface de communication au sein de la solution proposée.

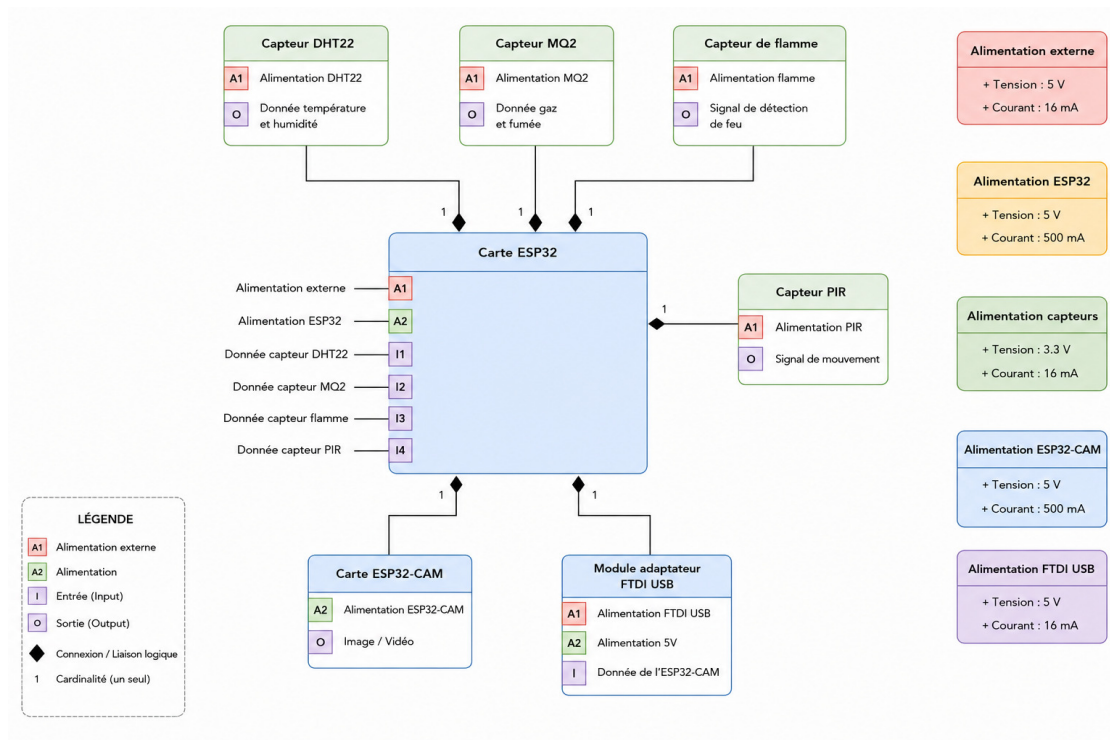


FIGURE 4.4 – Diagramme de blocs interne du système Smart House

La figure ci-après illustre l'agencement interne du dispositif Smart House réalisé dans le cadre de cette initiative. Cette représentation détaille l'infrastructure matérielle globale du mécanisme et les échanges entre les diverses pièces électroniques et modules IoT incorporés dans l'offre conçue. Le cœur du dispositif, appelé "Smart House System", gère la synchronisation et l'échange d'informations entre les différents appareils branchés. Il s'appuie essentiellement sur la puce ESP32, qui s'occupe de recueillir, traiter et expédier les informations issues des détecteurs. Le système inclut aussi le module ESP32-CAM, employé pour la surveillance visuelle, l'acquisition d'images et la diffusion

des transmissions en direct. Divers capteurs sont intégrés pour superviser de manière avisée le milieu résidentiel. Le détecteur DHT22 mesure la température et le taux d'humidité, le capteur MQ2 identifie les gaz et la fumée, le capteur de flamme sert à repérer les dangers d'incendie, alors que le détecteur PIR décèle les mouvements dans l'habitation. Le dispositif comprend également des dispositifs d'action, comme une ampoule connectée, permettant une réaction automatique selon les situations signalées par les capteurs. Finalement, une interface d'échange d'informations facilite la communication entre les dispositifs IoT, le serveur et l'interface web de supervision, assurant ainsi une administration centralisée, astucieuse et sûre de l'habitat connecté.

4.4.4 Diagramme de définition du bloc cam

Dans cette partie, nous présentons le diagramme de fonctionnement du système de surveillance Smart House basé sur la carte ESP32-CAM. Ce diagramme permet de montrer les relations entre la carte ESP32-CAM, le shield ESP32-CAM-MB et le système de surveillance chargé de recevoir les images et de les transmettre vers le backend.

La figure suivante présente le diagramme de blocs du système de surveillance Smart House basé sur la carte ESP32-CAM et le module ESP32-CAM-MB. Ce schéma illustre les relations entre les différents composants matériels ainsi que le flux de transmission des images vers le système de surveillance.

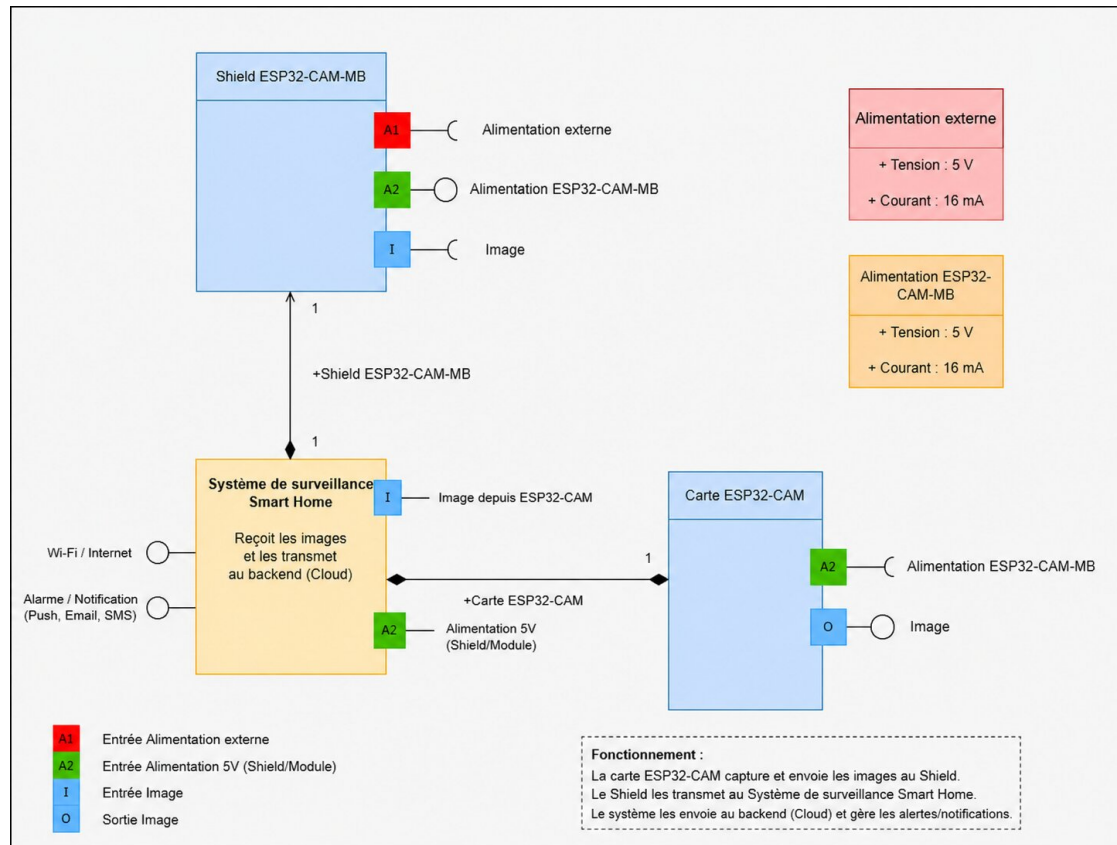


FIGURE 4.5 – Diagramme de blocs du système de surveillance Smart House avec ESP32-CAM

Ce diagramme montre que la carte ESP32-CAM capture les images puis les transmet au module ESP32-CAM-MB, qui assure son alimentation et facilite son fonctionnement. Ensuite, le système de surveillance Smart House reçoit les images et les envoie vers le backend via le réseau Wi-Fi / Internet. Le backend peut alors traiter les données et générer des alertes ou des notifications en cas d'événement détecté.

la carte ESP32-CAM capture les images et les envoie vers le shield ESP32-CAM-MB. Ensuite, le système de surveillance Smart House reçoit ces images et les transmet au backend via Wi-Fi ou Internet. Ce fonctionnement permet d'assurer le suivi visuel de la maison à distance et de faciliter l'intégration des images dans l'application web.

4.4.5 Diagramme d'activité du système IoT

Avant de présenter les détails de fonctionnement, il est nécessaire d'illustrer le déroulement général des échanges entre les équipements IoT et l'application web. La figure 4.6 montre les principales étapes réalisées par le système, depuis la lecture des données par les capteurs jusqu'à leur affichage dans l'interface utilisateur.

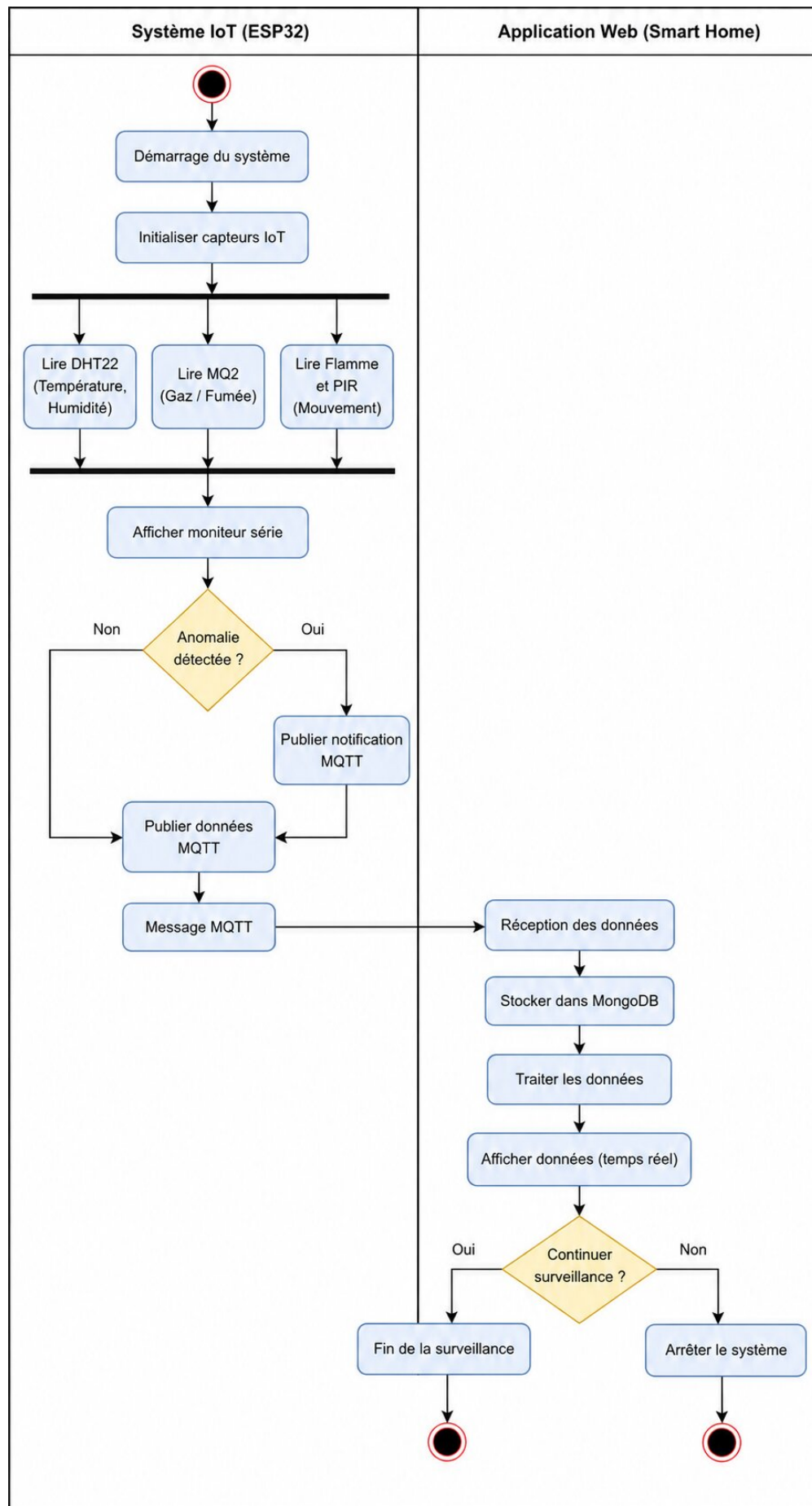


FIGURE 4.6 – Diagramme d'activité du système IoT et de l'application Smart House

La représentation ci-dessus dévoile le schéma d'activité du dispositif Smart House, conçu durant la seconde itération du projet. Ce graphique illustre le comportement général du système IoT et les communications entre la puce ESP32 et le programme web de contrôle. L'opération démarre par la mise en route du système et des divers détecteurs IoT intégrés à la résidence connectée. L'ESP32 procède ensuite à la saisie des informations issues des capteurs DHT22, MQ2, du détecteur de flammes et du capteur PIR, dans le but d'observer le cadre de vie en continu. Les informations recueillies sont présentées dans le moniteur série, puis examinées pour repérer toute irrégularité. Dès qu'une irrégularité est constatée, le système diffuse une alerte MQTT avant de transmettre les mesures relevées au backend via des messages MQTT. L'application web Smart House réceptionne alors ces éléments, les enregistre dans la base de données MongoDB, puis les traite pour garantir la visualisation instantanée des données sur l'interface utilisateur. Le schéma souligne aussi le cycle de veille ininterrompue assuré par le dispositif, afin de garantir une supervision constante de l'état des appareils et de l'environnement de la demeure connectée.

4.5 Réalisation

La réalisation de ce Sprint consiste à mettre en place le système de gestion des équipements IoT de la maison intelligente. Elle englobe le câblage des capteurs avec la carte ESP32, la programmation du microcontrôleur pour la collecte et la transmission des données, ainsi que le développement des interfaces web permettant au résident de superviser les équipements et de consulter les flux vidéo de la caméra en temps réel.

4.5.1 Choix de la carte électronique

Dans le cadre de ce projet, la carte ESP32 a été sélectionnée en raison de ses atouts et de ses caractéristiques idéales pour les environnements IoT et de domotique. Elle intègre le Wi-Fi et le Bluetooth, ce qui simplifie grandement les échanges entre les capteurs et l'interface web. Par ailleurs, cette carte propose une capacité de calcul appréciable, une faible consommation électrique et un prix compétitif, la rendant parfaitement adaptée à la création d'un dispositif d'automatisation et de suivi intelligent.

Avant de statuer sur ce choix, différentes cartes de développement ont été examinées, parmi lesquelles l'Arduino Uno, le Raspberry Pi et la NodeMCU. L'Arduino Uno, bien que facile d'accès, ne bénéficie pas d'une connectivité Wi-Fi native et ses ressources matérielles sont limitées pour les usages IoT. Le Raspberry Pi offre des performances de pointe, mais son investissement financier et sa consommation énergétique sont plus conséquents pour ce genre de réalisation. Enfin, la carte NodeMCU, si elle autorise la connexion Wi-Fi, affiche des performances moindres comparées à l'ESP32.

En conséquence, l'ESP32 se révèle être la solution la plus équilibrée en termes de performance, de connectivité, de consommation d'énergie et de coût, validant ainsi son adoption pour l'élaboration de ce système de maison connectée.

4.5.2 Câblage et connexions des composants

Cette partie présente les différentes connexions matérielles réalisées entre la carte ESP32 et les capteurs utilisés dans le système Smart House. Le câblage permet d'assurer l'alimentation des composants ainsi que la transmission des données entre les capteurs et la carte de contrôle. Les capteurs sont connectés aux broches GPIO de l'ESP32 selon les besoins fonctionnels du système. Les connexions VCC et GND permettent respectivement l'alimentation électrique et la mise à la masse des différents composants. Les données collectées par les capteurs sont ensuite transmises vers l'ESP32 pour être traitées et envoyées au serveur via le protocole MQTT. Le tableau suivant présente les principales connexions réalisées entre les composants matériels et la carte ESP32.

TABLE 4.4 – Connexions des composants avec la carte ESP32

Composant	Broche ESP32
Capteur DHT22	GPIO 4
Capteur MQ2	GPIO 34
Capteur de flamme	GPIO 13
Capteur PIR	GPIO 27
LED Rouge	GPIO 14
LED Verte	GPIO 15
Relais lampe	GPIO 26
ESP32-CAM	Communication Wi-Fi

Le tableau ci-dessus détaille les principales liaisons établies entre la carte ESP32 et les divers éléments de l'installation Domotique. Chaque sonde est connectée à un port GPIO désigné pour garantir la saisie et l'acheminement adéquat des informations vers le module de gestion. Ce montage permet à l'ESP32 de surveiller les données issues des sondes, de piloter les dispositifs de sortie comme les indicateurs lumineux et le contacteur, et également de communiquer avec l'ESP32-CAM et le broker MQTT pour le suivi et le contrôle en direct de l'ensemble.

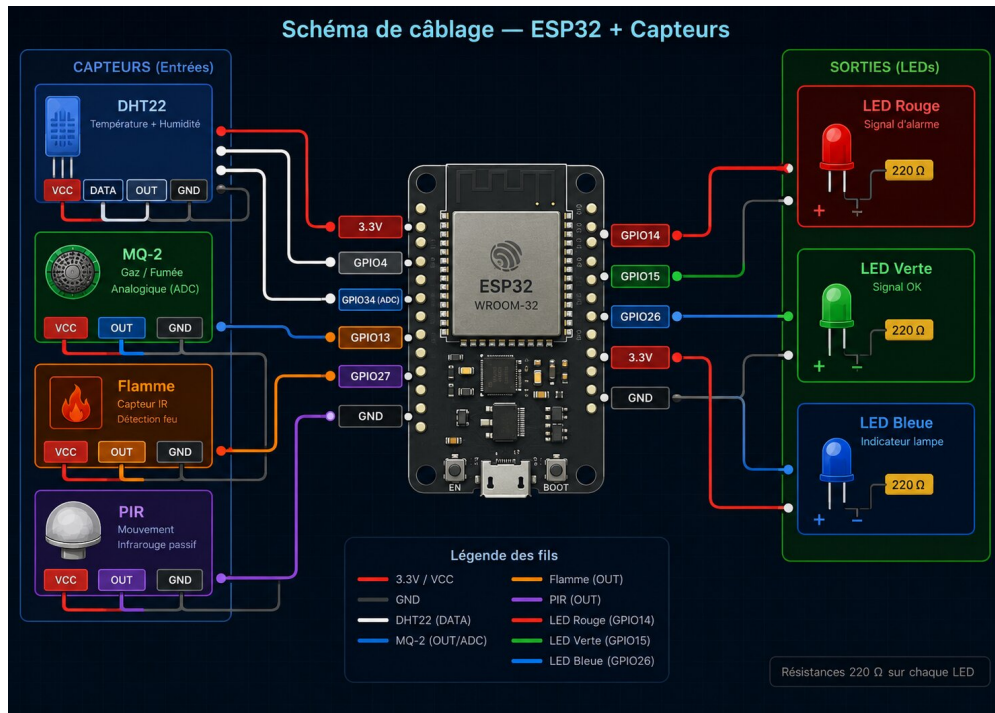


FIGURE 4.7 – Schéma du câblage

L'image ci-dessus expose la carte ESP32, de même que la disposition de ses divers ports d'entrée et de sortie. Ce module se caractérise par sa connexion Wi-Fi et Bluetooth intégrée, sa faible consommation électrique et ses multiples interfaces de communication. Les broches GPIO autorisent le raccordement de divers dispositifs électroniques, tels que des capteurs, des relais et des modules de contrôle. En raison de ses capacités et de sa polyvalence, l'ESP32 constitue une option pertinente pour les réalisations de domotique et les systèmes embarqués.

4.5.3 Implémentation du système embarqué

Le système embarqué développé dans le cadre de ce projet repose principalement sur la carte ESP32, programmée à l'aide de l'environnement Arduino IDE. Cette carte assure la collecte et le traitement des données provenant des différents capteurs intégrés dans la maison intelligente.

Le système permet la lecture des données issues du capteur DHT22 pour la mesure de la température et de l'humidité, du capteur MQ2 pour la détection de gaz et de fumée, du capteur de flamme pour la détection d'incendie ainsi que du capteur PIR pour la détection de mouvement. Les données collectées sont ensuite analysées localement par l'ESP32 afin d'identifier les situations anormales et générer des alertes en cas de danger.

La communication entre le système embarqué et le serveur backend est assurée via le protocole MQTT, permettant la transmission des données en temps réel vers

l'application web développée avec Angular et NestJS. Le système intègre également le module ESP32-CAM pour assurer la surveillance de la maison à travers la capture d'images et de vidéos. Cette architecture garantit un fonctionnement fiable, rapide et adapté aux applications IoT de maison intelligente.

4.5.4 Communication MQTT et transmission des données

La communication entre les différents composants du système Smart House est assurée à l'aide du protocole MQTT, largement utilisé dans les applications IoT grâce à sa légèreté et sa rapidité de transmission. Dans ce projet, le broker Mosquitto joue le rôle d'intermédiaire entre les cartes ESP32 et le serveur backend, permettant l'échange des données en temps réel.

Les cartes ESP32 publient périodiquement les données collectées par les capteurs sur différents topics MQTT dédiés. Chaque type de donnée possède un topic spécifique afin d'assurer une organisation claire et efficace des informations transmises. Par exemple, les données de température, de gaz et de détection de flamme sont publiées respectivement sur les topics `smarthome/temperature`, `smarthome/gaz` et `smarthome/flamme`.

Le backend développé avec NestJS est abonné à ces différents topics afin de recevoir automatiquement les données envoyées par les équipements IoT. Les informations reçues sont ensuite traitées, stockées dans la base de données MongoDB puis affichées en temps réel dans l'application web. Cette architecture permet d'assurer une communication fiable, rapide et continue entre les équipements connectés et la plateforme de supervision Smart House.

4.5.5 Intégration du module ESP32-CAM

Dans le cadre du système Smart House, le module ESP32-CAM a été intégré afin d'assurer les fonctionnalités de surveillance et de sécurité de la maison intelligente. Ce module embarqué combine un microcontrôleur ESP32 avec une caméra permettant la capture d'images et l'enregistrement de vidéos en temps réel.

L'ESP32-CAM est connecté au réseau Wi-Fi afin de transmettre les images et les vidéos vers l'application web. Ce module permet à l'utilisateur de surveiller son habitation à distance et de consulter les médias capturés directement depuis l'interface développée avec Angular.

Le système utilise également l'ESP32-CAM pour renforcer la sécurité de la maison en associant la capture d'images aux événements détectés par les différents capteurs, notamment le capteur PIR pour la détection de mouvement et le capteur de flamme pour la détection d'incendie. Cette intégration permet d'améliorer la supervision du système et d'assurer une surveillance intelligente et efficace de l'environnement domestique.

4.6 Test

Une fois le système assemblé et les différents composants configurés, une phase de tests a été réalisée afin de valider le bon fonctionnement de l'ensemble du système. Ces tests ont permis de vérifier la fiabilité de la communication entre les capteurs et la carte ESP32, la transmission correcte des données vers le backend via le protocole MQTT, ainsi que l'affichage en temps réel des informations dans l'interface web. Les résultats obtenus sont présentés dans la section suivante.

4.6.1 Test du câblage et des capteurs

Afin de vérifier le bon fonctionnement du système embarqué, plusieurs tests ont été réalisés sur les connexions entre la carte ESP32 et les différents capteurs utilisés dans le projet. Ces tests permettent de valider le câblage des composants ainsi que la lecture correcte des données provenant des capteurs.

La figure suivante présente le montage matériel réalisé pour le prototype Smart House. Elle illustre l'interconnexion des différents composants électroniques utilisés dans le système, notamment la carte ESP32, les capteurs, l'écran LCD ainsi que les éléments de câblage sur breadboard.

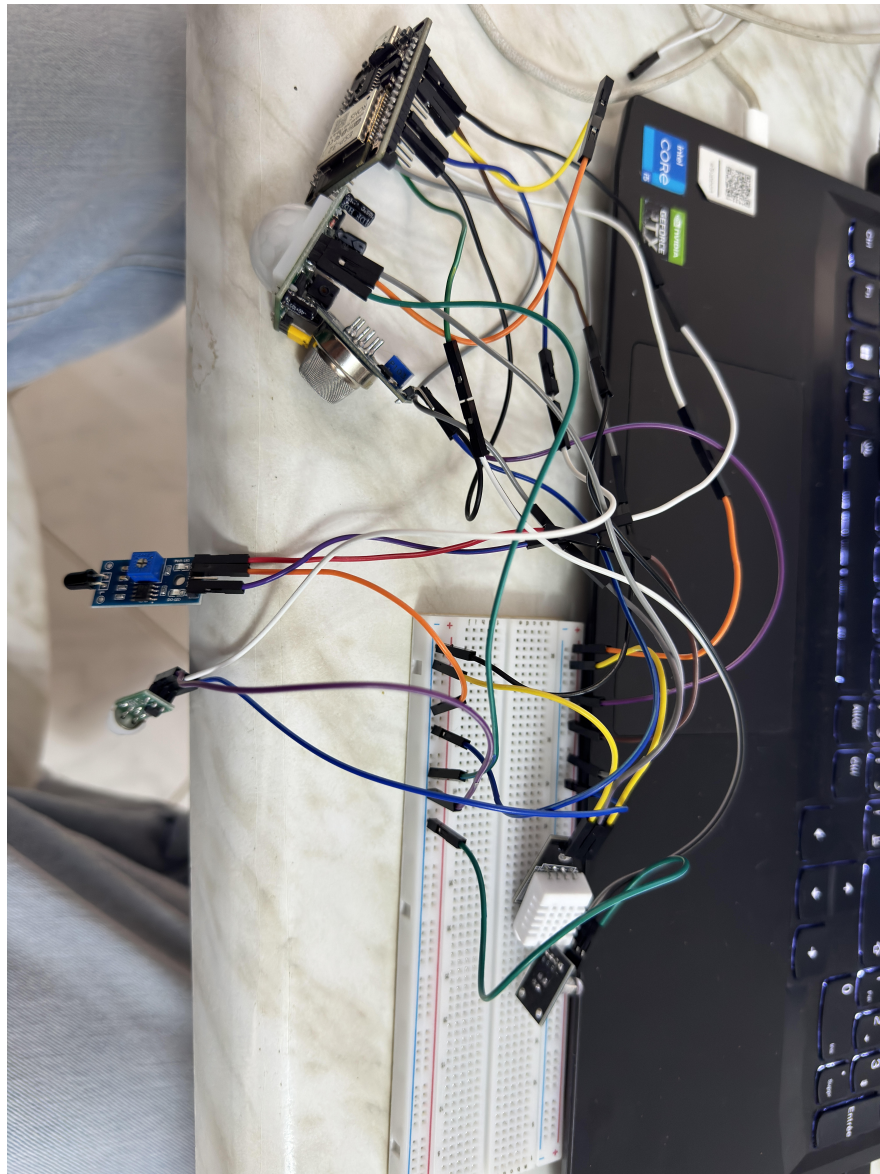


FIGURE 4.8 – Montage matériel du prototype Smart House

La figure précédente présente les différentes connexions réalisées entre la carte ESP32 et les capteurs DHT22, MQ2, PIR ainsi que le capteur de flamme. Les composants sont reliés aux broches GPIO de l'ESP32 afin d'assurer la collecte des données et le fonctionnement du système Smart House. Après la validation du câblage, des tests de lecture des données ont été effectués afin de vérifier la transmission correcte des informations des capteurs vers la carte ESP32. La figure 4.9 présente l'architecture matérielle du système Smart House réalisée à l'aide de la carte ESP32 et de plusieurs capteurs intelligents permettant la surveillance et l'automatisation de l'habitation. La carte ESP32 représente l'unité principale du système, assurant la collecte des données et la communication avec l'application web. Le capteur DHT22 est utilisé pour mesurer la température et l'humidité ambiante afin de surveiller les conditions climatiques de la maison. Le capteur MQ2 permet la détection des gaz et de la fumée

pour renforcer la sécurité contre les risques d'incendie ou de fuite de gaz. Le capteur PIR (Passive Infrared Sensor) détecte les mouvements humains à travers les variations du rayonnement infrarouge, ce qui permet d'identifier une présence ou une intrusion dans la maison. Le capteur de flamme assure la détection rapide d'une source de feu grâce à la reconnaissance des rayonnements infrarouges émis par une flamme. Les LEDs rouge et verte sont utilisées comme indicateurs visuels d'état et d'alerte, tandis que la lampe bleue s'allume automatiquement lors de la détection d'un mouvement par le capteur PIR afin d'assurer un éclairage intelligent et automatique.

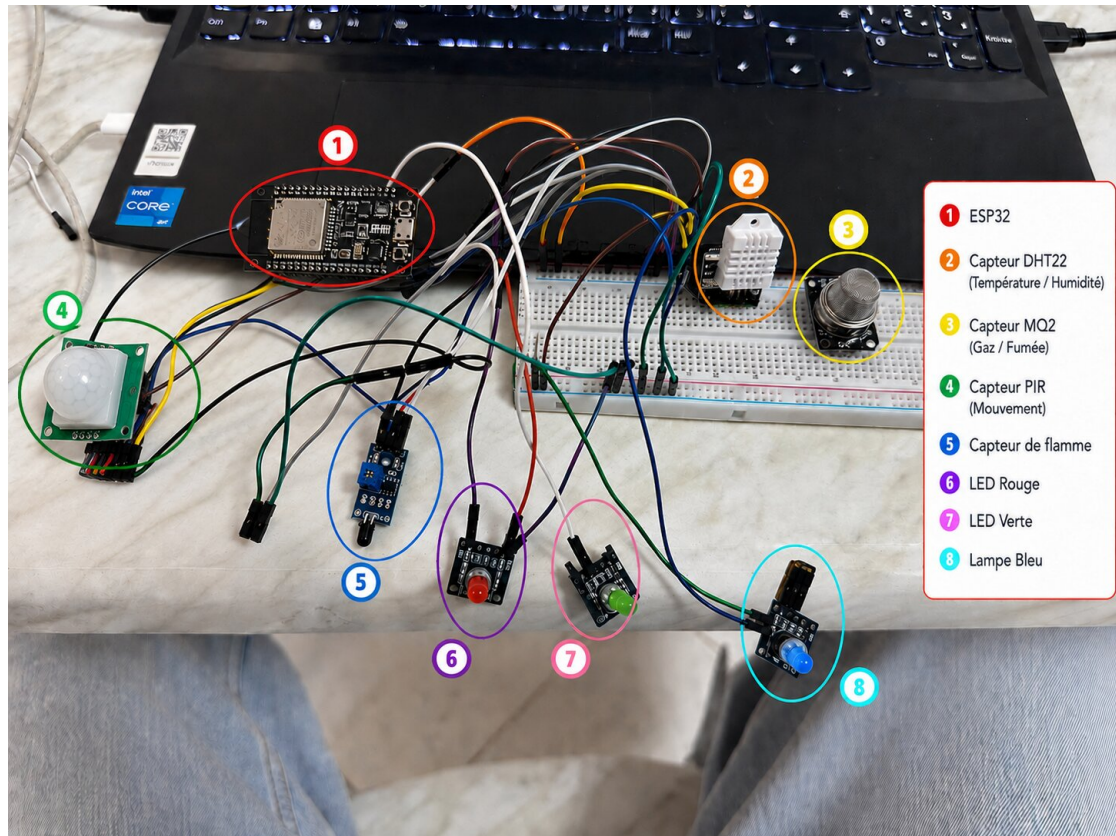


FIGURE 4.9 – Montage matériel détaillé

Après la validation du câblage, des tests de lecture des données ont été effectués afin de vérifier la transmission correcte des informations des capteurs vers la carte ESP32.

La figure présente l'architecture matérielle du système Smart House réalisée à l'aide de la carte ESP32 et de plusieurs capteurs intelligents permettant la surveillance et l'automatisation de l'habitation. La carte ESP32 représente l'unité principale du système, assurant la collecte des données et la communication avec l'application web.

Le capteur DHT22 est utilisé pour mesurer la température et l'humidité ambiante afin de surveiller les conditions climatiques de la maison. Le capteur MQ2 permet la détection des gaz et de la fumée pour renforcer la sécurité contre les risques d'incendie ou de fuite de gaz. Le capteur PIR (Passive Infrared Sensor) détecte les mouvements humains à travers les variations du rayonnement infrarouge, ce qui permet d'identifier

une présence ou une intrusion dans la maison.

Le capteur de flamme assure la détection rapide d'une source de feu grâce à la reconnaissance des rayonnements infrarouges émis par une flamme. Les LEDs rouge et verte sont utilisées comme indicateurs visuels d'état et d'alerte, tandis que la lampe bleue s'allume automatiquement lors de la détection d'un mouvement par le capteur PIR afin d'assurer un éclairage intelligent et automatique.

4.6.2 Test de transmission des données MQTT

Des tests de communication MQTT ont également été réalisés afin de vérifier l'échange des données entre les équipements IoT et le serveur backend. Le broker Mosquitto assure la transmission des messages publiés par les cartes ESP32 vers l'application web.

```
rst:0x1 (POWERON_RESET),boot:0x17 (SPI_FAST_FLASH_BOOT)
config: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:4876
ho 0 tail 12 room 4
load:0x40078000,len:16560
load:0x40080400,len:3500
entry 0x400805b4
=== Smart Home ESP32 v3.0 : DHT22 + MQ2 + Flamme + Lampe RGB ===
[LAMPE] Éteinte | Couleur : OFF
[WiFi] Connexion à Ooredoo-X16-1911C3.....
[WiFi] Connecté - IP : 192.168.0.110
[MQTT] Connexion... OK
=== Système prêt ===
--- Lecture capteurs ---
[DATA] temperature = 25.70 C
[DATA] humidity = 52.90 %
[DATA] MQ2 = 195.00 ADC
[DATA] FLAME = 0.00 state
-----
--- Lecture capteurs ---
[DATA] temperature = 25.70 C
[DATA] humidity = 52.90 %
[DATA] MQ2 = 187.00 ADC
[DATA] FLAME = 0.00 state
-----
```

FIGURE 4.10 – Affichage des données capteurs et de la connexion MQTT dans la console série

Cette figure montre que le système s'initialise correctement, se connecte au réseau local puis au broker MQTT avant de commencer l'acquisition des mesures. Les valeurs affichées concernent notamment la température, l'humidité, le capteur MQ2 ainsi que l'état du capteur de flamme. Cela confirme le bon fonctionnement de la chaîne de communication entre les capteurs, l'ESP32 et le système de supervision.

```
--- Lecture capteurs ---  
[DATA] temperature = 25.50 C  
[DATA] humidity = 57.30 %  
[DATA] MQ2 = 3113.00 ADC  
[ALERTE] critical - DANGER CRITIQUE : Gaz ou fumee en concentration elevee ! (val=3113.00)  
[DATA] FLAME = 1.00 state  
[ALERTE] critical - DANGER : Flamme detectee ! (val=1.00)  
[LED] Couleur -> RED
```

FIGURE 4.11 – Affichage des données et alertes sur le moniteur série

La figure 4.11 présente un exemple d’affichage des données collectées par les capteurs sur le moniteur série de l’ESP32. Les valeurs de température et d’humidité mesurées par le capteur DHT22 sont affichées en temps réel, ainsi que les données du capteur MQ2 relatives à la détection de gaz ou de fumée. Lorsque la concentration détectée dépasse le seuil défini, le système génère automatiquement une alerte critique signalant un danger potentiel. De plus, le capteur de flamme permet de détecter la présence d’un feu et déclenche immédiatement une notification d’alerte. Cet affichage facilite la surveillance instantanée de l’état de la maison intelligente et améliore la réactivité du système face aux situations dangereuses.

4.6.3 Test du module ESP32-CAM

Des tests ont été réalisés sur le module ESP32-CAM afin de vérifier le bon fonctionnement de la surveillance vidéo du système Smart House. Le module permet la capture d’images ainsi que la transmission du flux vidéo via le réseau Wi-Fi.

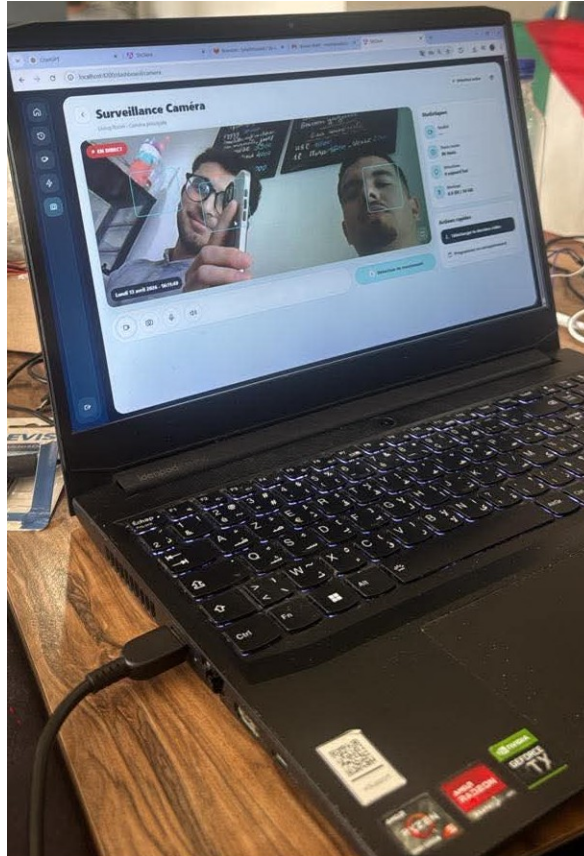


FIGURE 4.12 – Module ESP32-CAM utilisé dans le système

La figure précédente présente le module ESP32-CAM intégré dans le système de surveillance de la maison intelligente.

Cette figure illustre la transmission du flux vidéo depuis l'ESP32-CAM vers l'interface web afin d'assurer la surveillance en temps réel de la maison.

4.6.4 Validation du fonctionnement global du système

Les différents tests réalisés ont permis de valider le bon fonctionnement du système Smart House dans son ensemble. Les capteurs communiquent correctement avec la carte ESP32, les données sont transmises de manière fiable via le protocole MQTT et l'application web assure l'affichage en temps réel des informations collectées. Les résultats obtenus confirment la stabilité et l'efficacité de l'architecture IoT mise en place dans le cadre de ce projet.

4.7 Interfaces réalisées

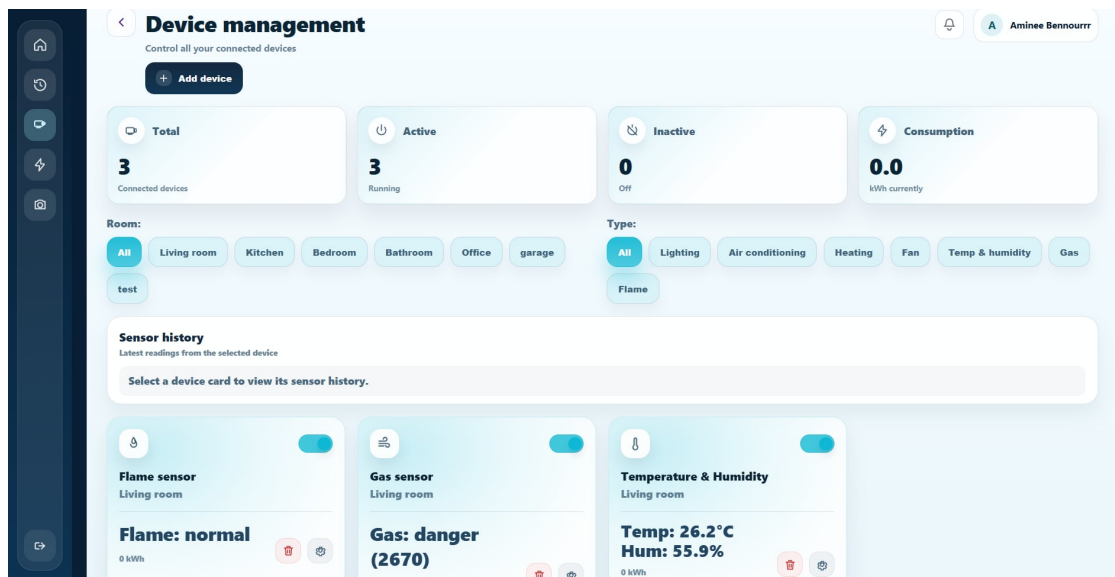


FIGURE 4.13 – Interface de gestion des équipements

Cette interface permet à l'habitant de visualiser et de gérer l'ensemble des équipements connectés de la maison intelligente. Elle présente des informations synthétiques sur les appareils enregistrés, notamment leur état de fonctionnement et leur consommation énergétique. L'utilisateur peut filtrer les équipements selon les pièces de la maison ou leur type, consulter les données remontées par les capteurs en temps réel et accéder aux différentes actions de gestion. Cette interface constitue un élément central de la plateforme Smart House en facilitant la supervision et le contrôle des objets connectés.

Afin d'offrir une meilleure visibilité sur l'évolution des données collectées par les capteurs, la plateforme met également à disposition un historique détaillé des mesures enregistrées. Cette fonctionnalité permet à l'utilisateur de consulter les relevés précédents et de suivre l'évolution des conditions environnementales au sein de son habitation.

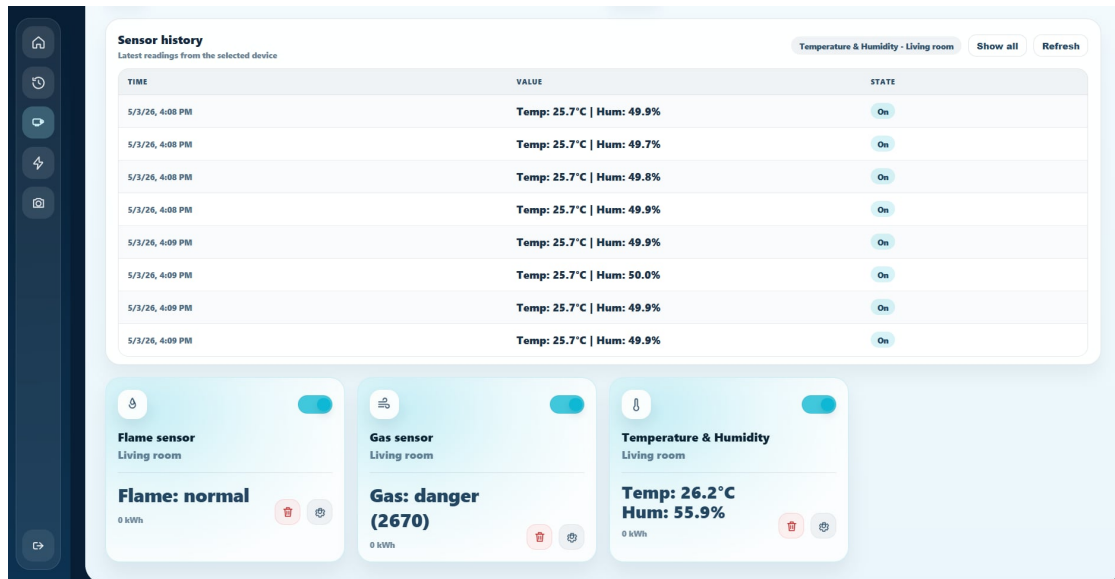


FIGURE 4.14 – Interface d'historique des mesures de température et d'humidité

Cette interface présente l'historique des données remontées par le capteur de température et d'humidité installé dans une pièce de la maison. Les mesures sont affichées sous forme de tableau indiquant la date et l'heure de chaque relevé, les valeurs enregistrées ainsi que l'état du capteur. L'utilisateur peut ainsi consulter les données passées, vérifier le bon fonctionnement du dispositif et analyser les variations de température et d'humidité au fil du temps. Cette fonctionnalité contribue à une meilleure surveillance de l'environnement domestique et facilite la prise de décision en matière de confort et d'efficacité énergétique.

Conclusion

Dans ce chapitre, nous avons présenté la réalisation du Sprint2 consacré à la gestion des équipements IoT du système Smart House. Nous avons détaillé l'ensemble des composants matériels utilisés, notamment la carte ESP32 et le module ESP32-CAM, ainsi que leur câblage et leur configuration. La communication entre les équipements IoT et le backend a été assurée via le protocole MQTT, garantissant une transmission fiable et en temps réel des données collectées par les capteurs. Les différents tests réalisés ont permis de valider le bon fonctionnement du système, depuis la lecture des données par les capteurs jusqu'à leur affichage dans l'interface web développée avec Angular. L'intégration du module ESP32-CAM a également permis d'assurer la surveillance visuelle de la maison à distance. Le chapitre suivant sera consacré à la gestion des notifications, de l'historique et des membres de la famille.

Chapitre 5

Sprint 3 : Notifications et Historique

Introduction

Dans cette itération, nous nous sommes concentrés sur le développement des fonctionnalités de notification, de consultation de l'historique des activités et de gestion des utilisateurs familiers au sein de la plateforme Smart House. L'objectif principal est d'améliorer la supervision et la sécurité de la maison intelligente en permettant au résident de recevoir des alertes en temps réel lorsqu'une anomalie est détectée par les différents capteurs connectés. Cette itération vise également à assurer la traçabilité des événements et des actions enregistrés dans le système grâce à un historique détaillé, tout en offrant une gestion sécurisée des utilisateurs autorisés à accéder à la plateforme.

5.1 Sprint Goal

L'objectif de ce Sprint est de concevoir et de mettre en œuvre les fonctionnalités liées à la gestion des notifications et à la consultation de l'historique des activités dans le système Smart House. Cette itération permet au résident de recevoir des alertes instantanées lors de la détection d'événements critiques, tels qu'un incendie, une fuite de gaz ou une intrusion détectée par les capteurs intelligents. Elle assure également l'enregistrement et le suivi des différentes actions effectuées au sein du système afin d'améliorer le contrôle, la supervision et la sécurité de l'habitat connecté. Par ailleurs, ce Sprint intègre des fonctionnalités de gestion des membres de la famille permettant de contrôler les accès et les autorisations des utilisateurs associés à la maison intelligente.

5.2 Sprint Backlog

Ce tableau récapitule le backlog du Sprint concernant les fonctions de notification, la visualisation de l'historique des événements et la gestion des membres de la famille dans l'application Smart House. Il liste les plus importantes user stories retenues pour ce sprint, en indiquant leur priorité, l'estimation de leur temps de développement en jours et les fonctionnalités reliées. L'objectif de ce Sprint est avant tout de perfectionner la surveillance de la maison connectée, de consolider la sécurité du système et de simplifier

l'administration des comptes utilisateurs dans l'application web.

TABLE 5.1 – Sprint Backlog - Notifications et Historique

ID	Acteur	User Story	Priorité	Estimation	Fonctionnalité
8	Résident	En tant que résident, je peux consulter les notifications.	Haute	7 J	Notifications
9	Résident	En tant que résident, je peux consulter l'historique des événements.	Moyenne	7 J	Historique des événements
10	Résident	En tant que résident, je peux gérer les membres de ma famille.	Haute	7 J	Gestion des membres de la famille

Via cette liste d'actions, nous soulignons les attentes du résident concernant l'observation et la régulation des incidents détectés par les dispositifs intelligents. Les possibilités implémentées aident le résident à recevoir et examiner les alertes transmises instantanément, à consulter l'historique des incidents consignés pour garantir une meilleure supervision des opérations du système, et également à administrer les proches autorisés à utiliser l'application Maison Intelligente. Ces options participent à l'optimisation de la structuration, de la sûreté et de l'usage du domicile connecté.

5.3 Analyse

Cette phase d'analyse a pour objectif d'identifier les besoins fonctionnels liés aux notifications, à la consultation de l'historique des événements ainsi qu'à la gestion des membres dans le système Smart House. Elle permet de définir les interactions entre le résident et l'application afin d'assurer une meilleure surveillance et une gestion efficace des événements détectés par les capteurs intelligents. Cette étape constitue une base essentielle pour la conception et le développement des fonctionnalités du Sprint3.

5.3.1 Diagramme de cas d'utilisation

Dans cette partie, nous présentons les différentes fonctionnalités liées à la consultation des notifications, à l'historique des actions et des valeurs des capteurs, ainsi qu'à la gestion des membres de la famille au sein du système Smart House. Ces fonctionnalités permettent au résident de suivre l'activité de la maison intelligente et de gérer les accès des différents utilisateurs associés à l'habitation connectée.

La figure suivante illustre le diagramme de cas d'utilisation correspondant à ces fonctionnalités.

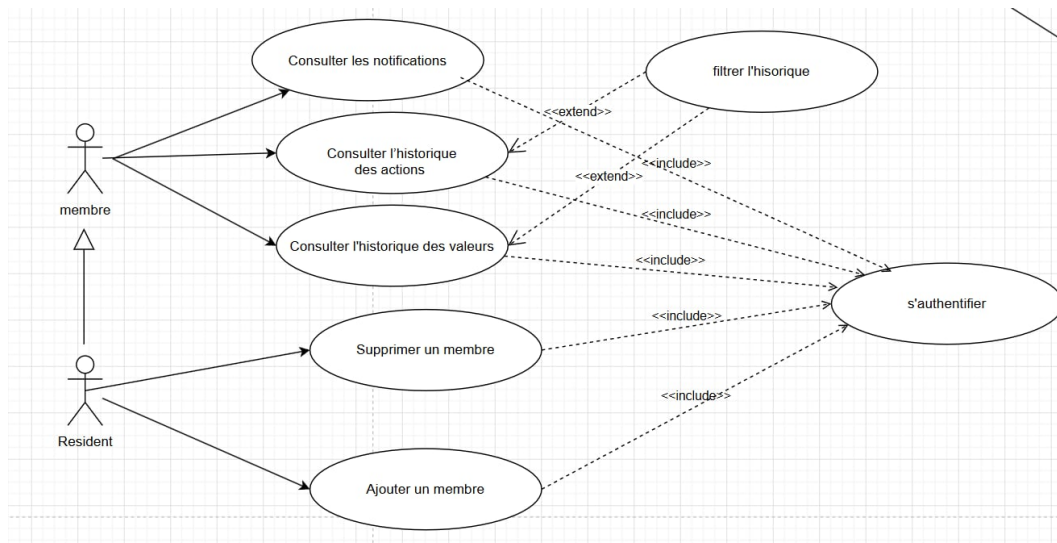


FIGURE 5.1 – Diagramme de cas d'utilisation — Gestion des notifications et des membres

Ce diagramme met en évidence les interactions entre les acteurs « membre » et « résident » avec les différentes fonctionnalités proposées par la plateforme. Le membre peut consulter les notifications reçues, consulter l'historique des actions effectuées dans le système ainsi que l'historique des valeurs des capteurs. Il a également la possibilité de filtrer l'historique afin de faciliter la recherche des informations souhaitées. Le résident, qui hérite des fonctionnalités du membre, dispose en plus des droits de gestion des membres de la famille. Il peut ainsi ajouter un nouveau membre ou supprimer un membre existant afin de contrôler les accès au système Smart House. Cette organisation permet d'assurer une gestion sécurisée et structurée des utilisateurs et des informations relatives à la maison intelligente.

5.3.2 Description textuelle des cas d'utilisation

Le tableau suivant présente la description textuelle du cas d'utilisation « Ajouter un membre ».

TABLE 5.2 – Description textuelle du cas d'utilisation : Ajouter un membre

Élément	Description
Titre	Ajouter un membre
Acteur	Résident
Description	Permettre au résident d'ajouter un nouveau membre à l'abonnement afin de lui donner accès au système Smart House.
Pré-condition	Le résident doit être authentifié dans le système.
Postconditions	Le nouveau membre est ajouté avec succès dans le système.
Scénario nominal	1. Le résident accède à l'interface de gestion des membres. 2. Le système affiche la liste des membres. 3. Le résident sélectionne l'option « Ajouter un membre ». 4. Le système affiche le formulaire d'ajout. 5. Le résident saisit les informations du nouveau membre. 6. Le système valide les informations saisies. 7. Le système ajoute le nouveau membre. 8. Le système affiche un message de confirmation.

Ce tableau décrit le cas d'utilisation relatif à l'ajout d'un membre dans l'abonnement Smart House. Il met en évidence les différentes étapes permettant au résident d'ajouter un nouvel utilisateur au système, ainsi que les conditions nécessaires au bon déroulement de cette opération. Après la gestion des membres, le système permet également au résident de consulter les notifications générées par les différents événements du Smart House. Le tableau suivant décrit les différentes interactions liées à la consultation et à la gestion des notifications. Le tableau suivant présente la description textuelle du cas d'utilisation « Consulter les notifications ».

TABLE 5.3 – Description textuelle du cas d'utilisation : Consulter les notifications

Élément	Description
Titre	Consulter les notifications
Acteur	Résident
Description	Permettre au résident de consulter les notifications générées par le système Smart House.
Pré-condition	Le résident doit être connecté au système.
Postconditions	Les notifications sont affichées et peuvent être marquées comme lues.
Scénario nominal	1. Le résident accède à l'interface des notifications. 2. Le système affiche la liste des notifications. 3. Le résident consulte les notifications reçues. 4. Le résident sélectionne l'option « Marquer tout comme lu ». 5. Le système met à jour l'état des notifications. 6. Le système affiche un message de confirmation.
Scénario alternatif	1. À l'étape 2 du scénario nominal, aucune notification n'est disponible. 2. Le système affiche un message indiquant qu'aucune notification n'a été trouvée.

Ce tableau présente les différentes étapes permettant au résident de consulter les notifications générées par le système Smart House. Il décrit également le comportement du système lors de l'affichage des notifications ainsi que le scénario alternatif dans le cas où aucune notification n'est disponible.

5.3.3 Diagramme de séquence système

Le diagramme de séquence système suivant illustre les différentes interactions entre le résident et le système Smart House lors de l'ajout d'un nouveau membre. Il décrit les échanges effectués depuis la demande d'ajout jusqu'à la confirmation ou l'affichage d'un message d'erreur selon la validité des informations saisies.

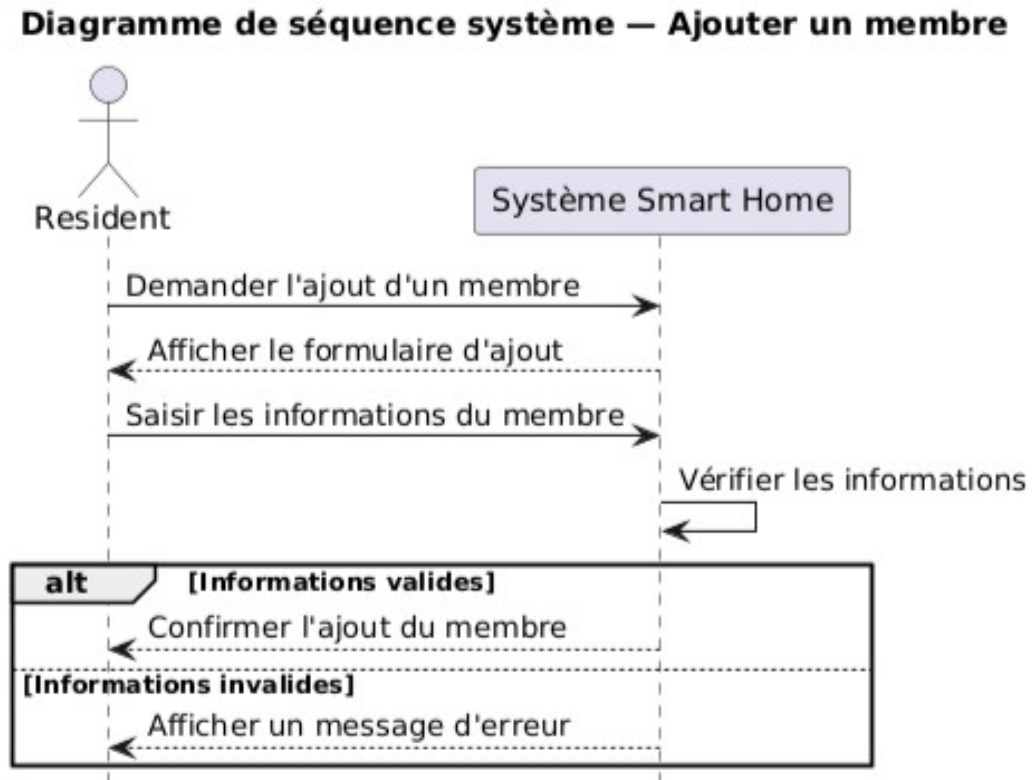


FIGURE 5.2 – Diagramme de séquence système — Ajouter un membre

Le diagramme précédent montre que le résident interagit directement avec le système Smart House afin d'ajouter un nouveau membre à l'abonnement. Le système affiche le formulaire d'ajout, vérifie les informations saisies puis confirme l'ajout lorsque les données sont valides. Dans le cas contraire, un message d'erreur est affiché afin d'informer le résident des informations invalides.

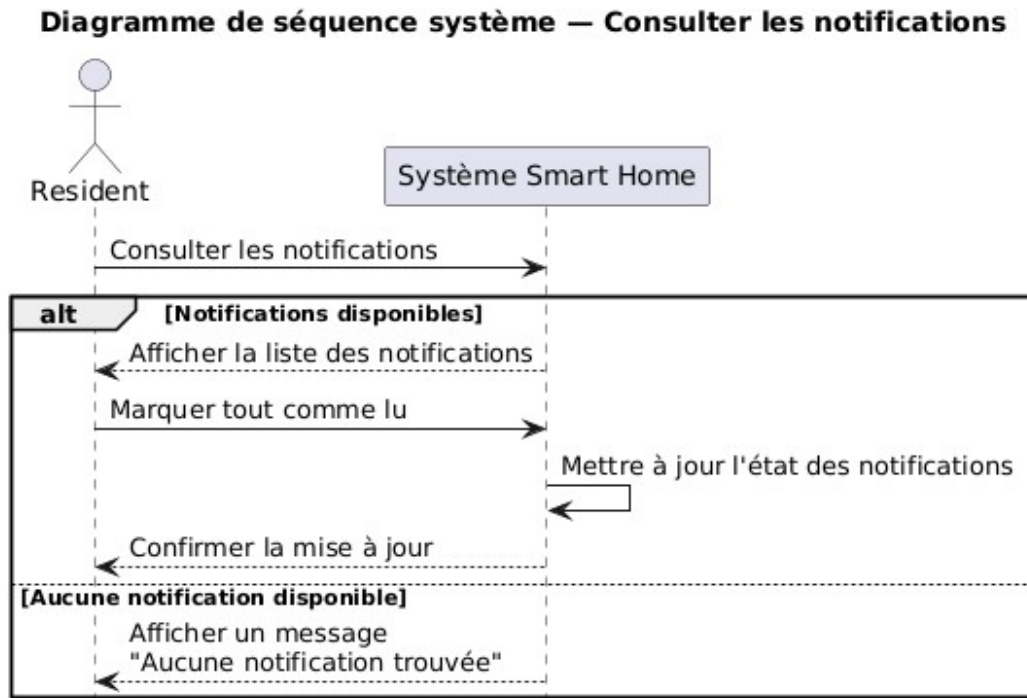


FIGURE 5.3 – Diagramme de séquence système — Consulter les notifications

Le diagramme précédent illustre le processus de consultation des notifications dans le système Smart House. Le résident demande l’affichage des notifications générées par le système. Lorsque des notifications sont disponibles, le résident peut les marquer comme lues, ce qui entraîne la mise à jour de leur état par le système. Dans le cas où aucune notification n’est disponible, le système affiche un message informatif au résident.

5.4 Conception

Cette section présente la conception du Sprint3 à travers le diagramme de classe. Ce diagramme permet de décrire la structure des principales entités du système ainsi que les relations entre elles.

5.4.1 Diagramme de classes

Dans cette partie, nous présentons le diagramme de classes relatif à la gestion des capteurs, des notifications et de l’historique au sein du système Smart House. Ce diagramme permet de représenter la structure statique du système à travers les différentes classes, leurs attributs, leurs méthodes ainsi que les relations établies entre elles.

La figure suivante illustre le diagramme de classes du système.

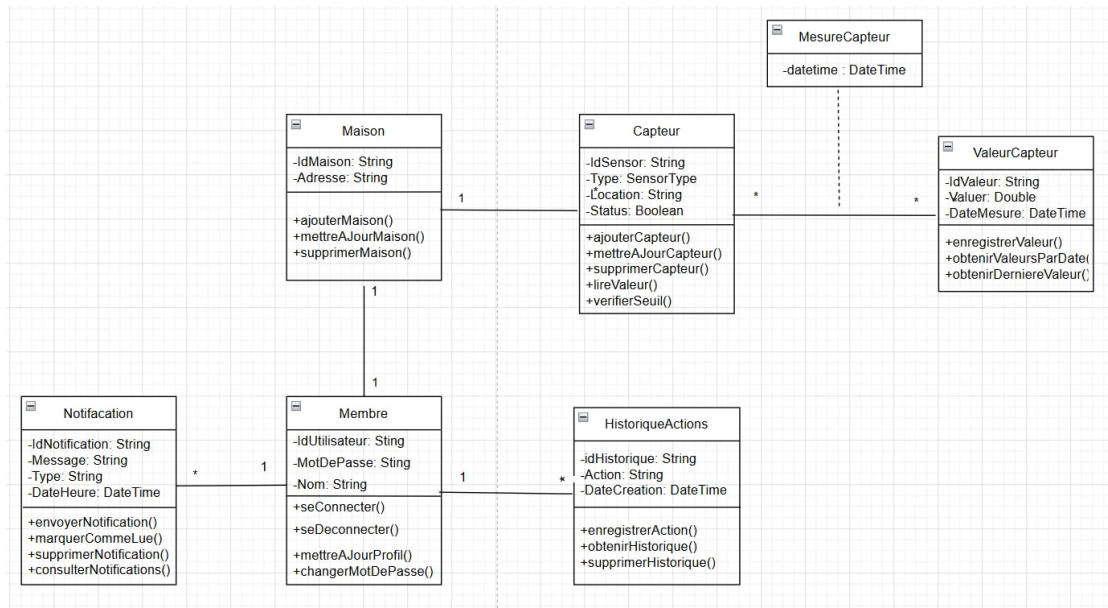


FIGURE 5.4 – Diagramme de classes du système Smart House

Ce diagramme met en évidence les principales classes intervenant dans le fonctionnement du système Smart House. La classe **Maison** représente l'habitation connectée et contient les informations liées à l'adresse de la maison. Chaque maison peut être associée à plusieurs capteurs permettant de surveiller différents paramètres de l'environnement. La classe **Capteur** représente les différents capteurs installés dans la maison. Elle contient des informations telles que le type du capteur, son emplacement et son état de fonctionnement. Les capteurs sont responsables de la lecture et de la transmission des valeurs mesurées. Les données collectées sont stockées dans la classe **ValeurCapteur**, qui enregistre les différentes mesures ainsi que leur date de collecte. La classe d'association **MesureCapteur** permet de représenter la relation entre les capteurs et les valeurs mesurées tout en conservant des informations complémentaires comme la date et l'heure de la mesure. La classe **Membre** représente les utilisateurs du système. Elle permet la gestion des opérations d'authentification, de modification du profil et du mot de passe. Chaque membre peut recevoir plusieurs notifications représentées par la classe **Notification**, qui contient les messages et les alertes générés par le système. Enfin, la classe **HistoriqueActions** assure la traçabilité des opérations effectuées dans le système en enregistrant les différentes actions réalisées par les utilisateurs ainsi que leurs dates de création. Cette organisation garantit une gestion structurée et efficace des données et des interactions au sein du système Smart House.

5.4.2 Diagramme de déploiement

Le diagramme de déploiement présenté ci-dessous illustre l'architecture physique et logicielle du système Smart House IoT. Il met en évidence les différents serveurs,

composants matériels et services externes utilisés dans la plateforme. L'ESP32-CAM assure la collecte des données provenant des capteurs ainsi que la capture des images via la caméra embarquée. Les données sont transmises à travers le protocole MQTT vers un serveur MQTT dédié. Ensuite, le serveur Backend développé avec NestJS traite les données, gère les notifications et assure la communication avec la base de données MongoDB ainsi qu'avec les APIs externes telles que Stripe pour le paiement en ligne, Google Auth pour l'authentification et OpenAI pour la détection des insultes. L'interface utilisateur est hébergée sur un serveur Frontend Angular permettant au résident de consulter et contrôler la maison intelligente à distance via des requêtes HTTP/REST.

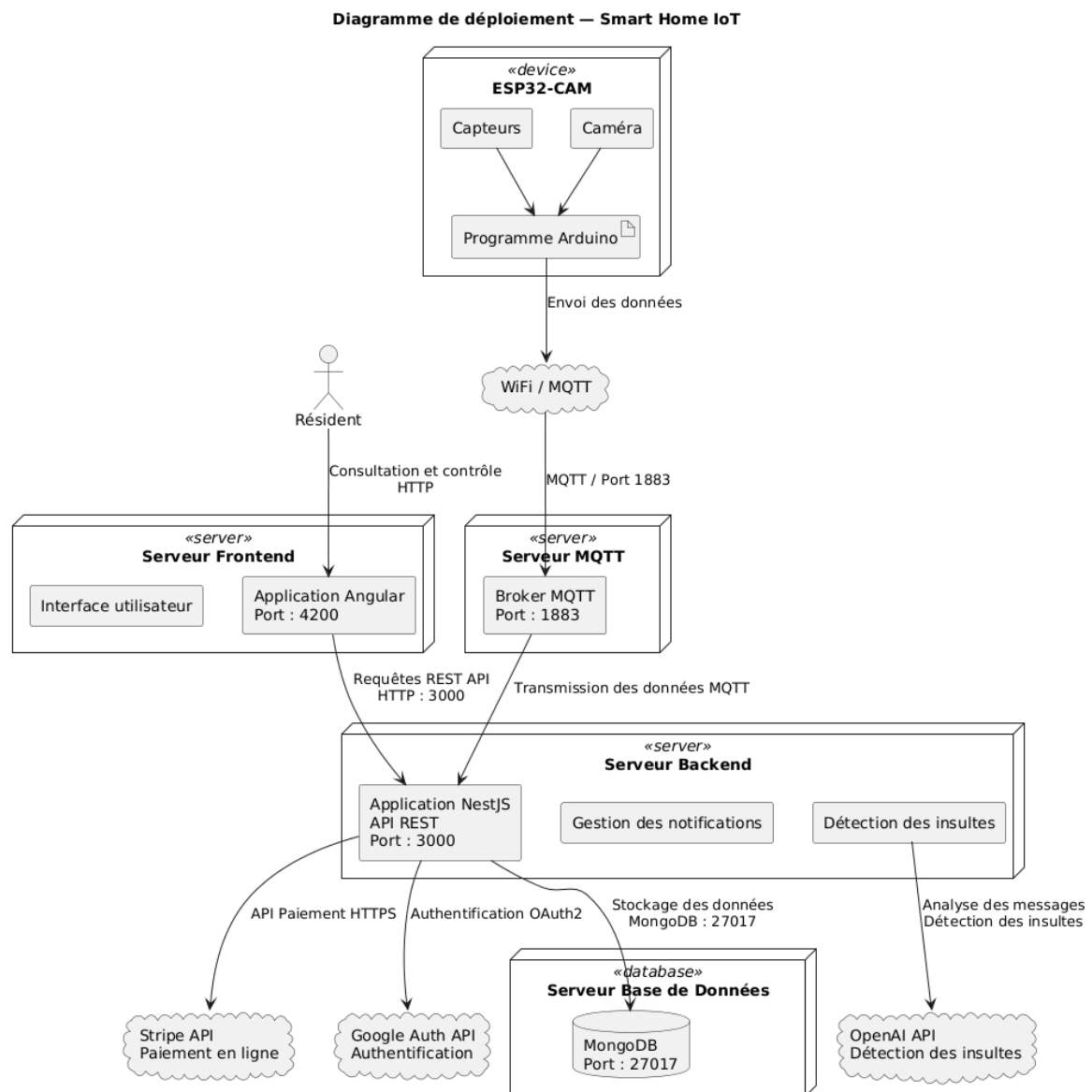


FIGURE 5.5 – Diagramme de déploiement du système Smart House IoT

Le diagramme montre que l'architecture du système repose sur une séparation claire entre les différents services afin d'améliorer la sécurité, la performance et la

maintenabilité. Le serveur Frontend héberge l'application Angular sur le port 4200, tandis que le serveur Backend NestJS expose les APIs REST sur le port 3000. La communication temps réel entre les objets connectés et le système est assurée par le Broker MQTT utilisant le port 1883. Les données sont stockées dans une base MongoDB hébergée sur un serveur dédié utilisant le port 27017. De plus, l'intégration des services externes permet d'ajouter des fonctionnalités avancées au système, notamment l'authentification sécurisée via Google Auth, la gestion des paiements avec Stripe et l'analyse intelligente des messages grâce à l'API OpenAI pour la détection des contenus insultants.

5.5 Présentation des interfaces réalisées

Cette section présente les différentes interfaces développées dans le cadre du Sprint3 du système Smart House. Elle illustre les principales fonctionnalités implémentées, notamment la consultation des notifications, la gestion de leur état ainsi que l'accès à l'historique des événements à travers l'application web.

a.Interface de consultation des notifications

L'interface suivante présente le module de gestion des notifications développé dans le cadre du Sprint3 du système Smart House. Elle permet au résident de consulter en temps réel les différentes alertes générées par les capteurs connectés ainsi que les événements détectés par le système.

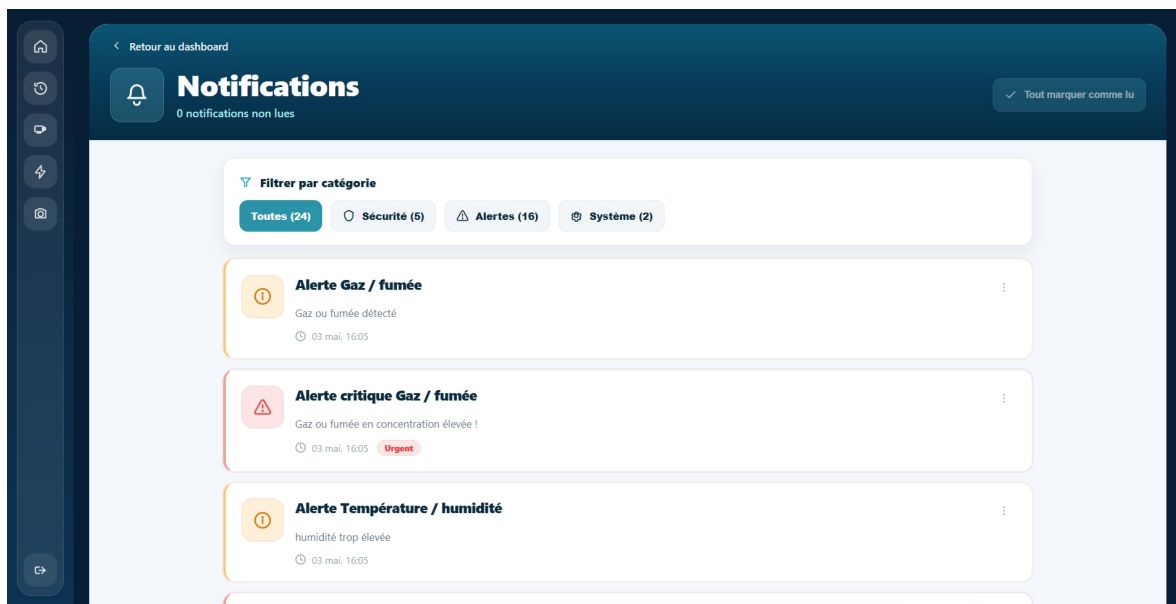


FIGURE 5.6 – Interface de consultation des notifications

Cette interface offre une vue claire et organisée des notifications reçues par le

résident. Les alertes sont classées par catégories telles que les notifications de sécurité, les alertes critiques ainsi que les notifications système. Le résident peut également filtrer les notifications selon leur type et marquer toutes les notifications comme lues afin d'améliorer la gestion des événements du Smart House. Cette interface contribue à assurer une surveillance efficace et une réaction rapide en cas d'incident détecté.

Après la gestion des notifications, le système permet également au résident de consulter l'historique complet des événements enregistrés dans l'application Smart House. L'interface suivante présente le module dédié à la consultation et au suivi des activités du système.

b.Interface de consultation de l'historique

L'interface suivante présente le module de consultation de l'historique développé dans le cadre du Sprint3 du système Smart House. Elle permet au résident d'accéder à l'ensemble des événements et actions enregistrés par le système afin d'assurer un suivi détaillé des activités de la maison intelligente.

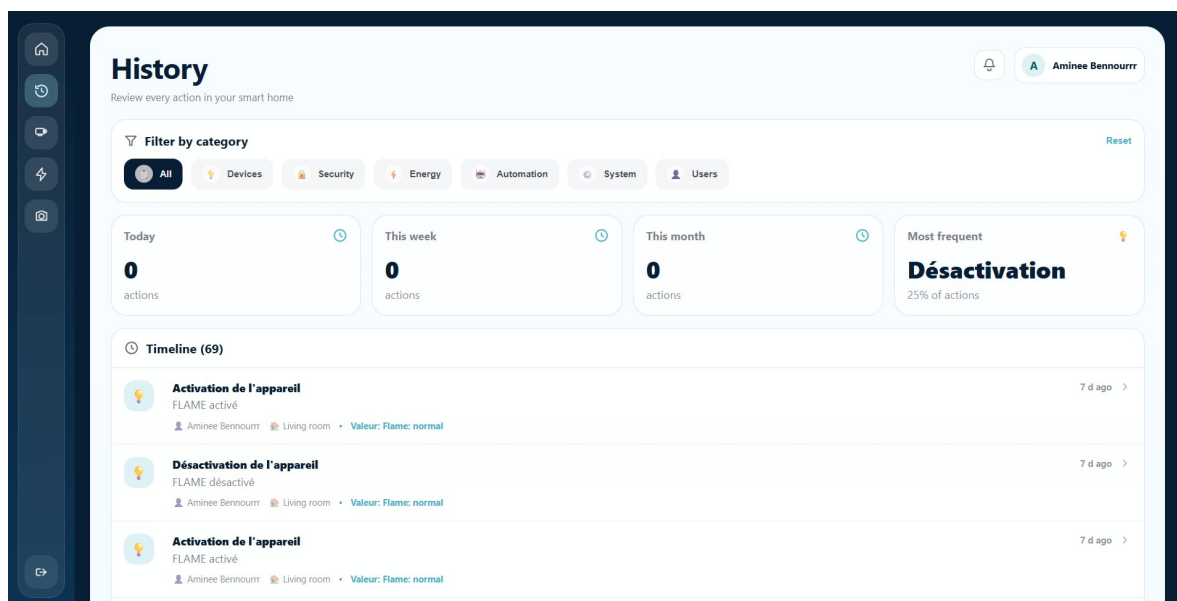


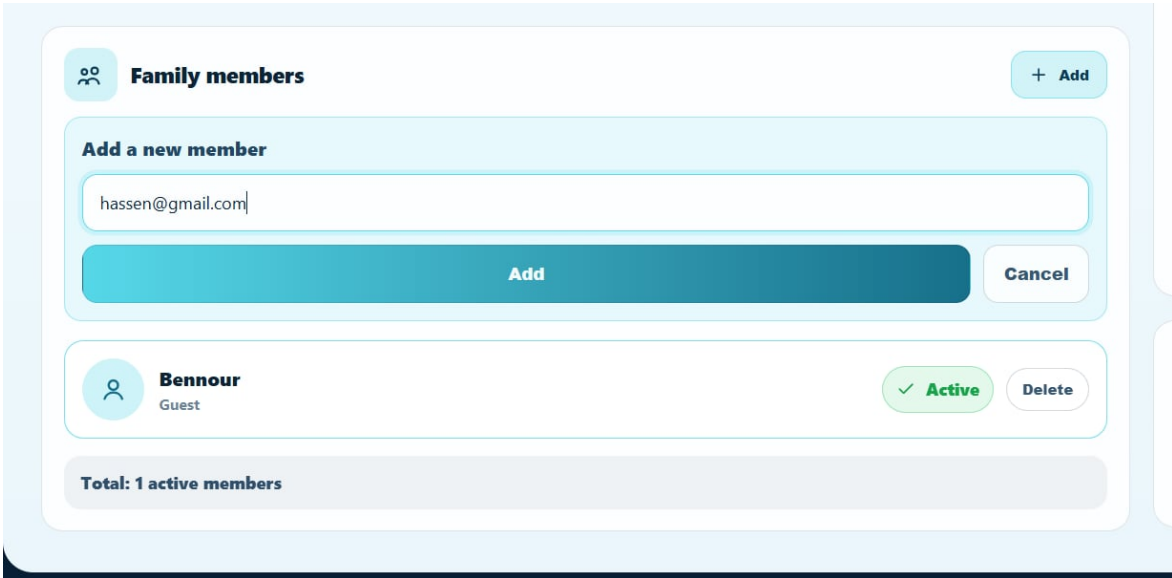
FIGURE 5.7 – Interface de consultation de l'historique

Cette interface offre une vue chronologique des différentes actions effectuées dans le système Smart House, telles que l'ajout ou la suppression d'appareils, les événements de sécurité ainsi que les opérations réalisées par les utilisateurs. Le résident peut filtrer les événements selon plusieurs catégories afin de faciliter la recherche et l'analyse des activités enregistrées. Des statistiques relatives aux actions quotidiennes, hebdomadaires et mensuelles sont également affichées afin de fournir une meilleure visibilité sur le fonctionnement global du système. Après la consultation des notifications et de l'historique des événements, le système Smart House permet également au résident de

gérer les membres associés à l'abonnement. L'interface suivante présente le module d'ajout d'un nouveau membre afin de lui accorder l'accès au système et aux différentes fonctionnalités de la maison intelligente.

c. Interface d'ajout d'un membre

L'interface suivante présente le module d'ajout d'un membre développé dans le cadre du système Smart House. Elle permet au résident de gérer les utilisateurs associés à l'abonnement en ajoutant de nouveaux membres afin de leur accorder l'accès aux différentes fonctionnalités de la maison intelligente.



The screenshot displays a user interface for managing family members. At the top, there is a header section with a person icon, the title "Family members", and a "+ Add" button. Below this is a section titled "Add a new member" which contains a text input field with the email "hassen@gmail.com". Under the input field are two buttons: a large teal "Add" button and a smaller white "Cancel" button. Below the "Add a new member" section is a list of existing members. The first member listed is "Bennour" with the role "Guest". To the right of the name is a green "Active" button with a checkmark and a white "Delete" button. At the bottom of the list, there is a summary bar that says "Total: 1 active members".

FIGURE 5.8 – Saisie des informations d'un nouveau membre

Cette interface permet au résident de renseigner l'adresse électronique du membre à ajouter au système. Après la saisie des informations requises, il peut confirmer l'opération en cliquant sur le bouton *Add*. Le système vérifie alors la validité des données avant de procéder à l'ajout du nouvel utilisateur à l'abonnement Smart House.

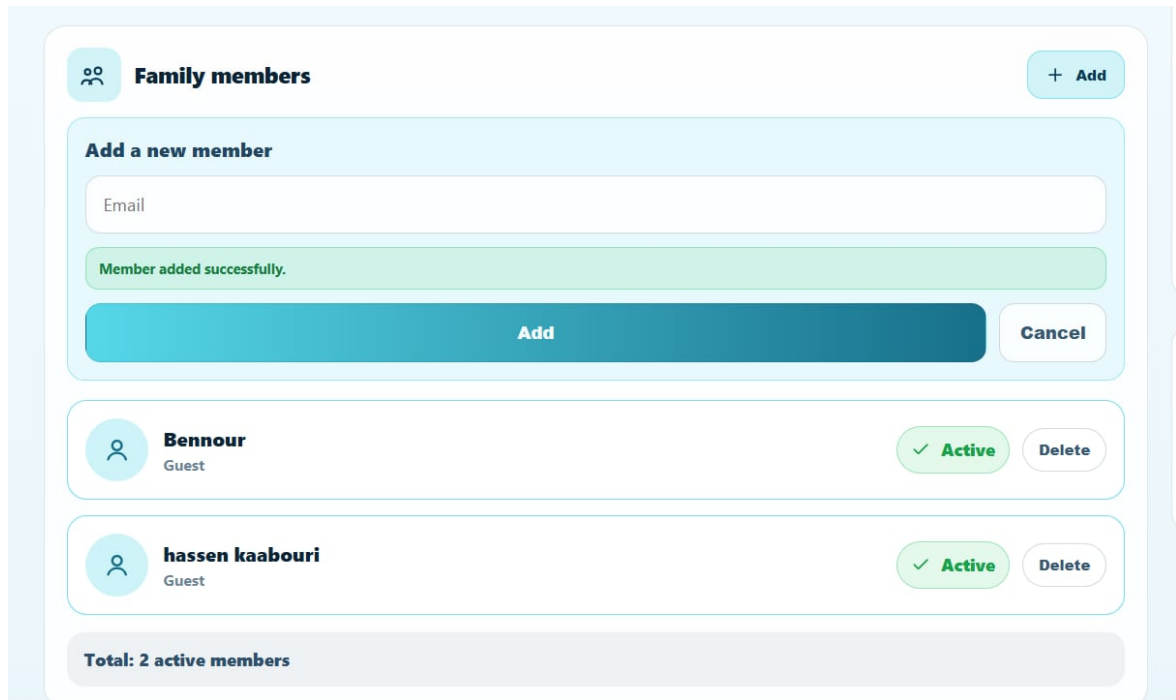


FIGURE 5.9 – Confirmation de l'ajout réussi d'un membre

Après la validation que l'utilisateur existe, un message de confirmation est affiché afin d'informer le résident que le membre a été ajouté avec succès. La liste des membres est automatiquement mise à jour et le nouvel utilisateur apparaît parmi les membres actifs. Cette fonctionnalité facilite le partage sécurisé de l'accès à la maison intelligente tout en garantissant une gestion simple et centralisée des utilisateurs.

Après l'ajout réussi d'un membre, le système affiche un message de confirmation et met automatiquement à jour la liste des utilisateurs associés à l'abonnement. Toutefois, afin de garantir une gestion fiable des membres, le système prend également en charge les cas d'échec lors de la tentative d'ajout.

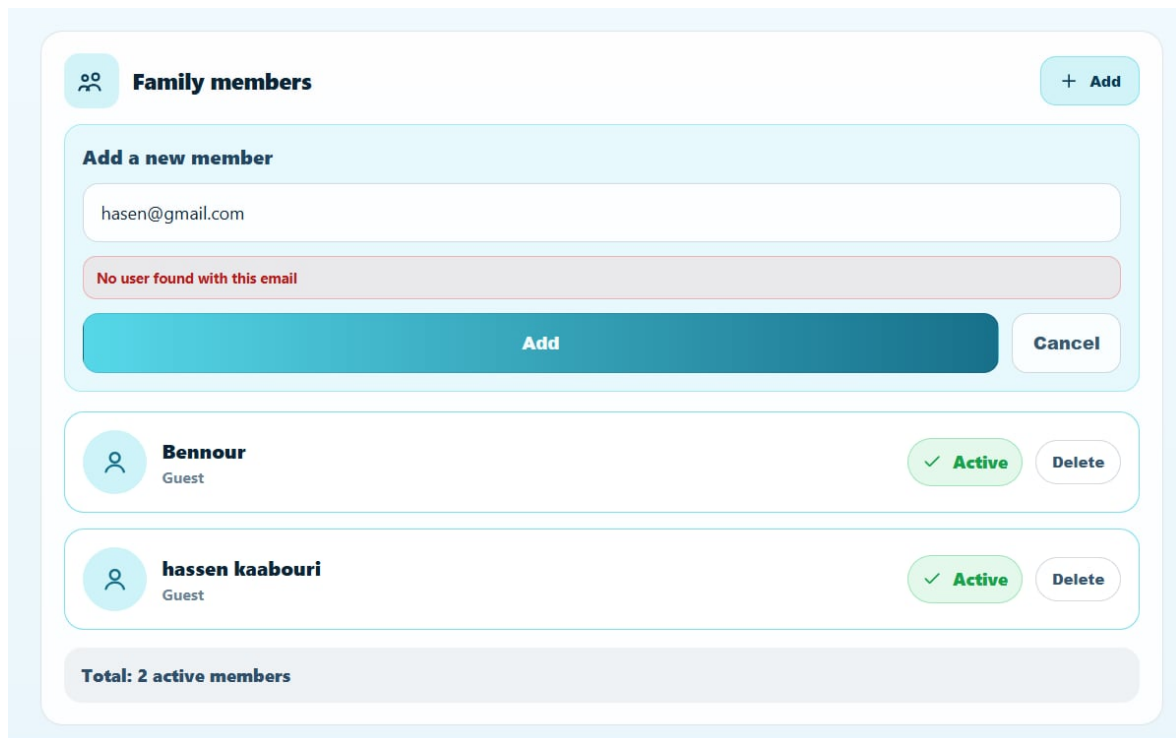


FIGURE 5.10 – Échec de l'ajout d'un membre

Dans le cas où l'adresse électronique saisie ne correspond à aucun utilisateur enregistré dans le système, l'opération d'ajout est annulée et un message d'erreur est affiché à l'écran. Cette notification informe le résident que l'utilisateur recherché n'existe pas et l'invite à vérifier l'exactitude de l'adresse électronique saisie avant de réessayer. Ce mécanisme permet d'éviter l'ajout de membres inexistants et contribue à maintenir l'intégrité des données du système Smart House.

Une fois les membres ajoutés, le système Smart House offre également au résident la possibilité de gérer les utilisateurs déjà enregistrés. Il peut notamment supprimer un membre devenu inactif ou ne nécessitant plus d'accès au système. L'interface suivante présente le module de suppression d'un membre permettant de retirer ses autorisations de manière simple et sécurisée.

d. Interface de suppression d'un membre

L'interface suivante présente le module de gestion des membres du système Smart House. Elle permet au résident de consulter les utilisateurs associés à l'abonnement ainsi que de supprimer un membre lorsque cela est nécessaire.

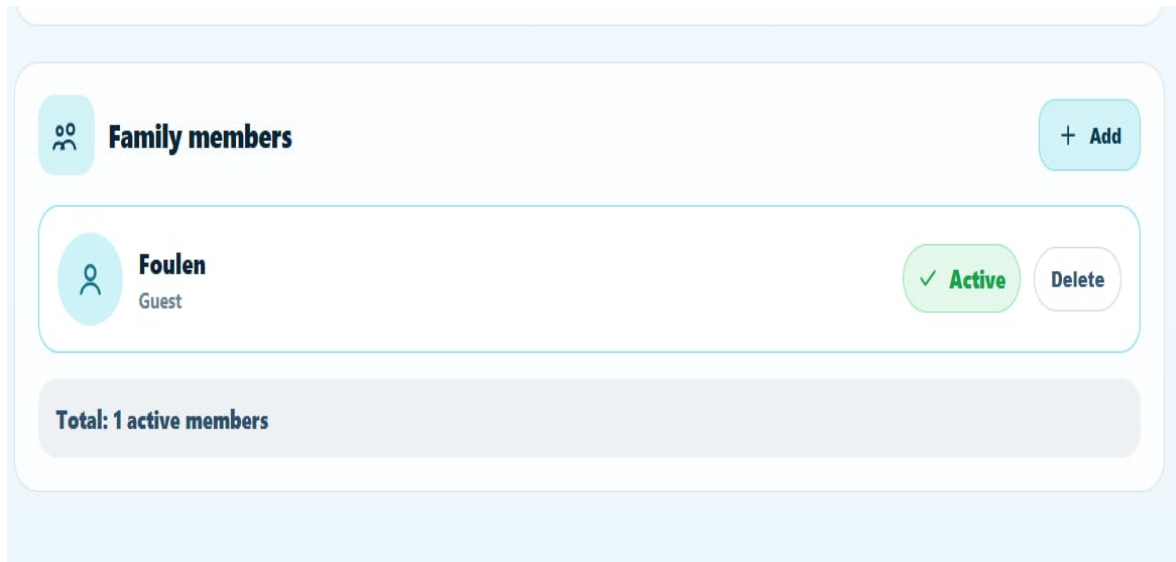


FIGURE 5.11 – Interface de suppression d'un membre

Cette interface affiche la liste des membres actifs associés au système Smart House avec leurs informations principales ainsi que leur statut. Le résident peut supprimer un membre à travers l'option dédiée afin de retirer son accès aux fonctionnalités du système. Cette fonctionnalité permet d'assurer une gestion flexible et sécurisée des utilisateurs tout en maintenant un contrôle efficace des accès à la maison intelligente.

Conclusion

Au cours de ce chapitre, nous avons présenté les différentes étapes de réalisation du Sprint3 du système Smart House consacré à la gestion des notifications, à la consultation de l'historique ainsi qu'à la gestion des membres de la famille. Nous avons d'abord étudié les besoins fonctionnels à travers les diagrammes d'analyse et de conception afin de mieux représenter les interactions entre le résident et le système.

Par la suite, nous avons présenté l'architecture de déploiement du système ainsi que les principales interfaces développées dans l'application web. Les fonctionnalités réalisées permettent au résident de recevoir des notifications en temps réel, consulter l'historique des événements et gérer les utilisateurs associés à l'abonnement de manière simple et sécurisée.

Les différents tests effectués ont permis de valider le bon fonctionnement des fonctionnalités implémentées ainsi que la communication entre les différents composants du système Smart House. Ce Sprint constitue ainsi une étape importante dans la mise en place d'une solution intelligente assurant une meilleure surveillance et gestion de la maison connectée.

Chapitre 6

Sprint 4 : Profil et abonnement

Introduction

Ce chapitre fait état de la quatrième itération du projet Smart House, centrée sur la mise en place des attributs de gestion du profil, des souscriptions et des contestations des habitants sur la plateforme. Durant cette période, diverses caractéristiques ont été intégrées pour optimiser la communication entre l'habitant et le dispositif via une interface contemporaine, évidente et fiable. Cette itération autorise spécifiquement l'habitant à administrer ses données individuelles, à examiner et ajuster sa souscription, de même qu'à acheminer des contestations en cas de difficultés survenues dans la plateforme. Ce chapitre expose par ailleurs les exigences fonctionnelles recensées, les divers croquis d'évaluation et d'élaboration réalisés, ainsi que les principaux écrans créés durant cette séquence du projet.

6.1 Sprint Goal

L'objectif de ce Sprint est de développer les fonctionnalités de gestion du profil utilisateur, des abonnements et des réclamations dans la plateforme Smart House. Elle permet à l'utilisateur de consulter et de modifier ses informations personnelles, de gérer son abonnement à travers les différentes offres disponibles et de soumettre des réclamations en cas de problème. Cette itération vise également à améliorer la sécurité, la fiabilité et l'expérience utilisateur grâce à des interfaces modernes, simples et adaptées aux besoins des habitants.

6.2 Sprint Backlog

Le tableau suivant présente le Sprint Backlog du Sprint4 dédié à la gestion du profil, des abonnements et des réclamations des résidents. Il regroupe les différentes user stories identifiées ainsi que leurs priorités, estimations et fonctionnalités associées.

TABLE 6.1 – Sprint Backlog – Profil & Abonnement

ID	Acteur	User Story	Priorité	Estimation	Fonctionnalité
11	Résident	En tant que résident, je peux gérer mon profil.	Moyenne	7 J	Gestion du profil résident
12	Résident	En tant que résident, je peux gérer mon abonnement.	Haute	7 J	Gestion des abonnements
13	Résident	En tant que résident, je peux soumettre une réclamation afin de signaler un problème.	Moyenne	7 J	Gestion des réclamations

Ce tableau met en évidence les principales fonctionnalités développées durant ce Sprint afin d'améliorer l'expérience du résident au sein de la plateforme Smart House. Les fonctionnalités implémentées permettent une meilleure gestion des informations personnelles, des abonnements et des réclamations à travers une interface simple et intuitive.

6.3 Analyse

L'objectif de cette étape d'analyse est de cerner les exigences fonctionnelles relatives à la gestion du profil, des abonnements et des réclamations au sein de la plateforme Smart House. Elle vise à établir les diverses interactions entre l'habitant et le système pour garantir une administration performante des données personnelles, des services souscrits ainsi que des requêtes transmises à l'administrateur. Les schémas produits illustrent les fonctionnalités clés développées au cours de cette période et représentent un fondement primordial pour l'élaboration et la mise en œuvre des interfaces du système.

6.3.1 Diagramme de cas d'utilisation du Sprint 4

Le diagramme de cas d'utilisation présenté ci-dessous illustre les différentes fonctionnalités offertes au résident dans le système Smart House concernant la gestion du profil, des réclamations ainsi que des abonnements. Ce diagramme met en évidence

les interactions entre les acteurs et les principales opérations disponibles au sein de la plateforme.

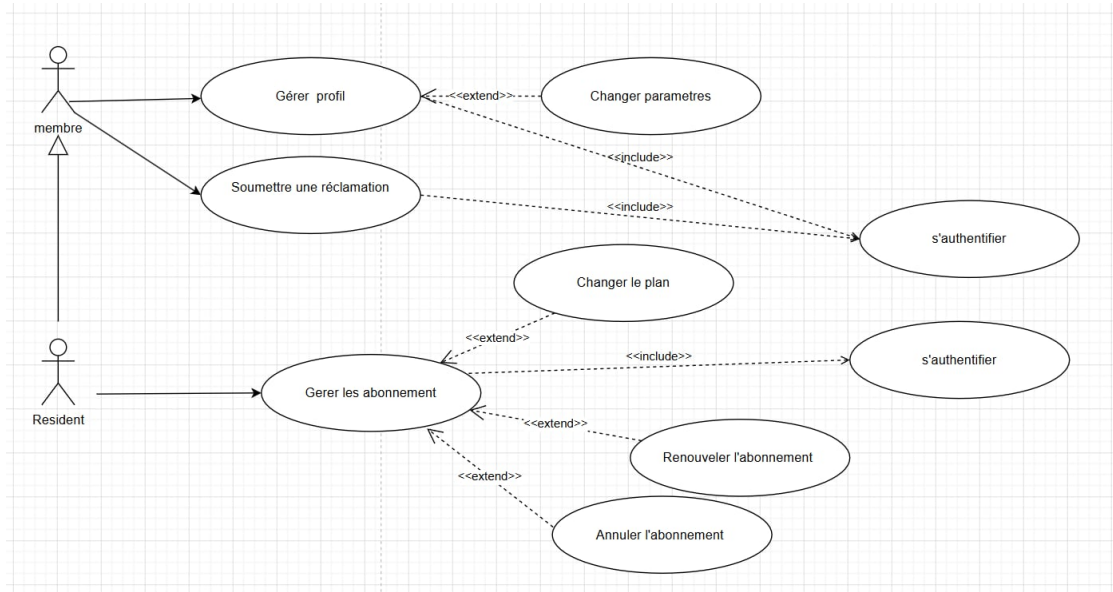


FIGURE 6.1 – Diagramme de cas d'utilisation de gestion du profil et des abonnements

Ce diagramme montre que le résident hérite des fonctionnalités de l'acteur membre, ce qui lui permet d'accéder aux opérations de gestion du profil et de soumission des réclamations. La fonctionnalité « Gérer profil » peut être étendue par le cas d'utilisation « Changer mot de passe », permettant ainsi à l'utilisateur de sécuriser son compte.

Le résident peut également gérer son abonnement à travers plusieurs opérations telles que le changement du plan d'abonnement, le renouvellement de l'abonnement ou encore l'annulation de celui-ci. Ces fonctionnalités sont représentées par des relations « extend », indiquant qu'elles constituent des extensions possibles du cas d'utilisation principal « Gérer les abonnements ».

Ainsi, ce diagramme met en évidence l'organisation des services liés à la gestion du compte utilisateur et des abonnements au sein de la plateforme Smart House.

6.3.2 Description textuelle des cas d'utilisation

Dans cette section, nous présentons les descriptions textuelles des principaux cas d'utilisation du Sprint4. Ces descriptions permettent de détailler les différentes interactions entre le résident et la plateforme Smart House ainsi que le comportement du système lors de l'exécution des fonctionnalités développées. Elles mettent également en évidence les conditions nécessaires au bon déroulement des opérations ainsi que les différents scénarios possibles.

Le tableau suivant présente la description textuelle du cas d'utilisation relatif à la gestion du profil du résident.

TABLE 6.2 – Description textuelle du cas d'utilisation : Gérer le profil

Élément	Description
Titre	Gérer le profil
Acteur	Résident
Description	Permettre au résident de consulter et modifier ses informations personnelles.
Pré-condition	Le résident doit être authentifié dans le système.
Postconditions	Les informations du profil sont mises à jour avec succès.
Scénario nominal	1. Le résident accède à son espace profil. 2. Le système affiche les informations personnelles. 3. Le résident modifie les informations souhaitées. 4. Le système vérifie les données saisies. 5. Le système enregistre les modifications.
Scénario alternatif	1. Les informations saisies sont invalides. 2. Le système affiche un message d'erreur. 3. Le résident corrige les informations puis valide à nouveau.

Ce tableau détaille la version textuelle du scénario d'utilisation concernant la gestion du profil résidentiel sur l'application Smart House. Il expose les diverses interactions entre le résident et le logiciel lorsqu'il consulte et change ses coordonnées. Il explique de surcroît les procédés de vérification actifs pour garantir l'exactitude, la sûreté et la discrétion des données introduites. Cette option permet alors au résident d'administrer son compte personnel avec aisance et de garder ses informations personnelles au goût du jour.

Le tableau suivant présente la description textuelle du cas d'utilisation relatif à la gestion des abonnements.

TABLE 6.3 – Description textuelle du cas d'utilisation : Gérer les abonnements

Élément	Description
Titre	Gérer les abonnements
Acteur	Résident
Description	Permettre au résident de consulter, renouveler ou modifier son abonnement.
Pré-condition	Le résident doit être connecté à la plateforme.
Postconditions	L'abonnement est mis à jour avec succès.
Scénario nominal	1. Le résident accède à la page des abonnements. 2. Le système affiche les offres disponibles. 3. Le résident sélectionne une offre. 4. Le système affiche les informations de paiement. 5. Le résident valide l'opération. 6. Le système confirme la mise à jour de l'abonnement.
Scénario alternatif	1. Le paiement échoue ou les informations sont incorrectes. 2. Le système affiche un message d'erreur. 3. Le résident peut réessayer l'opération.

Ce tableau présente le déroulement du module d'abonnement sur la plateforme Smart House. Il détaille les diverses phases par lesquelles le résident passe pour examiner les formules proposées, choisir celle qui lui convient, procéder au règlement puis valider la modification de son abonnement. Le schéma inclut également les procédures de vérification et les cas de figure de secours prévus par le système en cas de problème de règlement ou de données erronées. Cette option garantit ainsi une administration souple, fiable et performante des prestations offertes par la plateforme.

Le tableau suivant décrit le cas d'utilisation lié à la soumission des réclamations.

TABLE 6.4 – Description textuelle du cas d'utilisation : Soumettre une réclamation

Élément	Description
Titre	Soumettre une réclamation
Acteur	Résident
Description	Permettre au résident de signaler un problème rencontré dans le système.
Pré-condition	Le résident doit être authentifié.
Postconditions	La réclamation est enregistrée dans le système.
Scénario nominal	1. Le résident accède à la page des réclamations. 2. Le système affiche le formulaire de réclamation. 3. Le résident saisit les informations nécessaires. 4. Le système vérifie les données. 5. Le système enregistre la réclamation.
Scénario alternatif	1. Les informations saisies sont incomplètes. 2. Le système affiche un message d'erreur. 3. Le résident complète les informations puis valide à nouveau.

Ce tableau détaille les diverses phases relatives au dépôt d'une demande d'intervention sur le portail Smart House. Il met en lumière le parcours de l'habitant, depuis l'atteinte du formulaire de signalement jusqu'à l'inscription effective de la requête dans l'outil. La présentation expose aussi les dispositifs de vérification employés pour s'assurer de la pertinence et de l'exhaustivité des renseignements fournis, avant la transmission de la saisie. Si des erreurs surviennent ou si des champs sont manquants, le logiciel présente une notification d'anomalie, offrant à l'occupant la possibilité de rectifier les éléments avant une nouvelle confirmation.

Ces descriptions textuelles servent à exposer les fonctionnalités majeures élaborées au cours du Sprint4, ainsi que les interactions diverses entre l'habitant et la domotique. Elles facilitent l'appréhension des modes opératoires et des réactions du système, tout en formant un socle indispensable pour les étapes de conception, de codage et de vérification des fonctionnalités intégrées.

6.3.3 Diagramme de séquence du Sprint 4

Dans le cadre du module de gestion des réclamations, il est essentiel de modéliser les interactions entre le résident, le système Smart House et le système d'intelligence artificielle chargé de la détection des insultes. Le diagramme de séquence système suivant illustre le scénario de soumission d'une réclamation ainsi que le processus de vérification automatique du contenu du message avant son envoi.

Diagramme de Séquence Système (Boîte Noire) - Soumettre une Réclamation

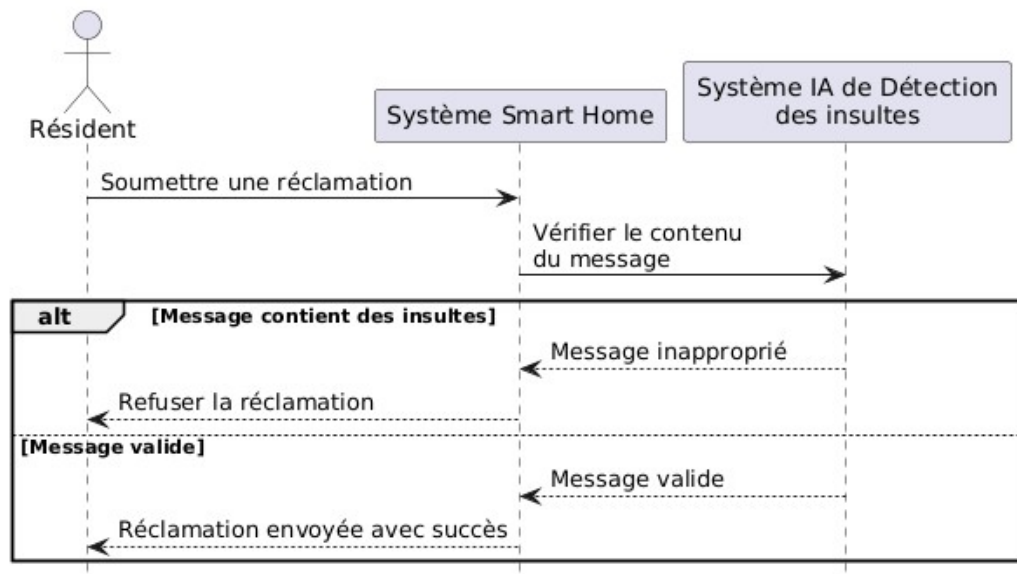


FIGURE 6.2 – Diagramme de Séquence Système (Boîte Noire) — Soumettre une Réclamation

Ce diagramme illustre les différentes phases d'une demande soumise par un résident. Suite à l'envoi du message, la plateforme Smart House le transmet de façon automatique à son module d'intelligence artificielle. Ce dernier analyse le texte afin d'identifier toute éventualité d'injures ou de propos désobligeants. Si le message s'avère offensant, la réclamation est rejetée, et un message d'erreur est affiché, signalant un contenu non conforme. Autrement, la demande est approuvée, puis consignée avec succès dans le système. Ce dispositif vise à garantir un cadre d'échange courtois et à optimiser la qualité des interactions entre les occupants et le gestionnaire de la solution Smart House.

6.4 Conception

Après avoir analysé les différents besoins fonctionnels du Sprint4, nous passons à la phase de conception du système. Cette étape permet de modéliser les principales composantes de la plateforme Smart House à travers différents diagrammes UML afin de représenter la structure statique et dynamique du système. Les diagrammes présentés

dans cette section facilitent la compréhension de l'architecture logicielle ainsi que les interactions entre les différentes entités liées à la gestion du profil, des abonnements et des réclamations.

6.5 Diagramme de classes Sprint 4

Ce diagramme de classes est composé de quatre classes principales : **Membre**, **Resident**, **Abonnement** et **Reclamation**.

La classe **Membre** représente l'utilisateur du système. Elle regroupe les informations nécessaires à l'authentification et à la gestion du profil, notamment l'identifiant utilisateur, le mot de passe et le nom. Elle offre également plusieurs opérations telles que la connexion, la déconnexion, la mise à jour du profil et la modification du mot de passe.

La classe **Resident** est une spécialisation de la classe **Membre**. Elle hérite de l'ensemble des attributs et méthodes de cette dernière et ajoute des fonctionnalités spécifiques liées à la gestion des abonnements, notamment le changement et le renouvellement de l'abonnement.

La classe **Abonnement** permet de gérer les informations relatives aux abonnements des utilisateurs. Elle contient l'identifiant de l'abonnement, la date de création ainsi que la date d'expiration. Les opérations associées permettent de créer, activer, renouveler, annuler et vérifier l'expiration d'un abonnement.

La classe **Reclamation** représente les réclamations soumises par les membres. Elle contient les informations relatives à une réclamation, telles que son identifiant et son message. Les méthodes définies permettent de créer, mettre à jour, consulter, envoyer et fermer une réclamation.

Les relations présentes dans ce diagramme montrent qu'un membre peut posséder plusieurs réclamations, tandis que plusieurs membres peuvent être associés à un même abonnement. De plus, la relation d'héritage entre les classes **Resident** et **Membre** permet de spécialiser les fonctionnalités du résident tout en réutilisant les caractéristiques communes des utilisateurs.

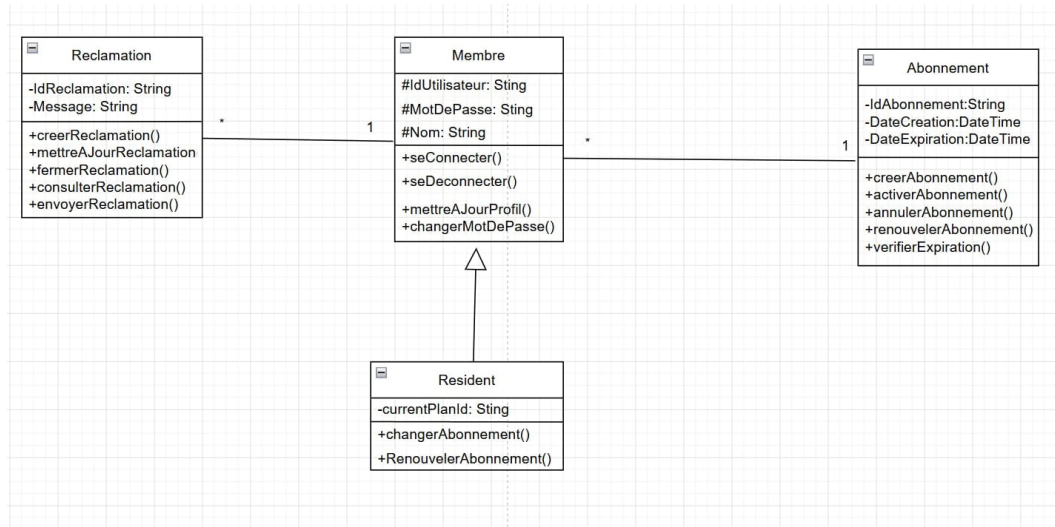


FIGURE 6.3 – Diagramme de classes Sprint 4

Ce diagramme met ainsi en évidence la structure des données et les interactions nécessaires à la gestion des membres, des abonnements et des réclamations au sein de la plateforme Smart House.

6.5.1 Diagramme d'activité — Gestion des abonnements

Le diagramme d'activité présenté ci-dessous illustre les différentes étapes liées à la gestion des abonnements au sein de la plateforme Smart House. Il décrit le processus suivi par le résident depuis l'accès à la page des abonnements jusqu'à l'activation ou le renouvellement de son abonnement après validation du paiement. Ce diagramme met également en évidence les interactions entre l'utilisateur et le système lors des différentes opérations de consultation et de paiement.



FIGURE 6.4 – Diagramme d'activité — Gestion des abonnements

De plus, cette représentation illustre les diverses issues envisageables lors de la déroulement du processus. Si le règlement est approuvé, l'application procède à l'activation ou au renouvellement de l'abonnement, enregistre les données pertinentes dans le répertoire puis communique un avis de succès. Au contraire, si le règlement est refusé ou que les données fournies sont erronées, l'application signale un dysfonctionnement et offre au domicilié la possibilité de retenter la démarche. Cette conceptualisation facilite ainsi une compréhension approfondie du mode opératoire général de la gestion des souscriptions et des divers échanges entre le domicilié et l'interface Smart House.

6.5.2 Diagramme d'activité — Soumettre une réclamation

Le diagramme d'activité suivant présente le processus de soumission d'une réclamation par le résident au sein de la plateforme Smart House. Il décrit les différentes étapes permettant à l'utilisateur d'accéder au formulaire de réclamation, saisir les informations nécessaires puis envoyer sa demande au système pour traitement.

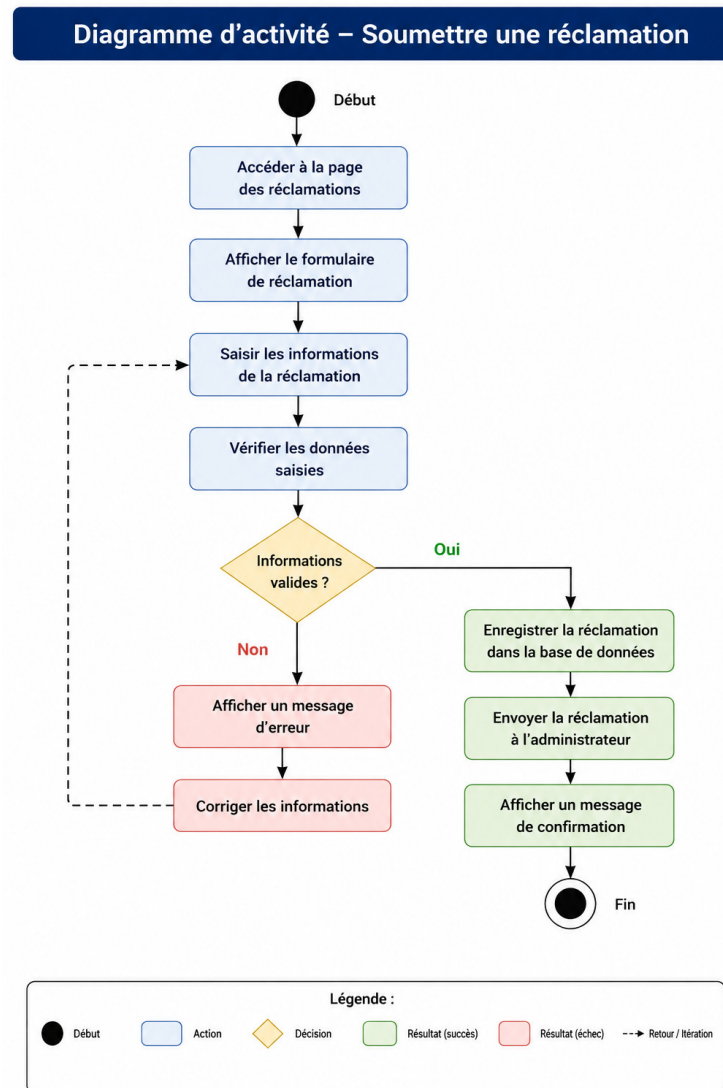


FIGURE 6.5 – Diagramme d'activité — Soumettre une réclamation

Ce diagramme d'activité représente le parcours de dépôt d'une réclamation sur la plateforme Smart House. Il détaille les différentes étapes effectuées par l'occupant, depuis l'accès à la rubrique des réclamations jusqu'à la transmission de la requête à l'administrateur. Le schéma met en lumière les échanges entre l'occupant et le système lors de la présentation du formulaire, de la saisie des renseignements, et de la vérification des informations avant l'enregistrement du dossier. De plus, ce diagramme expose les processus de contrôle instaurés pour assurer la justesse et la crédibilité des données

fournies. Si les informations sont conformes, le système enregistre le dossier, envoie la demande à l'administrateur, puis affiche une notification de succès. Si, au contraire, les informations fournies sont erronées ou partielles, le système signale une anomalie et permet à l'occupant de rectifier les informations avant une nouvelle vérification. Cette représentation permet ainsi d'appréhender plus clairement le fonctionnement général du système de suivi des réclamations, ainsi que les diverses interactions entre l'occupant et l'environnement Smart House.

6.6 Présentation des interfaces réalisées

Les interfaces réalisées durant ce Sprint permettent au résident de gérer facilement son profil, son abonnement ainsi que ses réclamations à travers une interface moderne, intuitive et sécurisée. Ces interfaces ont été développées avec Angular afin d'offrir une meilleure expérience utilisateur et une navigation fluide entre les différentes fonctionnalités de la plateforme.

a. Interface de gestion du profil

Cette interface permet au résident de consulter et modifier ses informations personnelles telles que le nom, l'adresse email et le mot de passe. Elle offre également une validation sécurisée des données saisies afin d'assurer la protection des informations personnelles de l'utilisateur.

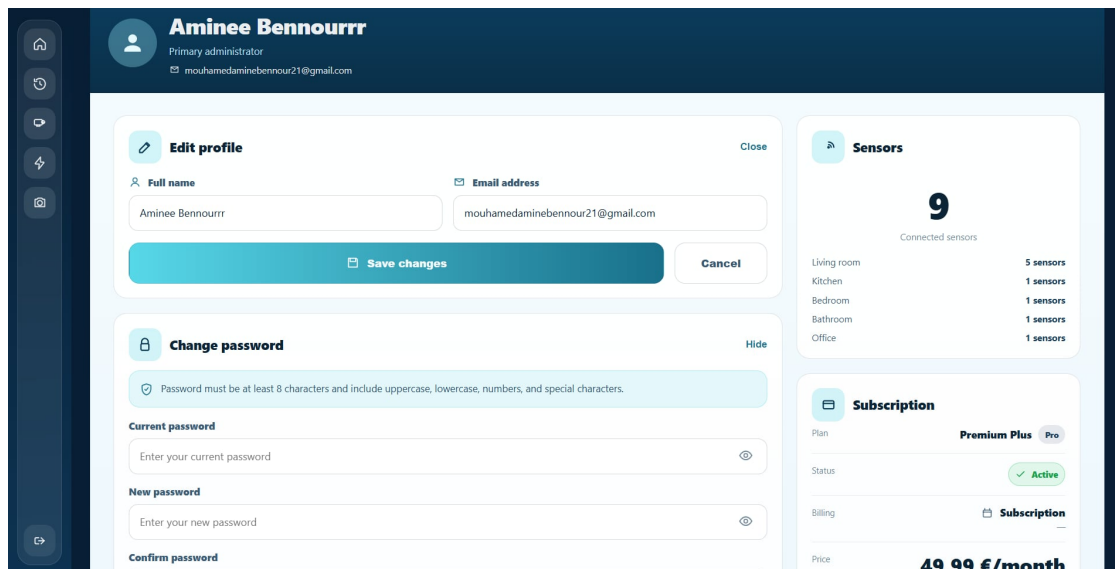


FIGURE 6.6 – Interface de gestion du profil

Cette interface de gestion du profil constitue une étape importante pour l'administration des informations personnelles de l'utilisateur. Afin d'assurer une meilleure gestion des services proposés par la plateforme, nous allons maintenant présenter l'interface

dédiée à la gestion des abonnements, permettant de consulter et de gérer les différentes offres disponibles. Cette interface permet au résident de consulter les différentes offres disponibles, renouveler son abonnement ou modifier son plan selon ses besoins. Le système affiche les détails des abonnements ainsi que les fonctionnalités associées à chaque offre.

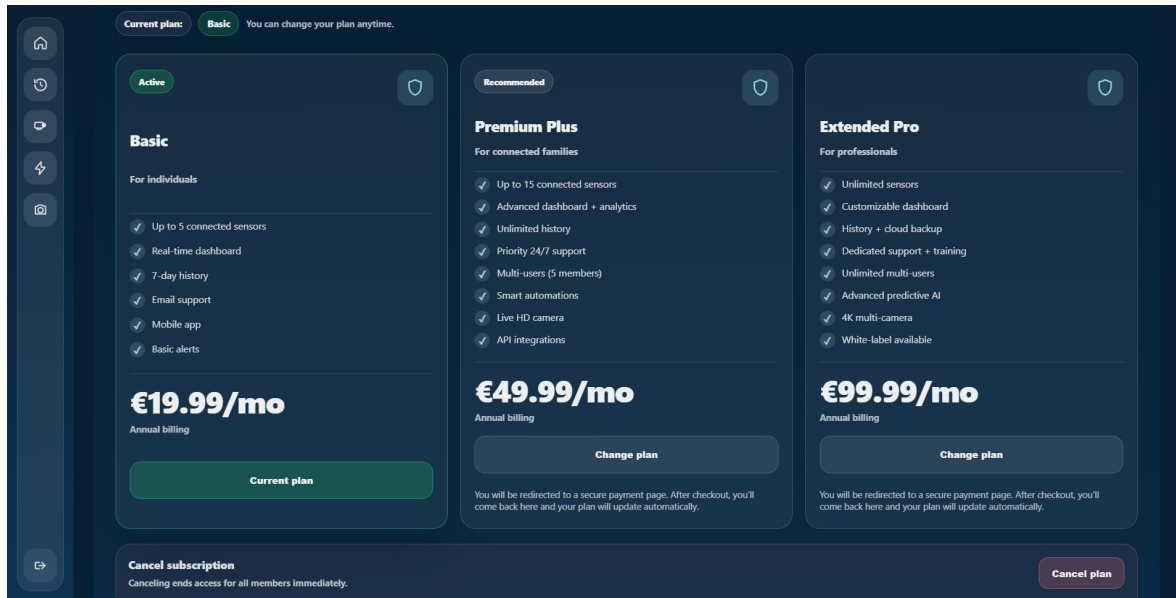


FIGURE 6.7 – Interface de gestion des abonnements

La figure ci-dessus présente l'interface de gestion des abonnements. Cette interface facilite la consultation des offres ainsi que la mise à jour des abonnements à travers une interface claire et intuitive. Dans la section suivante, nous présenterons l'interface de soumission des réclamations, permettant aux utilisateurs de signaler facilement les problèmes rencontrés et d'assurer un meilleur suivi des demandes.

b. Interface de soumission des réclamations

Cette interface offre au résident la possibilité de signaler rapidement un problème rencontré dans le système en envoyant une réclamation détaillée à l'administrateur. Elle permet également de suivre l'état des réclamations envoyées.

Energy management
Consumption, production and energy analytics

Send a complaint
Describe the issue and we will review it.

Title
Short summary

Message
Explain what happened...

Cancel Send

64 kWh
Total consumption
This week

69 kWh
Solar production
This week

22.40 TND
Estimated cost
~0.35 TND / kWh

9.1 kWh
Daily average
This week

Cost analysis
Daily cost trend

FIGURE 6.8 – Interface de soumission des réclamations

La figure ci-dessus montre l'interface de soumission des réclamations. Cette interface permet au résident de communiquer facilement avec l'administrateur afin de signaler les problèmes rencontrés sur la plateforme Smart House.

c.Détection des réclamations inappropriées par l'IA

Afin de garantir un environnement de communication respectueux et professionnel, le système intègre un mécanisme d'intelligence artificielle capable de détecter automatiquement les contenus offensants ou inappropriés dans les réclamations envoyées par les résidents. Lorsqu'un message contient des insultes ou des termes inacceptables, la réclamation est bloquée et un message d'avertissement est affiché à l'utilisateur.

Energy management
Consumption, production and energy analytics

Send a complaint
Describe the issue and we will review it.

Title
wrong price

Message
shit , this cant be true

Your complaint contains offensive language. Please rephrase and try again.

Cancel Send

64 kWh
Total consumption
This week

69 kWh
Solar production
This week

22.40 TND
Estimated cost
~0.35 TND / kWh

9.1 kWh
Daily average
This week

Cost analysis
Daily cost trend

FIGURE 6.9 – Détection automatique des contenus offensants dans une réclamation

La figure ci-dessus illustre le fonctionnement du système de détection intelligent lors

de la saisie d'une réclamation contenant un langage offensant. L'intelligence artificielle analyse le contenu du message en temps réel et empêche l'envoi de la réclamation tant que le texte n'est pas reformulé de manière appropriée. Cette fonctionnalité contribue à améliorer la qualité des échanges entre les résidents et l'administrateur de la plateforme Smart House.

Conclusion

Dans ce chapitre, nous avons présenté les différentes fonctionnalités développées durant le Sprint4 relatives à la gestion du profil, des abonnements et des réclamations. Nous avons également décrit les différentes phases d'analyse, de conception, de réalisation et de validation des fonctionnalités implémentées. Ce Sprinta permis d'améliorer l'expérience utilisateur et d'enrichir les services proposés par la plateforme Smart House.

Les différentes interfaces réalisées offrent au résident une gestion plus simple, intuitive et sécurisée de son espace personnel ainsi qu'une meilleure interaction avec les services de la plateforme. Les tests effectués ont également permis de valider le bon fonctionnement des fonctionnalités développées et de garantir la fiabilité du système.

Chapitre 7

Sprint 5 : Administration

Introduction

Ce Sprint est focalisé sur la conception des modules d'administration de la plateforme Smart House. Il a pour objectif d'offrir à l'administrateur un portail unique pour le suivi complet de l'écosystème intelligent. Les travaux accomplis lors de ce Sprint portent essentiellement sur la maîtrise des adhésions, le traitement des doléances des occupants et l'analyse des informations collectées par les capteurs IoT des diverses demeures intelligentes. Ce Sprint participe donc à l'optimisation de la supervision globale de la plateforme et à une administration performante des membres et des dispositifs connectés.

7.1 Sprint Goal

L'objectif principal de ce Sprint est de développer l'espace administrateur du système Smart House afin de permettre la gestion des abonnements, le traitement des réclamations ainsi que la consultation des données des capteurs des maisons connectées.

7.2 Sprint Backlog

TABLE 7.1 – Sprint Backlog - Administration

ID	Acteur	User Story	Priorité	Estimation	Fonctionnalité
15	Administrateur	En tant qu'administrateur, je peux gérer les abonnements.	Haute	9 J	Gestion des abonnements
16	Administrateur	En tant qu'administrateur, je peux gérer les réclamations.	Moyenne	7 J	Gestion des réclamations
17	Administrateur	En tant qu'administrateur, je peux consulter les valeurs des capteurs par maison.	Moyenne	5 J	Supervision des données IoT

Le tableau mentionné ci-dessus détaille les différentes user stories sélectionnées pour le Sprint 5, ainsi que les fonctionnalités liées à chaque besoin métier. Ce Sprint est majoritairement axé sur l'administration et la supervision de la plateforme Smart House. Les actions menées facilitent la gestion des abonnements, le suivi des doléances des utilisateurs, et la visualisation des données des capteurs pour optimiser la surveillance et l'entretien du système intelligent.

7.3 Analyse

Cette phase d'analyse a pour objectif d'identifier les différents besoins fonctionnels liés à l'administration de la plateforme Smart House. Elle permet de définir les interactions entre l'administrateur et le système afin de garantir une gestion centralisée et efficace des différentes fonctionnalités. L'analyse couvre principalement la gestion des abonnements, le traitement des réclamations des résidents ainsi que la supervision des données générées par les capteurs des maisons connectées. Cette étape constitue une base essentielle pour la conception et le développement des fonctionnalités du Sprint 5.

7.3.1 Diagramme de cas d'utilisation

La figure suivante présente le diagramme de cas d'utilisation du Sprint 5 consacré à l'administration du système Smart House. Ce diagramme illustre les différentes interactions entre l'administrateur et la plateforme afin d'assurer une gestion centralisée et efficace des fonctionnalités du système.

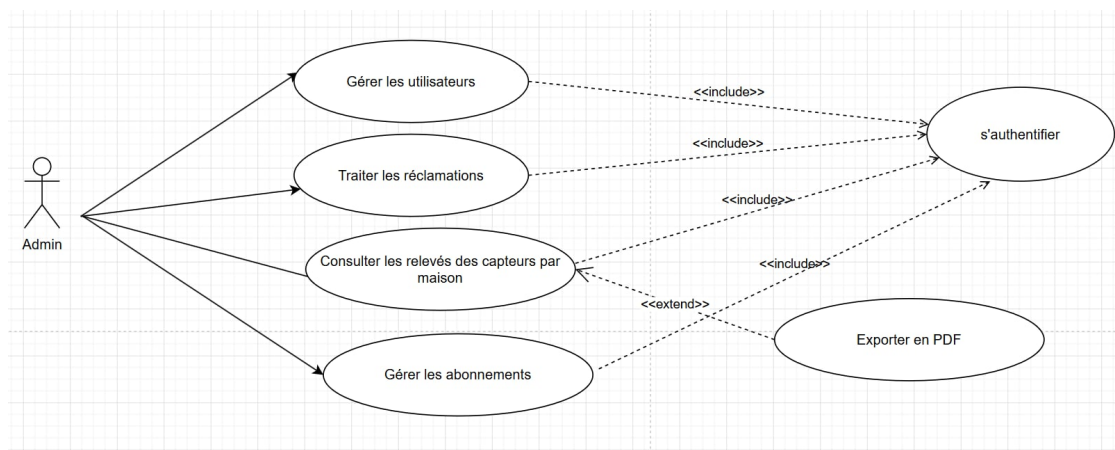


FIGURE 7.1 – Diagramme de cas d'utilisation du Sprint 5

La représentation visuelle ci-dessus illustre les fonctions essentielles dont dispose le responsable dans le portail d'administration de la solution Smart House. Via cette interface, le responsable a la possibilité de gérer les membres, de traiter les requêtes, de contrôler les informations provenant des capteurs des domiciles connectés, et de gérer les

adhésions des habitants. Le schéma indique en outre que certaines fonctions requièrent une validation d'identité préalable à l'exécution des diverses actions de gestion. Une fois l'accès accordé, le responsable peut examiner et altérer les détails concernant les membres et les adhésions, suivre les réclamations soumises par les habitants, et surveiller les données environnementales recueillies par les dispositifs IoT. En outre, la plateforme propose la fonction d'exporter les relevés des capteurs en format PDF pour simplifier la lecture, le stockage et l'étude des informations produites par les résidences intelligentes. Cette illustration permet ainsi d'observer l'intégralité des échanges entre le responsable et l'écosystème Smart House au cours de la période de développement 5.

7.3.2 Description textuelle des cas d'utilisation

Ce chapitre détaille les diverses fonctionnalités élaborées durant le Sprint 5, axé sur l'administration de la plateforme Smart House. Il met en lumière les solutions technologiques mises en œuvre côté backend, frontend et base de données, pour permettre une gestion unifiée des souscriptions, des requêtes clients et des informations relatives aux habitations connectées. Les interfaces développées visent à optimiser l'expérience des administrateurs et à simplifier la surveillance générale du système.

a. Gérer les réclamations

Le tableau suivant présente la description textuelle du cas d'utilisation « Gérer les réclamations ». Ce cas d'utilisation décrit les différentes interactions permettant à l'administrateur de consulter, suivre et traiter les réclamations envoyées par les résidents de la plateforme Smart House.

TABLE 7.2 – Cas d'utilisation : Gérer les réclamations

Titre	Gérer les réclamations
Acteur	Administrateur
Pré-condition	L'administrateur doit être authentifié dans le système
Post-condition	La réclamation est traitée et son état est mis à jour
Description	Permettre à l'administrateur de consulter les réclamations envoyées par les résidents, de visualiser leurs détails et de traiter les problèmes signalés.
Scénario nominal	1. L'administrateur s'authentifie dans le système. 2. Le système affiche la liste des réclamations. 3. L'administrateur sélectionne une réclamation. 4. Le système affiche les détails de la réclamation. 5. L'administrateur traite la réclamation. 6. Le système met à jour l'état de la réclamation. 7. Le système affiche un message de confirmation.
Scénario alternatif	1. Aucune réclamation n'est disponible. 2. Le système affiche un message indiquant qu'aucune réclamation n'a été trouvée.

Cette fonctionnalité est un atout majeur pour faciliter les échanges entre les utilisateurs et la gestion de la plateforme. Elle permet un suivi pertinent des soucis rapportés, assurant ainsi une qualité de prestation supérieure pour les foyers connectés. Ce texte descriptif détaille les diverses actions entreprises par l'administrateur lors du traitement des retours. Une fois connecté au système, il peut visualiser l'ensemble des demandes, examiner les spécificités d'une requête particulière, et exécuter les démarches requises pour corriger le dysfonctionnement signalé. Le système procède alors à la modification du statut de la demande et affiche une notification validant l'exécution réussie. Une procédure distincte est également prévue si aucune demande n'est enregistrée, pour avertir explicitement l'administrateur.

En conséquence, ce scénario d'utilisation participe à une meilleure gestion des problèmes et renforce le contrôle général de notre système de domotique.

b.Gérer les abonnements

Le tableau suivant présente la description textuelle du cas d'utilisation « Gérer les abonnements ». Ce cas d'utilisation décrit les différentes interactions entre l'administrateur et le système lors des opérations de consultation, de modification et de suppression des abonnements des utilisateurs.

TABLE 7.3 – Cas d'utilisation : Gérer les abonnements

Titre	Gérer les abonnements
Acteur	Administrateur
Pré-condition	L'administrateur doit être authentifié dans le système
Post-condition	Les informations des abonnements sont mises à jour avec succès
Description	Permettre à l'administrateur de consulter, modifier ou supprimer les abonnements des utilisateurs afin d'assurer une bonne gestion des services proposés par la plateforme.
Scénario nominal	1. L'administrateur s'authentifie dans le système. 2. Le système affiche la liste des abonnements. 3. L'administrateur sélectionne un abonnement. 4. Le système affiche les détails de l'abonnement. 5. L'administrateur modifie ou supprime l'abonnement. 6. Le système enregistre les modifications. 7. Le système affiche un message de confirmation.
Scénario alternatif	1. Les informations saisies sont invalides. 2. Le système affiche un message d'erreur. 3. L'administrateur corrige les informations. 4. Le cas d'utilisation reprend à l'étape 5 du scénario nominal.

Ce dispositif aide à garantir un maniement performant des prestations offertes par le logiciel Smart House, en donnant à la personne en charge une maîtrise globale des souscriptions enregistrées au sein du dispositif. Ce compte-rendu narratif souligne les différentes phases accomplies par la personne en charge pour organiser les souscriptions des utilisateurs. Suite à sa connexion au logiciel, la personne en charge peut prendre connaissance de l'inventaire des souscriptions disponibles, se pencher sur les particularités d'une souscription donnée puis apporter les ajustements requis ou effacer une souscription

selon les nécessités. Le dispositif enregistre par la suite les modifications effectuées et présente une notification de succès pour signaler à la personne en charge le bon achèvement de l'action. Une situation distincte est aussi anticipée si les renseignements fournis ne sont pas valides, offrant ainsi à la personne en charge la possibilité de rectifier les informations avant de poursuivre l'action. Par conséquent, ce scénario d'usage participe à l'optimisation de la gestion des prestations et assure un arrangement structuré et sûr des souscriptions au sein du dispositif Smart House.

7.4 Conception

La phase de conception représente une période capitale dans la création des fonctions destinées à l'administration de la domotique. Elle permet de fixer la structure des divers éléments constitutifs du dispositif et les liens qui les unissent, pour garantir un suivi optimal des adhésions, des plaintes et des informations relatives aux habitations intelligentes. Cette étape prépare également la concrétisation des fonctionnalités du Sprint 5, tout en assurant une conception limpide, logique et apte à s'adapter aux évolutions futures.

7.4.1 Diagramme de classes du Sprint 5

La figure suivante présente le diagramme de classes du Sprint 5 consacré à l'administration du système Smart House. Ce diagramme illustre les principales classes utilisées dans l'espace administrateur ainsi que les relations entre elles. Il permet de mieux comprendre la structure des données et le fonctionnement global des fonctionnalités d'administration.

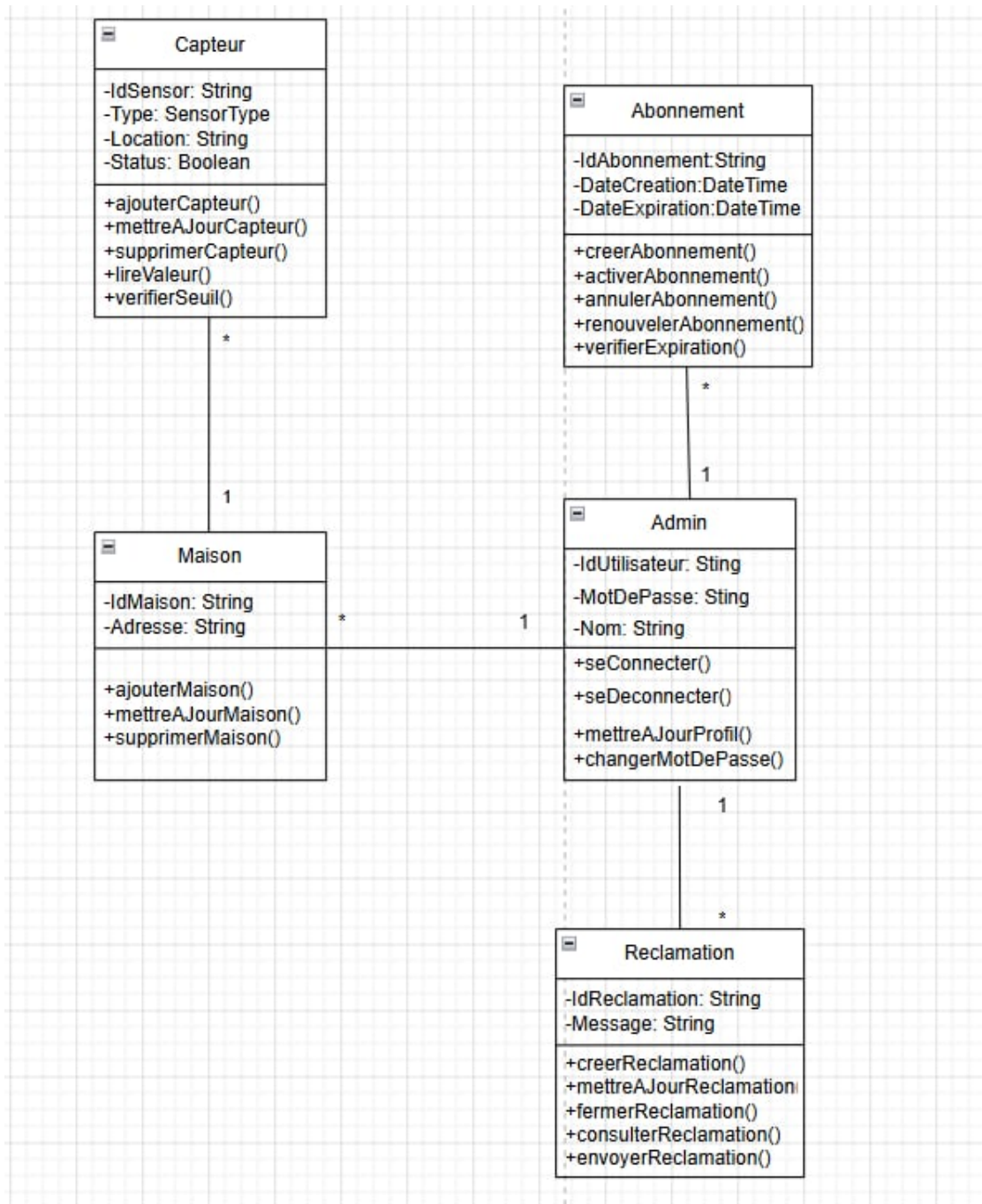


FIGURE 7.2 – Diagramme de classes du Sprint 5

Le diagramme de classes ci-dessus illustre les diverses entités intervenant dans l'administration du système Smart House. La classe **Administrateur** incarne la personne en charge de la surveillance de la plateforme et de la coordination des tâches administratives. Elle sauvegarde les données d'identification et les fonctions relatives à la connexion, à la déconnexion et à la modification du profil. La classe **Abonnement** sert à administrer les détails des souscriptions des utilisateurs. Elle regroupe divers champs tels que le code de l'abonnement, sa date de commencement et sa date d'échéance. Cette classe propose aussi diverses actions pour établir, confirmer, rejeter ou reconduire

un abonnement, ainsi que pour en contrôler la validité. Le schéma inclut également la classe ****Réclamation****, employée pour traiter les difficultés signalées par les habitants. Cette classe contient les informations sur les réclamations, comme leur référence et le message transmis. Elle autorise par ailleurs la réalisation de multiples actions comme l'enregistrement, la lecture, la modification et le clôturage des réclamations, afin de garantir un suivi précis des événements. La classe ****Maison**** représente les propriétés connectées recensées dans le système. Chaque propriété dispose d'un identifiant et d'une adresse, et peut être liée à de nombreux dispositifs de captation IoT. Cette classe propose diverses fonctionnalités pour inclure, corriger ou retirer une propriété de la plateforme. Pour conclure, la classe ****Capteur**** décrit les divers dispositifs de détection installés dans les habitations connectées. Elle rassemble des renseignements comme le genre de capteur, son emplacement ainsi que son statut. Cette classe permet aussi de gérer les actions d'insertion, d'actualisation, de retrait et de contrôle des données issues des capteurs.

7.4.2 Diagramme d'activité

Pour dépeindre plus fidèlement le processus adopté par l'administrateur au sein du système Smart House, une activité dédiée à la gestion des plaintes a été conçue. Cette représentation visuelle expose les diverses phases effectuées par le système, en commençant par la vérification de l'identité de l'administrateur pour aboutir à la résolution finale des doléances et à la transmission des réponses aux résidents.

La figure suivante présente le diagramme d'activité relatif au traitement des réclamations dans l'espace administrateur.

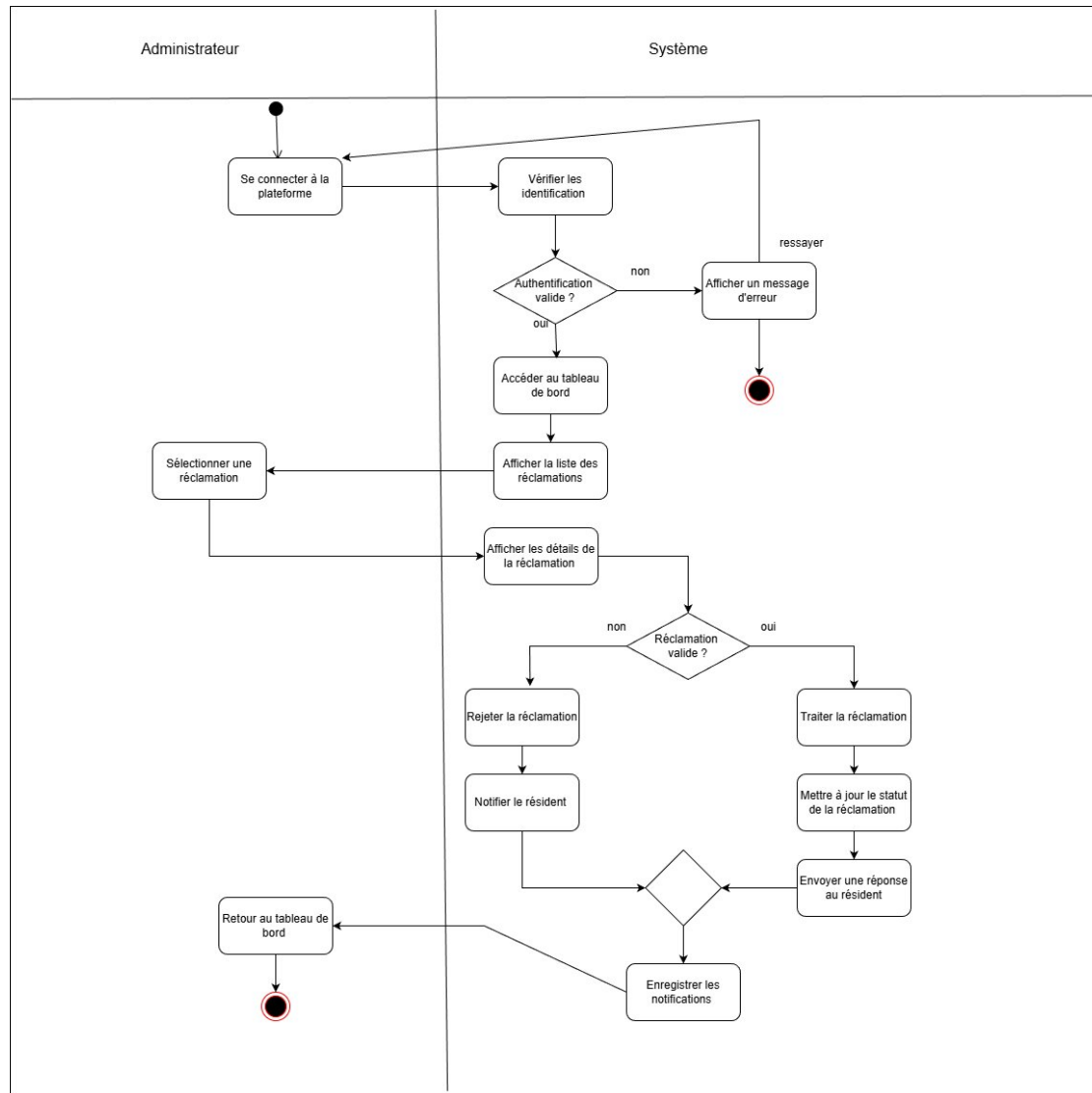


FIGURE 7.3 – Diagramme d'activité — Traitement des réclamations

Le diagramme d'activité ci-dessous dépeint l'intégralité de la procédure de traitement des litiges au sein de la solution Smart House. Le déroulement commence avec la saisie des identifiants de l'administrateur pour se connecter à l'interface de gestion. La plateforme contrôle alors les données soumises. Si la connexion n'aboutit pas, un message d'alerte apparaît. Dans le cas contraire, l'administrateur accède à l'espace de contrôle et examine la liste des mécontentements enregistrés. Une fois les différents litiges examinés, l'administrateur choisit un cas pour en voir les précisions. Le système vérifie alors si le litige a déjà été clos. S'il n'est pas encore pris en charge, l'administrateur peut réagir au litige et informer l'occupant intéressé. À l'inverse, si le litige est clos, le système met à jour sa situation de manière automatique et transmet une réponse à l'occupant. Pour conclure, toutes les alterations réalisées sont sauvegardées dans le référentiel de données avant la clôture de l'opération. Ce schéma offre ainsi une meilleure appréhension des échanges entre l'administrateur et le système, ainsi que le parcours de prise en charge

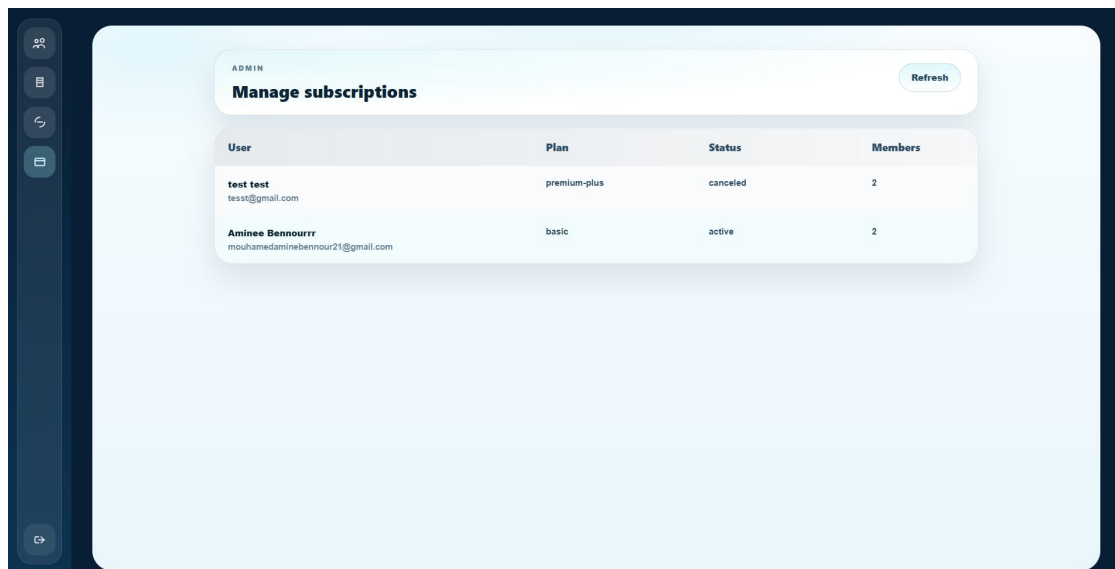
des réclamations sur la plateforme Smart House.

7.5 Réalisation et tests

Cette section présente les différentes fonctionnalités développées dans le cadre du Sprint 5 consacré à l'administration du système Smart House. Elle décrit les aspects techniques liés au backend, au frontend ainsi qu'à la base de données utilisés pour assurer la gestion des abonnements, le traitement des réclamations et la supervision des données des maisons connectées.

7.5.1 Interface de gestion des abonnements

La figure suivante présente l'interface de gestion des abonnements.



User	Plan	Status	Members
test test tess@gmail.com	premium-plus	canceled	2
Aminee Bennourrr mouhamedaminebennour21@gmail.com	basic	active	2

FIGURE 7.4 – Interface de gestion des abonnements

Cette interface donne la possibilité à l'administrateur de voir et d'organiser toutes les souscriptions présentes sur le système. Il propose diverses options qui simplifient la gestion des services offerts aux usagers, particulièrement la possibilité de changer, d'enlever, ou d'activer/désactiver les abonnements selon les nécessités de la plateforme. Les abonnements sont présentés dans une grille interactive regroupant les renseignements essentiels de chaque adhésion. Cette disposition accélère la recherche, la visualisation et la prise en main des informations par l'administrateur. Avec cette interface, l'administrateur peut assurer une surveillance accrue des services mis à disposition par la plateforme Smart House tout en assurant une organisation centrale et performante des adhésions des usagers.

La figure suivante présente l'interface de gestion des abonnements développée dans l'espace administrateur du système Smart House.

7.5.2 Interface de gestion des réclamations

La figure suivante présente l'interface de gestion des réclamations.

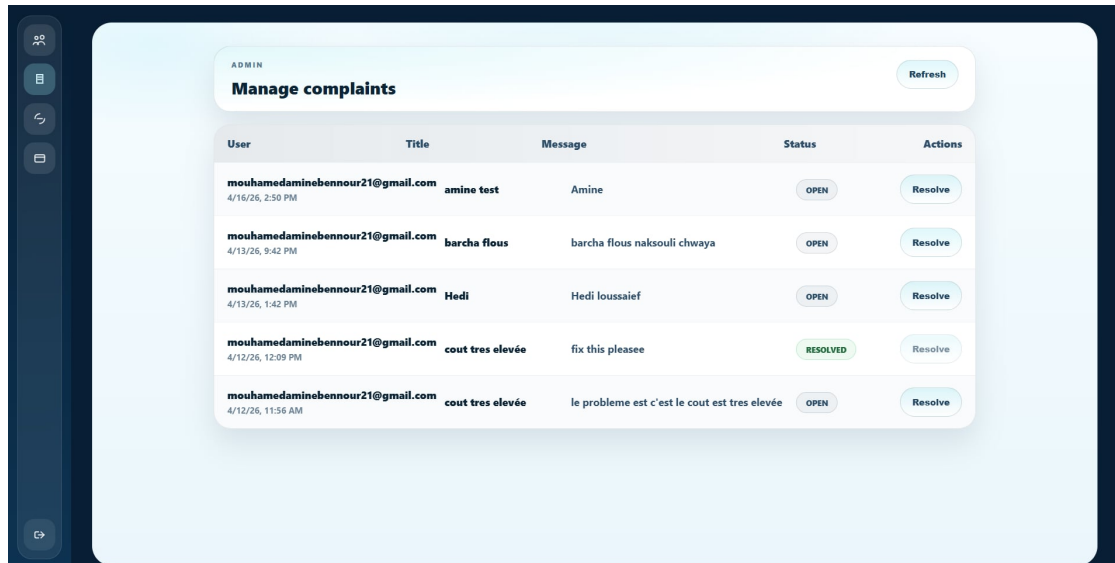


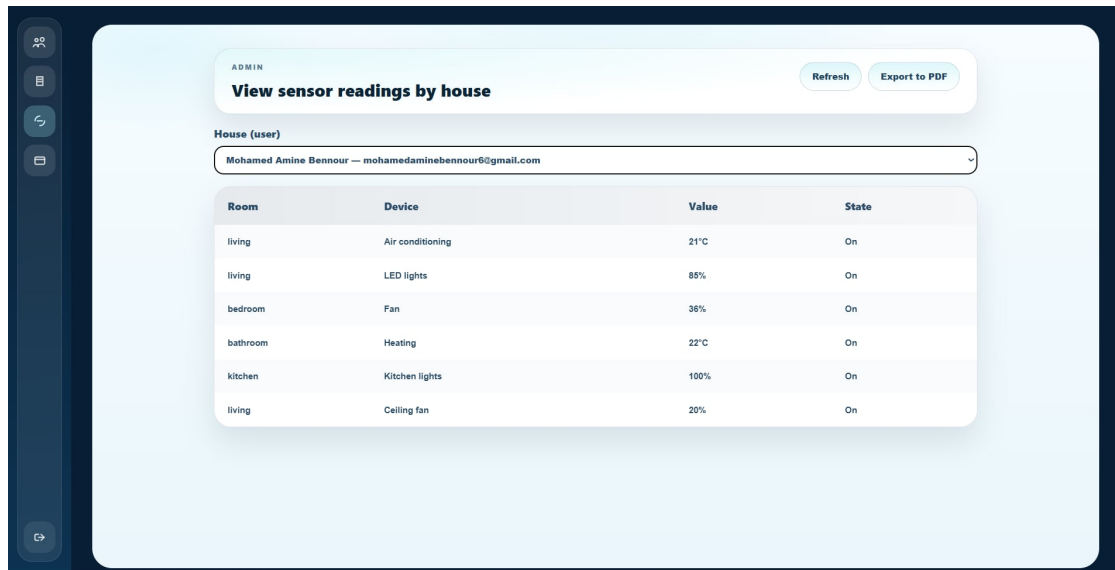
FIGURE 7.5 – Interface de gestion des réclamations

Cette interface permet à l'administrateur de consulter l'ensemble des réclamations envoyées par les résidents de la plateforme. Elle facilite le suivi des problèmes signalés en offrant une vue claire et organisée des différentes réclamations enregistrées dans le système. L'administrateur bénéficie aussi de divers outils pour réagir aux plaintes, ajuster leur statut et s'occuper du traitement des problèmes rapportés par les usagers. Les plaintes sont présentées dans une grille interactive pour simplifier la lecture rapide des données et optimiser la gestion des requêtes. Grâce à ce pupitre de commande, l'administrateur peut garantir un suivi approfondi des plaintes et renforcer les échanges avec les occupants tout en assurant une gestion performante des incidents sur le réseau Smart House.

En plus du traitement des réclamations, l'administrateur peut superviser les données des maisons connectées.

7.5.3 Interface de consultation des capteurs

La figure suivante présente l'interface de consultation des données des capteurs du système Smart House.



Room	Device	Value	State
living	Air conditioning	21°C	On
living	LED lights	85%	On
bedroom	Fan	36%	On
bathroom	Heating	22°C	On
kitchen	Kitchen lights	100%	On
living	Ceiling fan	20%	On

FIGURE 7.6 – Interface de consultation des données des capteurs

Cette solution d'interface donne la possibilité à l'administrateur de garder un œil sur les divers renseignements recueillis par les capteurs IOT implantés dans les demeures intelligentes. Elle procure une présentation globale des informations relatives à l'environnement dans le but de garantir une supervision performante de l'écosystème de la Maison Connectée. L'interface met en avant plusieurs indicateurs cruciaux comme la température, le taux d'humidité, les quantités de gaz ainsi que les détections de déplacement répertoriées par les capteurs. Ces éléments autorisent l'administrateur à suivre en direct l'état des habitations intelligentes et à déceler promptement toute irrégularité potentielle. En outre, l'interface comprend des représentations visuelles interactives qui simplifient la lecture et l'examen des données issues des capteurs en temps réel. Cette projection graphique optimise la saisie des fluctuations des mesures et concourt à une meilleure surveillance des appareils communicants au sein de la plateforme de la Maison Intelligente.

7.5.4 Interface de gestion des utilisateurs

La figure suivante présente l'interface de gestion des utilisateurs développée dans l'espace administrateur du système Smart House.

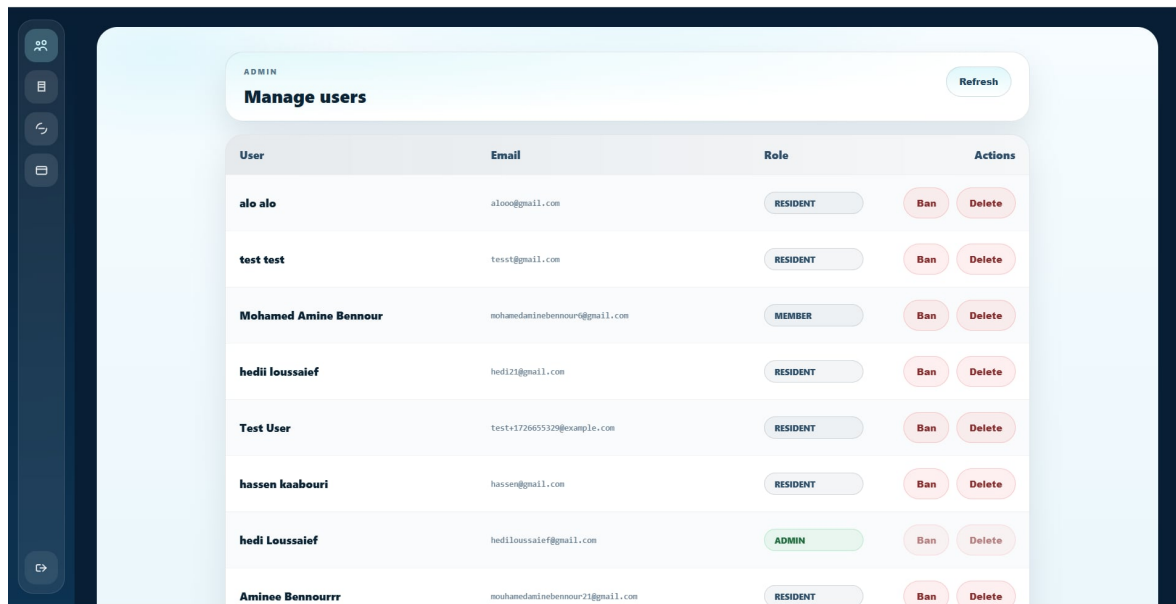


FIGURE 7.7 – Interface de gestion des utilisateurs

Ce tableau de bord donne la possibilité à l'administrateur de surveiller et de maîtriser tous les membres inscrits sur la plateforme Smart House. Il propose une visualisation consolidée des profils avec leurs données clés, notamment le nom, l'adresse mail, ainsi que le statut assigné dans le système. Ce panneau offre aussi une gestion des permissions simplifiée via les diverses options de manipulation à disposition de l'administrateur. Ce dernier peut mener à bien plusieurs actions comme le blocage passager d'un membre ou la radiation complète de son profil pour sauvegarder la sûreté et la bonne marche de la plateforme. L'illustration ci-dessus met en scène l'agencement des profils sous forme de grille interactive où chaque ligne présente un membre avec ses informations essentielles et son rôle au sein du système, comme ADMIN, RESIDENT ou MEMBER. Des commandes d'exécution sont aussi offertes pour permettre à l'administrateur de réaliser prestement les différentes opérations de contrôle, en particulier le bannissement (Ban) et la suppression (Delete) des profils. Un bouton de réactualisation (Refresh) donne en outre la possibilité d'actualiser en direct la liste des membres exposés dans le panneau. Par le biais de ce panneau, l'administrateur peut garantir une administration consolidée et sûre des membres tout en optimisant la maîtrise des accès au sein du système Smart House.

7.6 Tests fonctionnels

Afin de valider le bon fonctionnement des différentes fonctionnalités développées dans le cadre du Sprint 5, plusieurs tests fonctionnels ont été réalisés sur les principales opérations de l'espace administrateur du système Smart House.

TABLE 7.4 – Tests fonctionnels du Sprint 5

Fonctionnalité	Résultat
Ajouter un abonnement	Succès
Supprimer un abonnement	Succès
Traiter une réclamation	Succès
Afficher les données des capteurs	Succès

Les essais menés portent essentiellement sur le management des adhésions, le traitement des plaintes et la visualisation des informations des dispositifs connectés. Chaque fonction a été mise à l'épreuve en usage courant afin de s'assurer du bon déroulement des actions et de la justesse des observations. Les constats rapportés dans le tableau susmentionné attestent que toutes les fonctions passées en revue ont été réalisées avec succès. Les actions d'ajout et de retrait des adhésions se déroulent bien, le traitement des plaintes est réalisé efficacement et les informations des dispositifs sont visibles correctement dans la console de gestion. Ces essais confirment donc la robustesse des fonctions conçues lors de la cinquième itération, ainsi que leur adéquation avec les exigences établies dans le carnet de produit Smart House.

Conclusion

Ce chapitre aborde le Sprint 5, focalisé sur la gestion de notre système Smart House. Au cours de ce sprint, nous avons créé diverses fonctionnalités pour les administrateurs, incluant la gestion des souscriptions, le traitement des requêtes, le suivi des usagers, et la visualisation des informations provenant des capteurs IoT des domiciles connectés. Nous avons de même exposé les divers schémas d'analyse et de conception élaborés durant cette période, ainsi que les interfaces mises au point pour garantir une administration uniforme et performante de la plateforme. Pour conclure, les essais fonctionnels réalisés ont validé l'opérationnalité des différentes fonctions intégrées et ont contribué à améliorer la surveillance générale de l'écosystème Smart House.

Conclusion générale

Au terme de ce projet de fin d'études, nous avons réussi à concevoir et développer une plateforme Smart House intelligente permettant la supervision et la gestion d'une habitation connectée en temps réel. Ce travail nous a permis de mettre en pratique les connaissances acquises durant notre formation tout en découvrant les défis liés aux systèmes embarqués, à l'Internet des Objets (IoT) et au développement d'applications web modernes.

Le système réalisé repose sur une architecture IoT composée d'une carte ESP32, de plusieurs capteurs intelligents ainsi que d'une application web développée avec Angular et NestJS. Cette solution permet de surveiller différents paramètres environnementaux tels que la température, l'humidité, la détection de gaz, les mouvements et les risques d'incendie. Elle assure également l'envoi de notifications en temps réel, la gestion des utilisateurs, des abonnements et des équipements connectés.

Tout au long des différents sprints, nous avons appliqué le framework Scrum afin d'assurer une organisation efficace du travail et un suivi progressif du développement. Cette approche nous a permis de mieux gérer les tâches du projet, d'améliorer la collaboration au sein de l'équipe et d'adapter progressivement les fonctionnalités selon les besoins identifiés.

La réalisation de ce projet nous a également permis de renforcer nos compétences techniques dans plusieurs domaines, notamment le développement frontend et backend, la gestion des bases de données NoSQL, les protocoles de communication IoT comme MQTT, ainsi que la conception de systèmes intelligents et sécurisés. Les résultats obtenus sont satisfaisants et répondent aux principaux objectifs définis au début du projet.

Le système développé offre une solution moderne, évolutive et accessible pour la gestion d'une maison intelligente tout en garantissant une supervision efficace et une meilleure sécurité des habitations.

Cependant, plusieurs perspectives d'amélioration peuvent être envisagées afin d'enrichir davantage la plateforme. Parmi les évolutions possibles, nous pouvons citer l'intégration de l'intelligence artificielle pour l'analyse prédictive des événements, l'ajout de commandes vocales, le développement d'une application mobile native ainsi que l'intégration de nouveaux équipements connectés et de protocoles de communication plus avancés.

Ainsi, ce projet constitue une base solide pour le développement de futures solutions Smart House toujours plus intelligentes, performantes et adaptées aux besoins des utilisateurs.

Bibliographies/Webographies

- [1] UML. [En ligne]. Consulté le 12/05/2026.
<https://www.lucidchart.com/pages/fr/langage-uml>
- [2] SysML. [En ligne]. Consulté le 12/05/2026.
<https://fr.wikipedia.org/wiki/SysML>
- [3] Angular. [En ligne]. Consulté le 13/05/2026.
<https://angular.dev>
- [4] NestJS. [En ligne]. Consulté le 13/05/2026.
<https://nestjs.com>
- [5] MongoDB. [En ligne]. Consulté le 14/05/2026.
<https://www.mongodb.com>
- [6] MQTT Protocol. [En ligne]. Consulté le 14/05/2026.
<https://mqtt.org>
- [7] ESP32. [En ligne]. Consulté le 15/05/2026.
<https://www.espressif.com/en/products/socs/esp32>
- [8] ESP32-CAM. [En ligne]. Consulté le 15/05/2026.
<https://randomnerdtutorials.com/esp32-cam-video-streaming-face-recognition-ardu>
- [9] DHT22 Sensor. [En ligne]. Consulté le 16/05/2026.
<https://components101.com/sensors/dht22-pinout-specs-datasheet>
- [10] MQ2 Gas Sensor. [En ligne]. Consulté le 16/05/2026.
<https://components101.com/sensors/mq2-gas-sensor>
- [11] PIR Motion Sensor. [En ligne]. Consulté le 16/05/2026.
<https://lastminuteengineers.com/pir-sensor-arduino-tutorial>
- [12] Flame Sensor. [En ligne]. Consulté le 17/05/2026.
<https://microcontrollerslab.com/flame-sensor-module-pinout-interfacing-arduino>
- [13] Docker. [En ligne]. Consulté le 17/05/2026.
<https://www.docker.com>
- [14] GitLab. [En ligne]. Consulté le 17/05/2026.
<https://about.gitlab.com>
- [15] Arduino IDE. [En ligne]. Consulté le 18/05/2026.
<https://www.arduino.cc/en/software>
- [16] Visual Studio Code. [En ligne]. Consulté le 18/05/2026.
<https://code.visualstudio.com>

- [17] Postman. [En ligne]. Consulté le 18/05/2026.
<https://www.postman.com>
- [18] Node.js. [En ligne]. Consulté le 19/05/2026.
<https://nodejs.org/en>
- [19] TypeScript. [En ligne]. Consulté le 19/05/2026.
<https://www.typescriptlang.org>
- [20] JWT Authentication. [En ligne]. Consulté le 20/05/2026.
<https://jwt.io/introduction>
- [21] Framework Scrum. [En ligne]. Consulté le 20/05/2026.
<https://www.scrum.org/resources/what-is-scrum>
- [22] Internet of Things (IoT). [En ligne]. Consulté le 21/05/2026.
<https://aws.amazon.com/what-is/iot>
- [23] Smart House Systems. [En ligne]. Consulté le 21/05/2026.
<https://www.techtarget.com/iotagenda/definition/smart-home-or-building>